

Trabajo de Fin de Grado

Diseño y desarrollo de un sistema empotrado basado en bus CAN con acceso inalámbrico para la monitorización de un vehículo automóvil

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Gorka Eymard Santana Cabrera

TUTORIZADO POR:

Antonio Carlos Domínguez Brito, Jorge Cabrera Gámez

Fecha [03/2026]

SOLICITUD DE DEFENSA DE TRABAJO DE FIN DE TÍTULO

D./D^a Gorka Eymard Santana Cabrera, autor del trabajo Diseño y desarrollo de un sistema empujado basado en bus CAN con acceso inalámbrico para la monitorización de un vehículo automóvil, correspondiente a la titulación Grado de Ingeniería Informática.

SOLICITA

que se inicie el procedimiento de defensa del mismo, para lo que se adjunta la documentación requerida, haciendo constar que

☒ se autoriza / ☐ no se autoriza la grabación en audio de la exposición y turno de preguntas.

Asimismo, con respecto al registro de la propiedad intelectual/industrial del TFT, declara que:

☐ Se ha iniciado o hay intención de iniciarlo (defensa no pública).

☐ No está previsto.

Y para que así conste firma la presente. (fecha en firma electrónica)

El/La estudiante

Fdo. _____

A rellenar y firmar **obligatoriamente** por el/la/los/las tutores

En relación con la presente solicitud, se informa: (firmar donde corresponda)

Positivamente (en caso de detección de copia, esta firma quedará invalidada)

Fdo. _____

Negativamente (justificación en TFT05)

Fdo. _____

DIRECTOR DE LA ESCUELA DE INGENIERÍA INFORMÁTICA

Agradecimientos

La realización de este Trabajo de Fin de Título no habría sido posible sin el apoyo y la colaboración de diversas personas e instituciones a las que deseo expresar mi más sincero agradecimiento.

En primer lugar, quiero agradecer a los profesores Antonio C. Domínguez Brito y Jorge Cabrera Gámez, por su dedicación, orientación y apoyo durante el desarrollo de este trabajo. Su experiencia y los consejos recibidos han sido fundamentales para encauzar el desarrollo de este proyecto en la dirección correcta.

De igual modo, deseo expresar mi gratitud a la División de Robótica y Oceanografía Computacional del Instituto Universitario SIANI de la Universidad de Las Palmas de Gran Canaria, por poner a disposición de este proyecto los medios materiales y el equipamiento necesarios para llevar a cabo el desarrollo experimental y las pruebas del sistema.

Por último, agradezco a todas las personas que, de una forma u otra, han brindado su apoyo durante este período, haciendo posible la finalización de este trabajo.

Resumen

El presente Trabajo de Fin de Título (TFT) desarrolla un sistema de diagnóstico de vehículos basado en bus CAN. El sistema implementa los protocolos ISO-TP (ISO 15765-2) como capa de transporte, OBD-II (SAE J1979) y UDS (ISO 14229-1) como capas de aplicación, ejecutándose sobre una Raspberry Pi 4 mediante scripts Python. La sesión de diagnóstico se expone de forma inalámbrica a través de un servidor BLE con perfil GATT, accesible desde una aplicación móvil nativa compatible con Android e iOS que ofrece monitorización en tiempo real, escaneo de DTCs y visualización de logs. El sistema ha sido validado sobre un banco de pruebas basado en un simulador de ECU implementado en un Arduino MKRWAN 1310 y, adicionalmente, sobre un vehículo real del grupo VAG (un Škoda Yeti de 2015). El diseño toma como referencia la arquitectura de comunicaciones del grupo VAG (Volkswagen Audi Group).

Abstract

This Final Degree Project (TFT) develops a vehicle diagnostics system based on the CAN bus. The system implements ISO-TP (ISO 15765-2) as the transport layer, and OBD-II (SAE J1979) and UDS (ISO 14229-1) as the application layers, running on a Raspberry Pi 4 via Python scripts. The diagnostic session is exposed wirelessly through a BLE server with a GATT profile, accessible from a native mobile application compatible with Android and iOS, offering real-time monitoring, DTC scanning, and log visualization. The system has been validated on a test bench based on an ECU simulator implemented on an Arduino MKRWAN 1310 and, additionally, on a real VAG-group vehicle (a 2015 Škoda Yeti). The design uses the communication architecture of Volkswagen Audi Group (VAG) vehicles as reference.

Índice general

Listings	V
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del sistema desarrollado	2
1.3. Motivación	2
1.4. Estructura del documento	3
2. Estado actual y objetivos	4
2.1. Estado del Arte	4
2.1.1. CAN - Controller Area Network (ISO 11898)	4
2.1.2. ISO-TP - Transport Protocol (ISO 15765-2)	7
2.1.3. OBD-II - On-Board Diagnostics (SAE J1979)	8
2.1.4. UDS - Unified Diagnostic Services (ISO 14229-1)	12
2.1.5. BLE GATT y Nordic UART Service	14
2.2. Objetivos	17
2.2.1. Objetivo general	17
2.2.2. Objetivos específicos	17
2.2.3. Alcance y limitaciones	18
3. Aportaciones del trabajo	20
3.1. Principales aportaciones	20
3.2. Competencias específicas desarrolladas	21
3.3. Relación con los Objetivos de Desarrollo Sostenible	22
4. Metodología de desarrollo	24
4.1. Metodología de desarrollo	24
4.2. Fases y tareas	24
4.2.1. Fase 1 — Análisis de requisitos (50 horas estimadas)	24
4.2.2. Fase 2 — Diseño rápido y construcción (75 horas estimadas)	25
4.2.3. Fase 3 — Evaluación, refinamiento e implementación (125 horas estimadas)	25
4.2.4. Fase 4 — Documentación y presentación (50 horas estimadas)	26
4.3. Desviaciones respecto a la planificación inicial	26

5. Desarrollo del sistema	27
5.1. Arquitectura general del sistema	27
5.2. Dependencias y entorno de desarrollo	29
5.3. Proceso de desarrollo iterativo	30
5.3.1. Fase 1: Validación del enlace CAN	30
5.3.2. Fase 2: Primer prototipo — simulador ECU y CLI de diagnóstico . .	31
5.3.3. Fase 3: Simulación avanzada, monitorización y persistencia	33
5.3.4. Fase 4: Integración BLE y validación en vehículo real	37
5.4. Simulador de ECU — Arduino MKR	40
5.4.1. Hardware y configuración del bus CAN	40
5.4.2. Modelo físico del motor	41
5.4.3. Implementación de los modos OBD-II y servicios UDS	41
5.4.4. Mecanismos opcionales de variabilidad del bus	43
5.4.5. Inyección de DTCs por interrupción hardware	44
5.5. Herramienta de diagnóstico — Raspberry Pi	44
5.5.1. Arquitectura software	45
5.5.2. Capa de transporte CAN e ISO-TP	45
5.5.3. Módulo de logging en SQLite	46
5.5.4. Monitor de datos en tiempo real	47
5.5.5. Servidor BLE GATT	47
5.6. Aplicación móvil — React Native	48
5.6.1. Arquitectura de la aplicación	49
5.6.2. Pantallas y funcionalidades	49
5.6.3. Comunicación BLE	50
5.7. Integración y pruebas	51
5.7.1. Escenarios de prueba	52
5.7.2. Resultados de las pruebas de integración	52
6. Resultados	54
6.1. Resultados obtenidos	54
6.2. Validación sobre vehículo real	55
6.3. Validación funcional del protocolo OBD-II	57
6.4. Métricas de rendimiento de la comunicación	57
6.5. Comportamiento ante condiciones de bus realistas y errores	58
6.6. Interfaz de la aplicación móvil	59
6.7. Grado de consecución de los objetivos	63
7. Conclusiones y trabajo futuro	65
7.1. Conclusiones	65
7.2. Líneas de trabajo futuro	66
7.3. Reflexión sobre el uso de herramientas de inteligencia artificial	66
A. Tabla completa de PIDs OBD-II Modo 0x01	68
B. Catálogo de comandos del protocolo JSON sobre NUS	76

Listings

5.1. Configuración de la pila ISO-TP en IsoTpTransport 33

Índice de algoritmos

1.	Transmisión ISO-TP multi-frame del VIN en el simulador	43
2.	Ciclo de consulta de un PID OBD-II (Modo 0x01)	46
3.	Descubrimiento de PIDs soportados (probe_pids)	48
4.	Reensamblado NDJSON y emparejamiento petición-respuesta	51

Índice de figuras

2.1. Secuencia de intercambio ISO-TP para la obtención del VIN (20 bytes: modo 0x49, PID 0x02, contador 0x01 y 17 bytes ASCII). El escáner envía una petición Single Frame, la ECU responde con First Frame + Flow Control (BS=0, STmin=0) + dos Consecutive Frames.	9
2.2. Pila de protocolos del Nordic UART Service (NUS) sobre BLE. Los comandos y respuestas JSON se transmiten como payload ATT sobre las características RX (escritura) y TX (notificación) del servicio NUS, con un MTU negociado de 240 bytes.	15
2.3. Flujo completo de una conexión BLE GATT con el servicio NUS. Tras el descubrimiento y negociación de MTU, la app habilita las notificaciones en la característica TX y comienza el intercambio de comandos JSON. Un mecanismo de keepalive mantiene la conexión activa; la inactividad superior a 20 segundos provoca el cierre de sesión.	16
5.1. Arquitectura general del sistema de diagnóstico del vehículo.	28
5.2. Arduino IDE — monitor serie durante la fase de validación CAN. Se aprecia la confirmación de inicialización del bus ([OK] CAN Bus at 500 kbps) y el log de cada trama enviada.	30
5.3. Sesión SSH en la Raspberry Pi mostrando la salida de <code>candump can0</code> y una petición manual con <code>cansend</code> durante la validación del enlace CAN.	30
5.4. Monitor serie del Arduino durante la Fase 2: cada petición recibida ([RX]) y su respuesta ([TX SF]) se registran indicando modo, PID y bytes de datos.	31
5.5. CLI de diagnóstico (<code>cli.py</code>) en la Fase 2: menú interactivo con opciones de live data y lectura de DTCs, ejecutado en la Raspberry Pi vía SSH.	32
5.6. Monitor serie del Arduino en la Fase 3: se aprecian los eventos de ignición ([IGN] Key ON - engine starting) y la inyección de DTCs por interrupción hardware ([DTC] Fault injected).	34
5.7. CLI en modo de monitorización continua (opción 6): actualización periódica de los cinco PIDs principales con timestamp, valores decodificados y unidades, ejecutado vía SSH en la Raspberry Pi.	36
5.8. Consola de la Raspberry Pi ejecutando el servidor BLE: anuncio del servicio <code>diag_tool</code> y registro de los comandos JSON recibidos (RX) y respuestas enviadas (TX).	38
5.9. <i>Conexión</i> (izquierda) y <i>Panel</i> (derecha)	39

5.10. <i>Averías</i> (izquierda) y <i>Sesiones</i> (derecha)	40
6.1. Herramienta comercial de referencia (Car Scanner ELM OBD2 [1]) usada como patrón de comparación: cuadro de instrumentos (a) y lista de DTCs (b). . .	55
6.2. Sistema montado en el vehículo real: cable OBD-II conectado al puerto de diagnóstico, Raspberry Pi con el servidor activo y teléfono móvil con la aplicación mostrando datos de telemetría en tiempo real.	56
6.3. Traza de ejecución bajo condiciones de bus realistas: consola del simulador Arduino emitiendo tramas de difusión y ruido (a) y consola de la Raspberry Pi filtrando y decodificando las respuestas válidas (b).	59
6.4. Pantalla <i>Panel</i> : telemetría en tiempo real (izquierda) y pestaña UDS con control de sesión y lectura de DIDs (derecha).	60
6.5. Pantallas <i>Averías</i> (izquierda), con la lista de DTCs leídos de la ECU, y <i>Sesiones</i> (derecha), con el historial de sesiones almacenado en SQLite.	61
6.6. Detalle de una sesión con las muestras agrupadas por sensor (izquierda) y pantalla <i>Consola</i> con el envío de comandos JSON y el registro de comunicación BLE (derecha).	62
6.7. Pantalla <i>Ajustes</i> (izquierda), con la gestión de conexión y el modo de simulación, y la pantalla de ajustes (derecha).	63

Índice de cuadros

2.1.	Estructura de la trama CAN 2.0A (<i>Standard Frame Format</i>) según ISO 11898-1 [2].	5
2.2.	Estructura de los cuatro tipos de trama ISO-TP según ISO 15765-2 [3]. Los bytes de <i>padding</i> (0xAA) en FC son ignorados por el receptor. Leyenda de campos bajo la tabla.	8
2.3.	Distribución de bits de los 2 bytes de PCI de una <i>First Frame</i> ISO-TP según ISO 15765-2 [3]. El nibble 0x1 identifica el tipo de trama y los 12 bits restantes codifican FF_DL (longitud total del mensaje).	9
2.4.	Estructura de trama OBD-II: petición (CAN ID 0x7E0) y respuesta (CAN ID 0x7E8) para el PID 0x0C (RPM) según SAE J1979 [4].	10
2.5.	Trama CAN completa recibida en una respuesta negativa UDS/OBD-II: encapsulado ISO-TP (<i>Single Frame</i>) del payload <i>Negative Response</i> según ISO 14229-1 [5] e ISO 15765-2 [3].	12
2.6.	Principales códigos NRC (<i>Negative Response Code</i>) de ISO 14229-1 [5] empleados en respuestas negativas OBD-II y UDS.	13
2.7.	Estructura genérica de los mensajes NDJSON del protocolo sobre NUS. Cada objeto se serializa en una línea ASCII terminada en \n.	17
2.8.	Trazabilidad entre funciones definidas en la propuesta TFT01 y objetivos específicos del trabajo.	18
3.1.	Grado de relación de este trabajo con los 17 Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030.	22
6.1.	PIDs del modo 0x01 validados en el banco de ensayo.	57
6.2.	Latencia extremo a extremo de una consulta OBD-II (100 muestras, bus en modo determinista).	58
6.3.	Evaluación del grado de consecución de los objetivos específicos.	64
A.1.	Tabla completa de PIDs del Modo 0x01 OBD-II (<i>Show current data</i>) según SAE J1979 [4]. PIDs marcados con (*): obligatorios. A, B, C, D: bytes de respuesta en orden.	68
B.1.	Comandos del protocolo JSON sobre NUS.	77

Capítulo 1

Introducción

En este capítulo se realizará una introducción general al contexto del sector automotriz en el que se plantea el trabajo realizado y el motivo de porque se ha realizado este trabajo.

1.1. Contexto y motivación

El sector automovilístico ha seguido una tendencia hacia una sensorización masiva que conlleva la comunicación de datos en tiempo real, impulsada por la necesidad de monitorizar el estado del vehículo y para anticipar o detectar posibles fallos en el estado general del vehículo. Esta tendencia ha consistido en la incorporación de sensores cada vez más sofisticados, tales como sensores de presión, temperatura, posición. Estos generan una enorme cantidad de datos que deben ser procesados y transmitidos entre las distintas Unidades de Control Electrónico (ECU, del inglés *Electronic Control Unit*) que controlan el funcionamiento del vehículo además estos datos han de ser tratados según su prioridad y criticidad, requiriendo de la adopción de protocolos de comunicación robustos y eficientes.

Para lograr una comunicación fiable, en tiempo real y con gestión de prioridades los fabricantes comenzaron a implementar el bus CAN (*Controller Area Network*), un protocolo de comunicaciones serie desarrollado en Robert Bosch GmbH a partir de 1983, publicado en 1986 y estandarizado bajo la norma ISO 11898. Esencialmente el bus CAN consiste en un protocolo que permite la comunicación de múltiples microcontroladores compartiendo un mismo medio y resolviendo los conflictos de transmisión mediante un mecanismo de prioridades que se basa en el ID del mensaje siendo el bit 0 el dominante. Un ejemplo de la gestión de prioridades podría ser: un coche que va a 120 km/h y el conductor pisa el freno a fondo, esto provoca que el sensor antibloques de los frenos(ABS) envíe un mensaje a la ECU del motor para indicar que la ruedas se están bloqueando, sin embargo al mismo tiempo la ECU envía al tablero de instrumentos las revoluciones por minuto(RPM) pero su ID tiene un número mayor con lo que se quedará en silencio y cuando el ABS termine su transmisión la ECU enviará las RPM nuevamente.

Por otra parte, en 1996 se introdujo el estándar OBD-II (*On-Board Diagnostics, segunda generación*), que define un conector físico de 16 pines situado en el habitáculo(J1962), un

conjunto de protocolos de comunicación y una batería de códigos de error comunes (DTCs - *Diagnostic Trouble Code*). Con todo esto también indica que sensores deben vigilar el coche continuamente permitiendo la monitorización y evaluación del estado de los microcontroladores integrados en el vehículo.

1.2. Descripción del sistema desarrollado

Este proyecto se ha centrado en el diseño y desarrollo de un sistema de diagnóstico de vehículos que usan el protocolo OBD-II sobre bus CAN y que ofrece acceso inalámbrico usando Bluetooth Low Energy (BLE). Durante el desarrollo se han implementado tres componentes principales:

El componente principal es la **herramienta de diagnóstico**, creada sobre una Raspberry Pi 4 conectada al bus CAN usando el CAN HAT rs485 que cuenta con el controlador MCP2515. Esta herramienta implementa los siguientes protocolos en las distintas capas:

- **ISO-TP (ISO 15765-2)** como capa de transporte sobre CAN.
- **OBD-II (SAE J1979)** como capa de aplicación.
- **UDS (ISO 14229-1)** como capa de aplicación

La herramienta se encarga tanto de codificar peticiones como de decodificar los parámetros recibidos, almacenandolos en una base de datos SQLite y a su vez transmitiendolos inalámbricamente usando Bluetooth Low Energy (BLE) mediante un servidor GATT que se encuentra montado en la propia Raspberry Pi que implementa el servicio Nordic UART Service (NUS).

El componente desarrollado para realizar pruebas del protocolo y depuración es un **simulador de ECU** implementado sobre una plataforma Arduino MKR WAN 1310 con un shield CAN.

El tercer componente es una **aplicación móvil nativa** desarrollada con React Native y Expo, compatible con Android e iOS, que permite la conexión a la Raspberry Pi vía Bluetooth Low Energy. La aplicación ofrece al usuario una interfaz para la monitorización en tiempo real de los parámetros del vehículo, la consulta y gestión de códigos de avería (DTCs), asimismo presenta una pantalla de logs para depuración que permite al usuario consultar el funcionamiento de la comunicación.

El conjunto del desarrollo de los tres componentes ha resultado en un sistema de diagnóstico OBD-II portable, inalámbrico, validado en el simulador de ECU desarrollado en la placa de Arduino de manera previa a ser usado en un vehículo real.

1.3. Motivación

En el actual plan de estudios del Grado de Ingeniería Informática en la asignatura de Internet de las cosas se estudia el ecosistema de los sistemas empotrados y sus múltiples pro-

tos de comunicación. Dentro de la amplia gama de protocolos de comunicación el estudio del bus CAN y se comentó su uso en el sector automotriz, esto dió lugar a la motivación por la cual se ha desarrollado este Trabajo de Fin de Título en el que se estudia el uso del bus CAN y el protocolo OBD-II y demás protocolos que orquestan la comunicación de información en un vehículo.

Actualmente los automóviles cuentan con un sistema de numerosos microcontroladores que generan una cantidad muy voluminosa de datos de telemetría y además cuentan con el protocolo de diagnóstico abordo (OBD-II). Estos son accesibles a través del puerto OBD que se encuentra normalmente en la parte inferior del volante en el asiento del piloto. Este puerto permite que no solo los talleres y personas especializadas en el mundo de la mecánica y electromecánica, sino que cualquier persona con un "escáner" pueda ver el estado de su coche o pueda diagnosticar que fallo tiene o anticipar posibles fallos.

Actualmente en el mercado han aparecido numerosas soluciones para facilitar el acceso al diagnóstico OBD-II [6]. La mayor parte de estas soluciones emplean el adaptador ELM327, un circuito integrado que actúa como puente entre el bus CAN del vehículo y una interfaz accesible vía Bluetooth o Wi-Fi. Existen numerosas aplicaciones móviles compatibles con estos adaptadores, tales como Torque Pro u OBD Fusion; no obstante, estas soluciones no suelen permitir el registro estructurado de datos para análisis y la mayor parte de ellas introducen latencias significativas, dando lugar a una experiencia de usuario limitada.

1.4. Estructura del documento

El presente documento se organiza en siete capítulos cuyo contenido se resume a continuación.

El **Capítulo 2** describe el estado actual de las tecnologías usadas en el proyecto, recoge la motivación por la cual se ha decidido realizar el trabajo y la definición de los objetivos que se pretenden alcanzar.

El **Capítulo 3** describe las principales aportaciones del trabajo, las competencias específicas del Grado de Ingeniería Informática desarrolladas durante su realización y la relación del proyecto con los Objetivos de Desarrollo Sostenible de la ONU.

El **Capítulo 4** presenta la metodología de desarrollo seguida, basada en prototipos incrementales, detalla las fases y tareas en las que se estructuró el trabajo.

El **Capítulo 5** desarrolla en detalle los distintos prototipos desarrollados.

El **Capítulo 6** se presentan los resultados obtenidos verificados frente a una herramienta de diagnóstico comercial.

El **Capítulo 7** detalla las conclusiones del trabajo, las líneas de trabajo futuro identificadas y el uso de herramientas de inteligencia artificial durante el desarrollo.

Finalmente, el documento incluye la bibliografía de referencia y cuatro anexos con fragmentos representativos del código fuente desarrollado y el esquema de la base de datos.

Capítulo 2

Estado actual y objetivos

Este capítulo expone los protocolos (CAN, OBD-II, ISO-TP, UDS) y herramientas tanto software como hardware (Raspberry Pi) que se usan en el marco en el que se encuentra el trabajo planteado a desarrollar, así como los objetivos definidos para la realización del mismo.

2.1. Estado del Arte

Tal y como se ha comentado en la introducción, la industria automotriz ha evolucionado hacia una sensorización masiva en la que el vehículo moderno cuenta con un complejo ecosistema compuesto por numerosos microcontroladores que producen una enorme cantidad de datos en tiempo real. Muchos de estos componentes son críticos, ya que se encargan de monitorizar módulos de seguridad y sistemas de asistencia a la conducción (ADAS). Con lo que este ecosistema de microcontroladores requiere de una infraestructura de red robusta, capaz de gestionar la criticidad y la priorización de la información.

Para entender adecuadamente el diseño y desarrollo del sistema de diagnóstico desarrollado en este trabajo, se van a estudiar los protocolos de comunicación sobre los que se mueven los datos. Con lo que a continuación en este capítulo se realizará un estudio detallado sobre los distintos estándares, comenzando por la capa física y de enlace del bus CAN (*Controller Area Network*), hasta llegar a los protocolos de la capa de aplicación como el UDS (*Unified Diagnosis Services*) y pasando la capa de transporte ISO-TP.

Posteriormente, se realizará una explicación sobre las plataformas de hardware y las herramientas software empleadas para la implementación y validación del sistema, analizando las arquitecturas de Raspberry Pi y Arduino, así como las características del protocolo inalámbrico *Bluetooth Low Energy* (BLE) utilizado para la transmisión y visualización de los datos en smartphones y tablets.

2.1.1. CAN - Controller Area Network (ISO 11898)

El bus CAN es un protocolo de comunicaciones serie orientado a mensajes, diseñado para comunicación entre múltiples nodos en entornos con alta interferencia electromagnética [2].

Su desarrollo fue iniciado en Robert Bosch GmbH a partir de 1983 y publicado en 1986, siendo estandarizado bajo la norma ISO 11898 [7, 8].

En su variante CAN 2.0A, que es la empleada en este trabajo, cada trama de datos está compuesta por los campos que se recogen en la siguiente tabla 2.1.

Cuadro 2.1: Estructura de la trama CAN 2.0A (*Standard Frame Format*) según ISO 11898-1 [2].

SOF	ID	RTR	IDE	r0	DLC	DATA	CRC	CD	ACK	AD	EOF
1 b	11 b	1 b	1 b	1 b	4 b	0–64 b	15 b	1 b	1 b	1 b	7 b
<i>Arbitration + Control</i>						<i>Datos</i>	<i>CRC</i>		<i>ACK</i>		<i>EOF</i>

El campo **Identificador (ID)** (11 bits en CAN 2.0A) determina tanto la prioridad del mensaje como su propósito lógico [2]. En el contexto OBD-II, el escáner externo emplea el identificador funcional **0x7DF** para direccionamiento broadcast o el identificador **0x7E0** para dirigirse directamente a la ECU, mientras que la ECU responde con **0x7E8** [9].

El mecanismo de arbitraje no destructivo garantiza que, en caso de colisión, el mensaje con identificador de menor valor gana el bus sin que ningún nodo pierda datos [2], pues en caso de colisión se comparan bit a bit los identificadores manteniendo la transmisión del que tenga valor menor (más MSB=0).

El campo **DLC** (*Data Length Code*, 4 bits) indica el número de bytes de datos de la trama, de 0 a 8 [2]. OBD-II utiliza tramas de 8 bytes, rellenando con **0x55** o **0xAA** los bytes no utilizados (*padding*) [9].

En este trabajo, el bus opera a **500 kbps**, velocidad estándar definida por ISO 15765-4 para los sistemas OBD-II modernos en la configuración CAN 11/500 [9].

La interfaz **SocketCAN** de Linux, accesible desde la Raspberry Pi a través del controlador MCP2515 [10, 11], expone el bus CAN como un *socket* de red estándar, lo que permite su uso desde Python mediante la librería `python-can` con una abstracción de alto nivel.

Fundamentos físicos del bus CAN

El bus CAN es un bus serie diferencial de dos hilos: **CAN_H** (*CAN High*) y **CAN_L** (*CAN Low*). La información se codifica en la diferencia de tensión entre ambos conductores, lo que confiere alta inmunidad al ruido electromagnético (EMI), imprescindible en entornos con motores eléctricos, bobinas de encendido y alternadores [12]. El estándar ISO 11898-2 define dos estados lógicos:

- **Dominante** (lógico 0): $\text{CAN_H} \approx 3,5 \text{ V}$ y $\text{CAN_L} \approx 1,5 \text{ V}$; diferencia diferencial $\geq 2 \text{ V}$. Un bit dominante *siempre gana* sobre un bit recesivo en el arbitraje.
- **Recesivo** (lógico 1): $\text{CAN_H} \approx \text{CAN_L} \approx 2,5 \text{ V}$; diferencia diferencial $\approx 0 \text{ V}$. Cede el bus ante cualquier bit dominante.

El bus termina en sus extremos con resistencias de 120Ω para evitar reflexiones de señal [12]. La longitud máxima depende de la velocidad: a 500 kbps (velocidad estándar OBD-II [9]) el bus puede extenderse hasta ≈ 100 m; a 1 Mbps, hasta ≈ 40 m.

Arbitraje no destructivo CSMA/CR

El bus CAN implementa **CSMA/CR** (*Carrier Sense Multiple Access / Collision Resolution*), también denominado arbitraje no destructivo por dominancia de bit [2]. A diferencia de Ethernet (CSMA/CD), en CAN ninguna trama se destruye durante el arbitraje. El mecanismo opera de la siguiente forma:

1. Antes de transmitir, cada nodo escucha el bus; si detecta actividad, espera a que quede libre.
2. Cuando dos o más nodos comienzan a transmitir simultáneamente, cada uno monitoriza el bus mientras emite su identificador bit a bit, comenzando por el más significativo.
3. Si un nodo transmite un bit **recesivo** (1) pero lee un bit **dominante** (0), otro nodo con identificador de menor valor numérico ha ganado el bus. El nodo perdedor cesa su transmisión *sin haber corrompido la trama ganadora* y reintentará en el siguiente período de interframe.
4. El nodo ganador completa su trama sin interrupción.

Esta propiedad garantiza latencia determinista para los mensajes más prioritarios, propiedad básica para sistemas de seguridad activa como ABS, ESP o airbag [2].

Tipos de trama CAN

El protocolo CAN clásico define cuatro tipos de trama [2]:

- **Data Frame**: transporta hasta 8 bytes de payload. Es la trama ordinaria; en OBD-II todas las peticiones y respuestas son Data Frames.
- **Remote Frame**: solicita al nodo propietario de un identificador que emita la Data Frame correspondiente ($RTR = 1$). Rara vez usada en sistemas modernos.
- **Error Frame**: emitida por cualquier nodo que detecta un error. Compuesta por un *Error Flag* (6 bits dominantes) y un *Error Delimiter* (8 bits recesivos). Destruye la trama errónea y fuerza la retransmisión.
- **Overload Frame**: señala que un nodo necesita tiempo adicional de procesamiento. Estructuralmente similar al Error Frame pero emitida en el período de interframe.

Mecanismos de detección de errores

CAN implementa cinco mecanismos ortogonales de detección de error [2]:

1. **CRC-15**: CRC de 15 bits sobre SOF, arbitración, control y datos, con polinomio $x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$. Los receptores recalculan y comparan.

2. **Bit Monitoring:** el transmisor lee el bus mientras transmite; si detecta discrepancia fuera del campo de arbitraje, señala error inmediatamente.
3. **Bit Stuffing:** tras 5 bits consecutivos del mismo nivel lógico, el transmisor inserta un *stuff bit* del nivel opuesto. Si el receptor detecta 6 bits consecutivos iguales, señala error de stuffing.
4. **Frame Check:** verifica que los campos fijos (CRC Delimiter, ACK Delimiter, EOF) tienen el nivel lógico definido por el estándar.
5. **ACK Check:** el transmisor verifica que al menos un receptor ha confirmado la recepción correcta escribiendo dominante en el ACK Slot.

Variantes CAN 2.0A y CAN 2.0B

- **CAN 2.0A** (*Standard Frame Format*) utiliza un identificador de 11 bits ($2^{11} = 2048$ identificadores). Es la variante empleada en este trabajo bajo la configuración CAN 11/500 de ISO 15765-4 [9].
- **CAN 2.0B** (*Extended Frame Format*) amplía el identificador a 29 bits mediante la concatenación de 11 bits de base con 18 bits de extensión, separados por los bits SRR e IDE, alcanzando $2^{29} \approx 537$ millones de identificadores [2].

ISO 15765-4 contempla la configuración CAN 29/500 para fabricantes que emplean identificadores de 29 bits [9]. La coexistencia de ambas variantes en el mismo bus es posible: ante igual base de identificador, la trama estándar (IDE dominante) tiene mayor prioridad que la extendida (IDE recesivo).

2.1.2. ISO-TP - Transport Protocol (ISO 15765-2)

El protocolo ISO-TP, definido en la ISO 15765-2 [3], opera en la capa de transporte, entre la capa de enlace CAN y la capa de aplicación OBD-II. Su función es segmentar y reensamblar mensajes en tramas CAN cuyo campo de datos está limitado a 8 bytes. El campo *First Frame Data Length* (FF_DL) de 12 bits limita los mensajes a un máximo de 4095 bytes. ISO 15765-2:2016 introduce además una secuencia de escape (FF_DL = 0x000 seguido de una longitud de 4 bytes) para mensajes más largos, aunque en el contexto OBD-II el límite de 4095 bytes [3] es suficiente.

ISO-TP define cuatro tipos de trama, identificados por el nibble más significativo del primer byte de datos (campo PCI, *Protocol Control Information*), tal como se muestra en la tabla 2.2.

N	longitud <i>payload</i> (SF, 1–7) o número de secuencia módulo 16 (CF, 0x1–0xF → 0x0)
H:LL	longitud total del mensaje en 12 bits (FF, máx. 4095 bytes)
FS	<i>Flow Status</i> : 0 = <i>ContinueSending</i> , 1 = <i>Wait</i> , 2 = <i>Overflow</i>
BS	<i>Block Size</i> : CFs antes del siguiente FC (0 = sin límite)
STmin	separación mínima entre CFs: 0x00–0x7F = 0–127 ms; 0xF1–0xF9 = 100–900 μ s

Para mensajes de hasta 7 bytes de datos útiles (suficiente para la mayor parte de peticiones OBD-II en modo 0x01), se utiliza el **Single Frame** (SF). Su byte PCI tiene el formato 0x0N,

Cuadro 2.2: Estructura de los cuatro tipos de trama ISO-TP según ISO 15765-2 [3]. Los bytes de *padding* (0xAA) en FC son ignorados por el receptor. Leyenda de campos bajo la tabla.

Tipo	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
SF <i>Single Frame</i>	0x0N	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
FF <i>First Frame</i>	0x1H	0xLL	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆
CF <i>Consecutive Frame</i>	0x2N	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
FC <i>Flow Control</i>	0x3{FS}	BS	STmin	0xAA	0xAA	0xAA	0xAA	0xAA

donde $N \in \{1 \dots 7\}$ e indica la longitud del *payload*, seguido directamente por los bytes de datos [3].

Para mensajes más largos, como la respuesta al PID (*Parameter Identifier*) 0x09 0x02 (VIN, 20 bytes en total), el flujo de transmisión multi-frame ilustrado en la figura 2.1 sigue la secuencia:

1. El transmisor envía una **First Frame** (FF): los dos primeros bytes de PCI codifican el nibble 0x1 y la longitud total en 12 bits (FF_DL), seguidos de los primeros 6 bytes de datos [3]. La distribución a nivel de bit de estos dos bytes de PCI se detalla en la tabla 2.3.
2. El receptor responde con un **Flow Control** (FC): el nibble inferior del primer byte PCI es el *Flow Status* (FS), que puede valer 0 (*ContinueSending*: el transmisor puede proceder), 1 (*Wait*: el receptor solicita esperar) o 2 (*Overflow*: el receptor no puede aceptar el mensaje). Los bytes siguientes indican el *Block Size* (BS: número máximo de CFs antes del siguiente FC; BS = 0 significa que no se requiere ningún FC adicional) y el *Separation Time minimum* (STmin: 0x00–0x7F = 0–127 ms con resolución de 1 ms; 0xF1–0xF9 = 100–900 μ s con resolución de 100 μ s) [3].
3. El transmisor envía los **Consecutive Frames** (CF) necesarios. Cada CF lleva en el nibble inferior del byte PCI (0x2) un contador de secuencia (SN) que comienza en 1 y se incrementa módulo 16 (es decir, 0xF $\rightarrow$ 0x0) [3]. Si BS > 0, el transmisor pausa tras enviar BS CFs y aguarda un nuevo FC antes de continuar.

La implementación Python utiliza la librería **can-isotp** sobre el socket ISOTP de SocketCAN [13], que gestiona de forma transparente la segmentación, el Flow Control y la temporización, exponiendo al nivel de aplicación una interfaz de lectura/escritura de mensajes completos.

2.1.3. OBD-II - On-Board Diagnostics (SAE J1979)

OBD-II es el estándar de diagnóstico del vehículo, obligatorio en todos los vehículos en Estados Unidos desde el año 1996, que define los servicios y parámetros que toda ECU debe implementar [4, 14]. En la práctica, la comunicación OBD-II se estructura como un protocolo de petición-respuesta a nivel de aplicación, transportado sobre ISO-TP [9].

Cuadro 2.3: Distribución de bits de los 2 bytes de PCI de una *First Frame* ISO-TP según ISO 15765-2 [3]. El nibble 0x1 identifica el tipo de trama y los 12 bits restantes codifican FF_DL (longitud total del mensaje).

First Frame — 2 bytes de PCI (16 bits)	
PCItype	FF_DL
4 bits	12 bits
bits 15–12	bits 11–0
0x1	longitud total del mensaje (máx. 4 095 bytes)
Byte 0 = 0x1H (nibble 0x1 + bits 11–8 de FF_DL); Byte 1 = bits 7–0 de FF_DL.	
Bytes 2–7 de la trama CAN transportan los 6 primeros bytes de datos.	

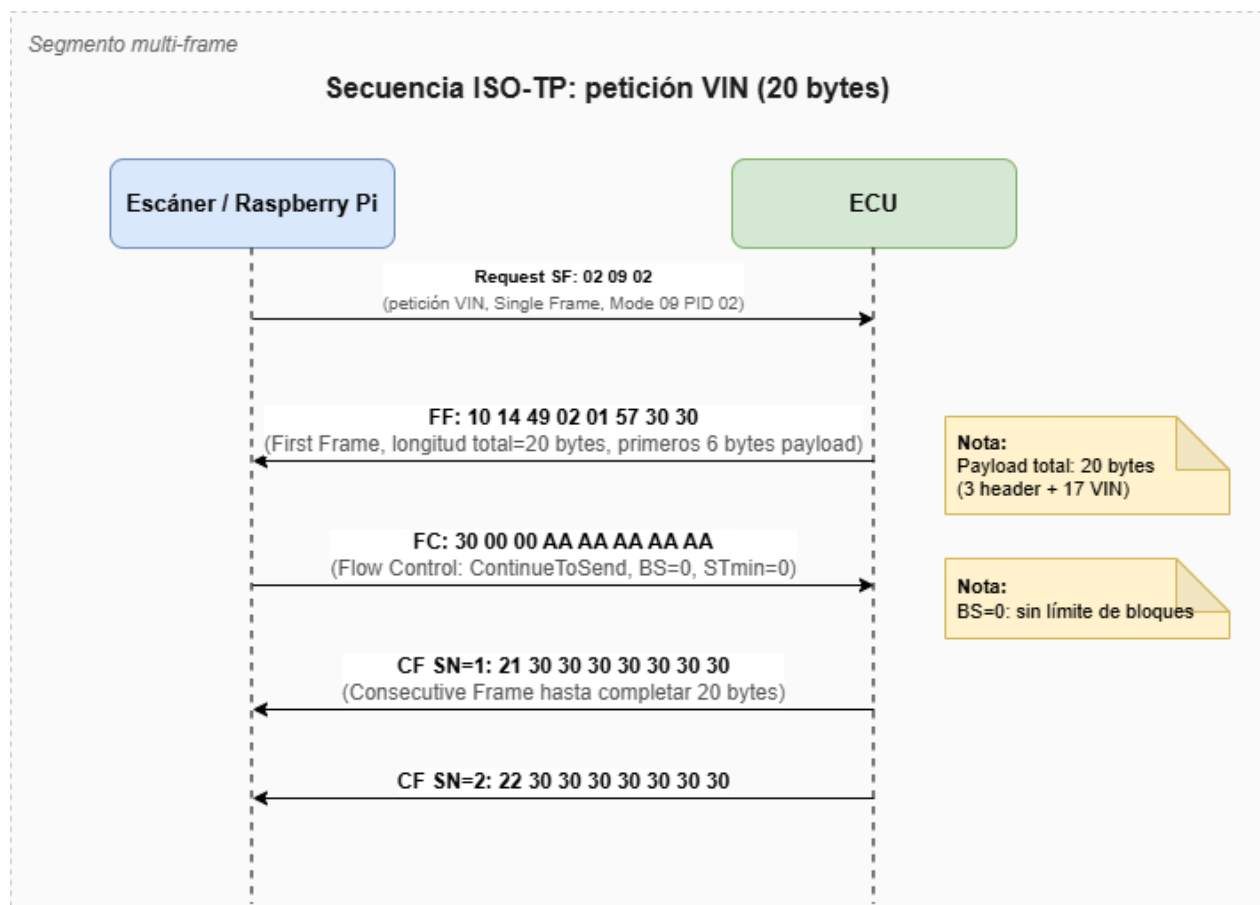


Ilustración 2.1: Secuencia de intercambio ISO-TP para la obtención del VIN (20 bytes: modo 0x49, PID 0x02, contador 0x01 y 17 bytes ASCII). El escáner envía una petición Single Frame, la ECU responde con First Frame + Flow Control (BS=0, STmin=0) + dos Consecutive Frames.

Cuadro 2.4: Estructura de trama OBD-II: petición (CAN ID 0x7E0) y respuesta (CAN ID 0x7E8) para el PID 0x0C (RPM) según SAE J1979 [4].

Petición (Escáner → ECU, CAN ID: 0x7E0)

	PCI	Modo	PID	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
<i>Valor</i>	0x02	0x01	0x0C	0xAA	0xAA	0xAA	0xAA	0xAA
<i>Significado</i>	SF, 2 bytes	Modo 01	RPM	<i>padding</i>				

Respuesta (ECU → Escáner, CAN ID: 0x7E8)

	PCI	Modo	PID	Byte A	Byte B	Byte 5	Byte 6	Byte 7
<i>Valor</i>	0x04	0x41	0x0C	0x1A	0xF0	0xAA	0xAA	0xAA
<i>Significado</i>	SF, 4 bytes	Modo+0x40	RPM	datos RPM		<i>padding</i>		

$$\text{Decodificación: } \text{RPM} = \frac{(A \ll 8) | B}{4} = \frac{0x1AF0}{4} = 1724,0 \text{ rpm}$$

Cada petición OBD-II incluye un byte de modo y, opcionalmente, uno o más bytes de PID para solicitar la información únicamente de esos PIDs. La respuesta de la ECU incluye el modo de respuesta (modo + 0x40), el o los PIDs solicitados y los bytes de datos codificados, cuya interpretación depende de la fórmula de decodificación definida por el estándar para cada PID [4].

SAE J1979 define diez servicios de diagnóstico, numerados del modo 0x01 al 0x0A [4]. Estos servicios cubren desde la lectura de datos en tiempo real hasta la gestión de códigos de avería, la consulta de resultados de los monitores a bordo y la identificación del vehículo. A continuación se describe cada uno de ellos:

- **Modo 0x01 - Datos en tiempo real (PIDs activos).** Permite consultar el valor actual de parámetros operativos del motor [4]. Se han implementado 25 PIDs (la tabla completa con todos los PIDs definidos en el OBD-II se encuentra en el Anexo A) agrupados en cinco categorías:
 1. **motor** (carga 0x04, RPM 0x0C, avance encendido 0x0E, tiempo desde arranque 0x1F, carga absoluta 0x43).
 2. **térmicos** (temperatura refrigerante 0x05, temperatura aire admisión 0x0F, temperatura aceite 0x5C, temperatura exterior 0x46).
 3. **combustible** (presión 0x0A, MAP 0x0B, flujo MAF 0x10, nivel 0x2F, presión rampa 0x23, consumo 0x5E, trim corto/largo B1 0x06/0x07).
 4. **cinemática** (velocidad 0x0D, distancia desde borrado DTC 0x31)
 5. **eléctrico y posición** (tensión de batería 0x42, presión barométrica 0x33, posición acelerador 0x11/0x47, pedal acelerador D/E 0x49/0x4A).
 6. **descubrimiento** (PID especial para solicitar los PIDs soportados por la ECU).

- **Modo 0x02 - Datos congelados (*freeze frame*).** Devuelve una instantánea de los valores de los PIDs del modo 0x01 capturada automáticamente por la ECU en el instante exacto en que se almacenó un DTC [4]. La petición reutiliza los mismos PIDs que el modo 0x01 pero añade un byte de *frame number* que selecciona el marco congelado a consultar. Es una herramienta clave de diagnóstico, ya que permite reconstruir las condiciones de funcionamiento (RPM, carga, temperatura) que provocaron el fallo.
- **Modo 0x03 - Lectura de DTCs almacenados.** Devuelve la lista de códigos de avería (*Diagnostic Trouble Codes*) presentes en la memoria de la ECU [4]. Cada DTC se codifica en dos bytes según el formato SAE J2012 [15]: los dos bits más significativos determinan la categoría del sistema (P = *Powertrain*, C = *Chassis*, B = *Body*, U = *Network/Communication*), el siguiente par de bits indica si el código es estándar (0) o específico del fabricante (1), y los 12 bits restantes forman el código numérico de la avería. Por ejemplo, P0300 corresponde al código hexadecimal 0x0300: categoría Powertrain, código estándar, fallo de encendido aleatorio o múltiple [15, 16].
- **Modo 0x04 - Borrado de DTCs.** Borra todos los DTCs almacenados y reinicia el estado del indicador *Check Engine* [4]. Este servicio no requiere PID adicional; la ECU responde con el byte de confirmación 0x44 (modo + 0x40). Esta operación es **irreversible**: elimina tanto los DTCs confirmados como los pendientes y reinicia todos los monitores OBD-II, por lo que debe ejecutarse únicamente tras haber registrado y analizado los códigos presentes.
- **Modo 0x05 - Resultados de los test de los sensores de oxígeno.** Devuelve los resultados de los test de monitorización de las sondas lambda (tensiones de conmutación, tiempos de respuesta) [4]. Este servicio solo se emplea en los buses de diagnóstico anteriores a CAN; en los vehículos con bus CAN (ISO 15765-4) su funcionalidad se traslada al modo 0x06, por lo que se considera obsoleto.
- **Modo 0x06 - Resultados de los monitores a bordo.** Devuelve los resultados de los test de los sistemas monitorizados de forma no continua por la ECU (catalizador, sistema EGR, sondas de oxígeno, etc.) [4]. Cada test se identifica mediante un MID (*Monitor ID*) y un TID (*Test ID*), y la respuesta incluye el valor medido junto con sus límites mínimo y máximo admisibles, lo que permite verificar si un monitor ha superado o no su umbral antes de que se genere un DTC.
- **Modo 0x07 - Lectura de DTCs pendientes.** Devuelve los DTCs detectados durante el ciclo de conducción actual o el último, que aún no se han confirmado [4]. Un código se vuelve pendiente cuando un fallo se detecta una sola vez; si el fallo se repite en un segundo ciclo de conducción, el código pasa a confirmado (modo 0x03) y se enciende el testigo *Check Engine*. El formato de codificación de los DTCs es idéntico al del modo 0x03.
- **Modo 0x08 - Control de componentes a bordo.** Permite al equipo de diagnóstico activar o desactivar la operación de un componente o sistema de la ECU (por ejemplo, forzar el ciclo de una electroválvula del sistema EVAP) [4]. Es un servicio de actuación bidireccional, poco habitual y dependiente del fabricante, que debe usarse con precaución dado que actúa físicamente sobre el vehículo.

- **Modo 0x09 - Información del vehículo.** Permite obtener el número de identificación del vehículo (VIN, PID 0x02), que se devuelve como una cadena ASCII de 17 caracteres mediante una secuencia ISO-TP multi-frame [4, 3].
- **Modo 0x0A - Lectura de DTCs permanentes.** Devuelve los DTCs permanentes, una categoría de códigos confirmados que **no** puede borrarse mediante el modo 0x04 ni desconectando la batería [4]. Solo se eliminan cuando la propia ECU verifica, tras varios ciclos de conducción, que la avería se ha corregido. Su finalidad es impedir que se enmascaren fallos de emisiones borrando los códigos justo antes de una inspección técnica. El formato de codificación coincide con el del modo 0x03.

Cuando la ECU no puede atender una petición (modo no soportado, condiciones no adecuadas, PID no implementado), responde desde su identificador físico (0x7E8) con una trama cuyo primer byte de datos es 0x7F (NRC Service Identifier), seguido del SID rechazado y el código NRC según ISO 14229-1 [5]. El byte NRC (*Negative Response Code*) identifica la causa del rechazo: 0x11 (servicio no soportado), 0x12 (subfunción no soportada), 0x22 (condiciones no cumplidas) o 0x33 (seguridad no desbloqueada).

2.1.4. UDS - Unified Diagnostic Services (ISO 14229-1)

UDS (*Unified Diagnostic Services*), definido en la ISO 14229-1 [5], es el protocolo de diagnóstico de capa de aplicación adoptado por la industria automotriz moderna.

Cada petición UDS se estructura con un byte de identificador de servicio (**SID** *Service Identifier*) seguido de parámetros opcionales. La ECU responde con el SID de respuesta positiva (SID + 0x40) y los datos correspondientes [5]. Cuando la ECU no puede atender la petición, emite una **Negative Response** de tres bytes: 0x7F (identificador de respuesta negativa), el SID rechazado y el **Negative Response Code** (NRC). Este payload viaja encapsulado en un *Single Frame* ISO-TP dentro de una trama CAN, tal y como se muestra en la tabla 2.5: la ECU responde con el identificador 0x7E8, el primer byte es el PCI ISO-TP (0x03, que indica *Single Frame* de 3 bytes), seguido de los tres bytes de la respuesta negativa y el relleno (*padding*) hasta completar los 8 bytes del campo de datos.

Cuadro 2.5: Trama CAN completa recibida en una respuesta negativa UDS/OBD-II: encapsulado ISO-TP (*Single Frame*) del payload *Negative Response* según ISO 14229-1 [5] e ISO 15765-2 [3].

CAN ID	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
0x7E8	0x03	0x7F	SID	NRC	0xAA	0xAA	0xAA	0xAA
<i>ECU resp.</i>	<i>PCI (SF, len=3)</i>		<i>Payload UDS (Negative Response)</i>		<i>Padding</i>			

Tanto UDS como OBD-II usan el mismo esquema de codificación para las respuestas positivas (ID+0x40) y comparten el mismo mecanismo para respuestas negativas (*Negative Response*) definido por la norma ISO [5]. Los servicios UDS que se han implementado en el sistema son *DiagnosticSessionControl* (0x10), que controla el modo de sesión activo en

la ECU, y *ReadDataByIdentifier* (0x22), que permite leer datos solicitados a través de un identificador DID (*Data Identifier*) de 2 bytes.

Los códigos NRC más relevantes se recogen en la tabla 2.6.

Cuadro 2.6: Principales códigos NRC (*Negative Response Code*) de ISO 14229-1 [5] empleados en respuestas negativas OBD-II y UDS.

NRC	Nombre	Causa
0x10	<i>generalReject</i>	Rechazo genérico cuando ningún otro NRC describe el fallo.
0x11	<i>serviceNotSupported</i>	El SID no está implementado en esta ECU.
0x12	<i>subFunctionNotSupported</i>	El modo o subfunción no está soportado.
0x13	<i>incorrectMessageLengthOrInvalidFormat</i>	La longitud del mensaje o el formato de los parámetros es incorrecto.
0x14	<i>responseTooLong</i>	La respuesta excede el tamaño máximo transmisible.
0x21	<i>busyRepeatRequest</i>	La ECU está ocupada; debe repetirse la petición.
0x22	<i>conditionsNotCorrect</i>	Las condiciones del vehículo (motor, velocidad, etc.) no permiten ejecutar el servicio.
0x24	<i>requestSequenceError</i>	La petición llega fuera de la secuencia esperada.
0x31	<i>requestOutOfRange</i>	El DID o parámetro solicitado no existe o está fuera del rango soportado.
0x33	<i>securityAccessDenied</i>	El servicio requiere autenticación previa mediante <i>SecurityAccess</i> (0x27).
0x35	<i>invalidKey</i>	La clave enviada en <i>SecurityAccess</i> no es válida.
0x36	<i>exceedNumberOfAttempts</i>	Se superó el número de intentos de autenticación permitidos.
0x37	<i>requiredTimeDelayNotExpired</i>	No ha transcurrido el tiempo de espera obligatorio tras un fallo de autenticación.
0x78	<i>requestCorrectlyReceived-ResponsePending</i>	Petición recibida; respuesta pendiente por tiempo de procesamiento de la ECU.
0x7E	<i>subFunctionNotSupportedInActiveSession</i>	La subfunción no está disponible en la sesión de diagnóstico activa.
0x7F	<i>serviceNotSupportedInActiveSession</i>	El servicio no está disponible en la sesión de diagnóstico activa.

La diferencia entre el protocolo UDS y el protocolo OBD-II radica principalmente en el objetivo de cada uno y el "lenguaje" que usa cada uno, es decir:

- **OBD-II:** tiene como objetivo principal definir un marco y lenguaje único universal para todos los fabricantes, que se encargue de del análisis de emisiones, afectando casi exclusivamente a la ECU motriz y de transmisión. Además este protocolo se limita a lectura de PIDs.
- **UDS:** tiene un objetivo más específico pues pese a trabajar en un marco similar al OBD-II, este tiene como propósito la ingeniería del vehículo permitiendo la lectura y

escritura de parámetros en todas las ECUs. No obstante este protocolo trabaja con distintos códigos por fabricante haciendo que no haya códigos universales que permitan la escritura/lectura en cualquier ECU.

2.1.5. BLE GATT y Nordic UART Service

La comunicación inalámbrica entre la Raspberry Pi y la aplicación móvil se implementa mediante Bluetooth Low Energy (BLE), usando el perfil GATT (*Generic Attribute Profile*). GATT define cómo dos dispositivos intercambian datos sobre BLE estructurando la información en una jerarquía de *servicios*, *características* y *descriptores* [17]. Cada **servicio** agrupa un conjunto de **características**, que son los contenedores de datos elementales (un valor, una medida, un comando), y cada característica puede llevar asociados **descriptores** que aportan metadatos o configuran su comportamiento. El modelo es asimétrico: el dispositivo que aloja los datos actúa como **servidor GATT** (en este trabajo, la Raspberry Pi) y el que accede a ellos como **cliente GATT** (la aplicación móvil). Por norma general es el cliente quien inicia las operaciones, leyendo (*Read*) o escribiendo (*Write*) el valor de las características que expone el servidor.

En un entorno donde el servidor está continuamente cambiando los datos de forma asíncrona resulta ineficiente este esquema, pues obliga al cliente a estar continuamente sondeando (*polling*) al servidor. GATT define las **notificaciones** para evitar este problema, estas son un mecanismo *push* por el cual el servidor envía el nuevo valor de una característica al cliente en el momento en que cambia, sin que este lo solicite [17]. La notificación no requiere confirmación a nivel ATT (*fire-and-forget*), lo que minimiza la latencia y el consumo frente al sondeo; existe además la variante *indicación*, que sí exige reconocimiento del cliente a costa de mayor latencia. Para que el servidor pueda notificar, el cliente debe habilitar previamente el flujo escribiendo en el descriptor **CCCD** (*Client Characteristic Configuration Descriptor* y el UUID 0x2902) de la característica correspondiente. Este patrón es el que permite a la Raspberry Pi empujar la telemetría y las respuestas a la aplicación móvil en tiempo real.

El servicio empleado en este trabajo es el **Nordic UART Service (NUS)** [18], un servicio GATT propietario de Nordic Semiconductor ampliamente soportado que emula un puerto serie bidireccional sobre BLE mediante un UUID de 128 bits de base aleatoria (6E400001-B5A3-F393-E0A9-E50E24DCCA9E). NUS define dos características principales [18]:

- **TX Characteristic** (6E400003-B5A3-F393-E0A9-E50E24DCCA9E): el servidor envía datos al cliente mediante notificaciones GATT. El cliente activa las notificaciones escribiendo en el descriptor CCCD (*Client Characteristic Configuration Descriptor*) [18].
- **RX Characteristic** (6E400002-B5A3-F393-E0A9-E50E24DCCA9E): el cliente escribe datos al servidor mediante operaciones GATT Write [18].

El MTU negociado durante el establecimiento de la conexión determina el tamaño máximo de cada paquete de datos. En este trabajo se configura un MTU de 240 bytes, que es tamaño más que suficiente para transportar los objetos JSON más grandes de la aplicación (respuestas con múltiples muestras de telemetría) en un único paquete, evitando la fragmentación a nivel de aplicación.

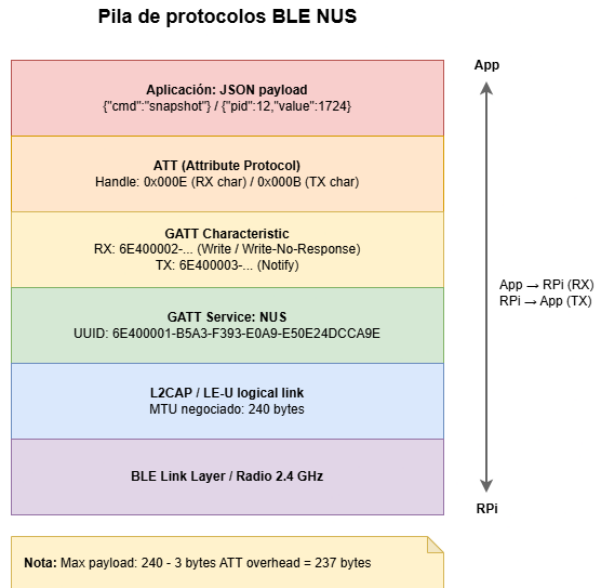


Ilustración 2.2: Pila de protocolos del Nordic UART Service (NUS) sobre BLE. Los comandos y respuestas JSON se transmiten como payload ATT sobre las características RX (escritura) y TX (notificación) del servicio NUS, con un MTU negociado de 240 bytes.

El flujo de establecimiento de conexión se puede observar en la siguiente figura 2.3 que también incluye las fases de advertising, conexión, negociación de MTU, descubrimiento de servicios, activación de notificaciones y autenticación por token.

Sobre la capa de transporte NUS se define un **protocolo de comandos JSON** específico de la aplicación que estructura los intercambios en forma de petición-respuesta. Cada objeto JSON de petición incluye un campo `cmd` que identifica la operación solicitada y, opcionalmente, parámetros adicionales. Todas las respuestas incluyen un campo `status` (`ok` / `error`) y un campo `data` con el resultado.

La delimitación de mensajes se resuelve mediante **NDJSON** (*Newline-Delimited JSON*): cada objeto JSON se serializa en una única línea ASCII y se termina con el carácter de nueva línea `\n`. El receptor acumula los bytes recibidos en un buffer y extrae un mensaje completo cada vez que detecta el delimitador. Esta convención elimina la necesidad de codificar la longitud del mensaje en una cabecera y es compatible con la fragmentación BLE: si el payload excede el MTU (240 bytes), se segmenta en múltiples notificaciones GATT que el receptor concatena hasta encontrar el `\n` final.

La estructura de los mensajes del protocolo se recoge en la tabla 2.7.

Adicionalmente, el servidor puede enviar mensajes *push* sin petición previa: `{type: "samples"}` con un ciclo de muestras del monitor de datos en vivo, `{type: "error"}` ante un fallo de lectura de PID, y `{type: "heartbeat"}` como sonda de keepalive cuando el cliente permanece inactivo más de 8 segundos. El cliente debe responder al heartbeat con `{type: "heartbeat_ack"}` para mantener la sesión activa; la inactividad superior a 15 segundos provoca la desconexión automática y el reinicio del servidor BLE.

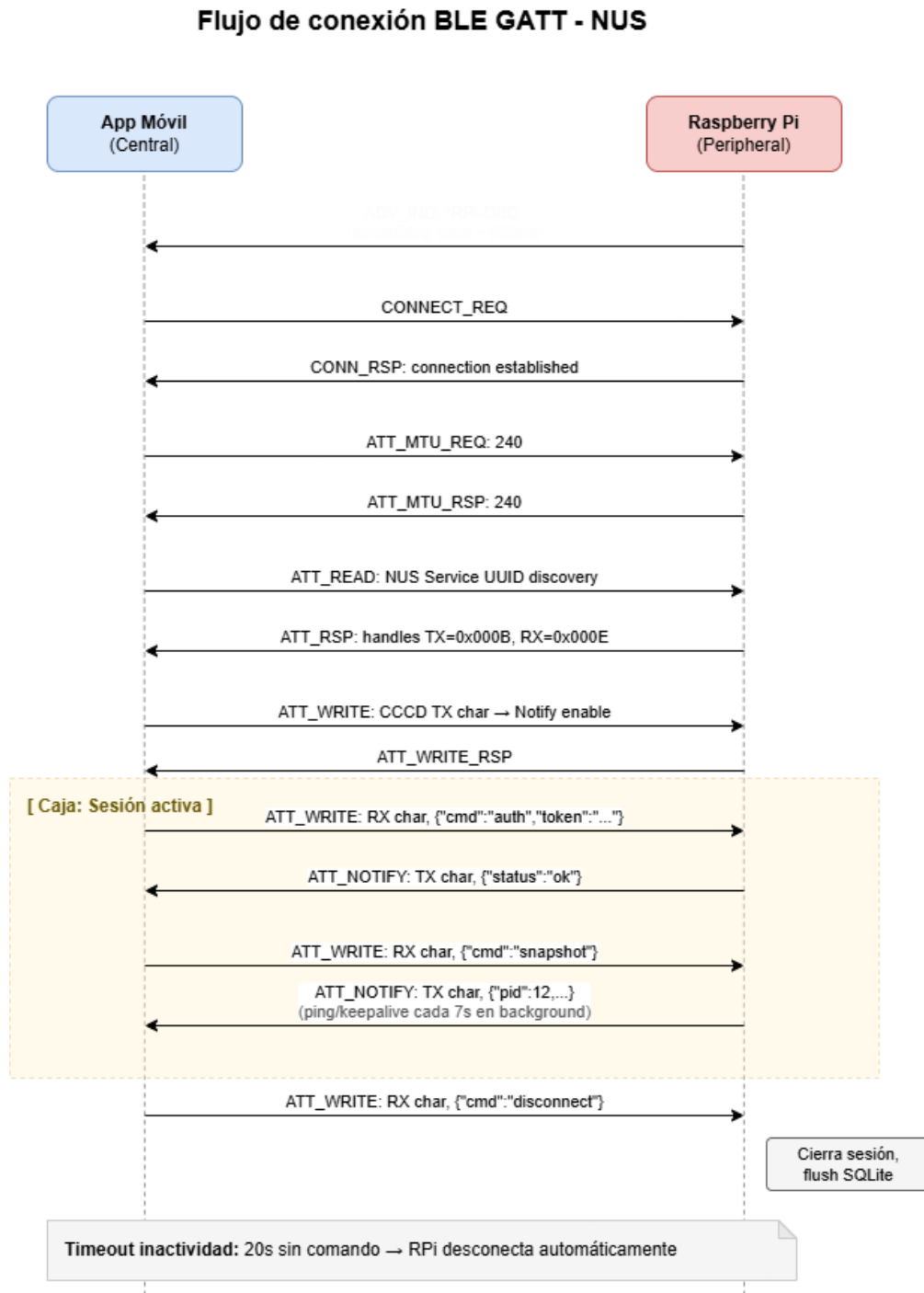


Ilustración 2.3: Flujo completo de una conexión BLE GATT con el servicio NUS. Tras el descubrimiento y negociación de MTU, la app habilita las notificaciones en la característica TX y comienza el intercambio de comandos JSON. Un mecanismo de keepalive mantiene la conexión activa; la inactividad superior a 20 segundos provoca el cierre de sesión.

Cuadro 2.7: Estructura genérica de los mensajes NDJSON del protocolo sobre NUS. Cada objeto se serializa en una línea ASCII terminada en `\n`.

Tipo de mensaje	Campos	Descripción
Petición	<code>cmd</code> (+ params)	Cliente → servidor. <code>cmd</code> identifica la operación; los parámetros opcionales se incluyen en el mismo objeto.
Respuesta OK	<code>status="ok"</code> , <code>data</code>	Servidor → cliente. <code>data</code> contiene el resultado (escalar, objeto o lista).
Respuesta error	<code>status="error"</code> , <code>message</code>	Servidor → cliente. <code>message</code> describe la causa del fallo.
Push	<code>type</code> (+ datos)	Servidor → cliente sin petición previa (<code>samples</code> , <code>error</code> , <code>heartbeat</code>).

Ejemplo petición: `{"cmd":"uds_read_did","did":"0xF190"}\n`

Ejemplo respuesta: `{"status":"ok","data":{"did":"0xF190","name":"VIN",...}}\n`

2.2. Objetivos

A partir del contexto descrito, se definen los siguientes objetivos para este proyecto.

2.2.1. Objetivo general

Diseñar y desarrollar un sistema empotrado de diagnóstico del vehículo basado en el protocolo OBD-II sobre bus CAN, con acceso inalámbrico vía Bluetooth Low Energy y validado mediante un banco de pruebas basado en Arduino.

La propuesta de este trabajo (TFT01) define tres funciones de usuario de alto nivel que el sistema debe satisfacer.

2.2.2. Objetivos específicos

Los seis objetivos específicos de la memoria que concretan estas funciones son: La tabla 2.8 recoge la trazabilidad entre las funciones de usuario definidas en la propuesta y los objetivos específicos que las implementan.

O1. Implementar la pila de protocolos OBD-II completa: desarrollar sobre Raspberry Pi una implementación funcional de OBD-II (SAE J1979) como capa de aplicación y utilizando la librería `iso-tp` de python como abstracción para la capa de transporte, cubriendo los modos de servicio 0x01 (datos en vivo), 0x03 (lectura de DTCs), 0x04 (borrado de DTCs) y 0x09 (información del vehículo, incluyendo VIN).

O2. Desarrollar un simulador de ECU: implementar sobre Arduino MKR con shield

Cuadro 2.8: Trazabilidad entre funciones definidas en la propuesta TFT01 y objetivos específicos del trabajo.

Función TFT01	Descripción	Obj.
Monitorización en tiempo real	Visualización numérica de parámetros de sensores disponibles en el vehículo (RPM, temperaturas, presiones, voltajes, etc.)	O1, O2, O4, O5, O6
Consulta y gestión de errores	Lectura y borrado de códigos de error (DTCs) almacenados en la memoria de las ECUs	O1, O2, O3, O4, O6
Identificación	Consulta de datos del vehículo (VIN) e información de los distintos módulos integrados	O1, O2, O4, O5

CAN un simulador de ECU que responda a peticiones OBD-II estándar con datos de telemetría realistas, con el objetivo de disponer de un banco de pruebas reproducible y controlado que no dependa de la disponibilidad de un vehículo real.

- O3. Implementar servicios UDS y simular variabilidad del bus:** incorporar los servicios UDS DiagnosticSessionControl (0x10) y ReadDataByIdentifier (0x22) de ISO 14229-1, y dotar al simulador Arduino de mecanismos opcionales de variabilidad (ruido en los valores de los sensores y emisión de tramas de difusión) para validar la robustez de la herramienta de diagnóstico.
- O4. Proporcionar acceso inalámbrico mediante BLE:** implementar un servidor BLE GATT sobre la Raspberry Pi que exponga la funcionalidad de diagnóstico a través del servicio Nordic UART Service (NUS), eliminando la necesidad de conexión física entre el dispositivo de diagnóstico y el usuario.
- O5. Desarrollar una aplicación móvil multiplataforma:** implementar una aplicación para Android e iOS con React Native que permita al usuario conectarse al sistema de diagnóstico vía BLE, monitorizar parámetros del vehículo en tiempo real, gestionar los códigos de avería y consultar el historial de sesiones almacenadas.
- O6. Registrar y almacenar los datos de diagnóstico:** implementar un sistema de logging estructurado en base de datos SQLite que almacene las sesiones, las muestras de telemetría y los comandos ejecutados, para su consulta y análisis posterior.

2.2.3. Alcance y limitaciones

El sistema desarrollado implementa los modos OBD-II estándar definidos por SAE J1979 [4] (modos 0x01, 0x03, 0x04 y 0x09) y los servicios UDS 0x10 y 0x22 de ISO 14229-1 [5]. Quedan fuera del alcance los servicios de diagnóstico propietarios de fabricante, que requieren acuerdos de licencia y herramientas específicas por marca, así como los modos OBD-II 0x02, 0x05, 0x06, 0x07 y 0x08, cuya implementación no se alineaba con los objetivos del trabajo.

La estrategia de validación del sistema se apoya principalmente en un banco de pruebas basado en Arduino MKR WAN 1310, que implementa un simulador de ECU con modelo

de dinámica de motor y capacidad de inyección de DTCs. Este enfoque permite reproducir condiciones de diagnóstico independientemente de la disponibilidad de un vehículo real. Adicionalmente, el sistema se validó sobre un vehículo real del grupo VAG (Škoda Yeti de 2015), confirmando la interoperabilidad con una ECU de producción a través del conector OBD-II, como se detalla en el Capítulo 6.

Capítulo 3

Aportaciones del trabajo

En esta capítulo se presentan las aportaciones de este trabajo así como las competencias y ODS que sigue.

3.1. Principales aportaciones

Este trabajo presenta un conjunto de aportaciones técnicas que, consideradas en su conjunto, constituyen una plataforma de diagnóstico del vehículo OBD-II completa, de código abierto y bajo coste, diferenciada de las soluciones comerciales existentes en varios aspectos relevantes.

Implementación directa de la pila de protocolos. A diferencia de los adaptadores comerciales de tipo ELM327, que abstraen el protocolo OBD-II mediante comandos AT propietarios, el sistema desarrollado implementa directamente las capas ISO-TP y OBD-II sobre hardware de propósito general. Esto proporciona control total sobre los tiempos de comunicación, la gestión de errores y la extensibilidad del sistema, sin depender de firmware cerrado de terceros.

Banco de pruebas reproducible basado en Arduino. El simulador de ECU desarrollado sobre Arduino MKR con shield CAN constituye una herramienta de validación independiente del vehículo real. El simulador implementa un modelo físico del motor, responde a los cuatro modos OBD-II estándar (0x01, 0x03, 0x04, 0x09) y a los servicios UDS 0x10/0x22, e incorpora mecanismos opcionales de variabilidad del bus (ruido en los valores de los sensores y emisión de tramas de difusión) e inyección de códigos de avería mediante interrupción hardware.

Arquitectura software modular con separación de capas. La herramienta de diagnóstico implementada en Python sobre Raspberry Pi sigue un diseño basado en interfaces y capas desacopladas mediante inyección de dependencias. Esta arquitectura facilita la sustitución de cualquier componente (transporte, decodificador, logger) sin modificar el resto del sistema, y permite el uso de un transporte (*mock*) para el desarrollo y las pruebas sin necesidad de hardware CAN.

Acceso inalámbrico mediante BLE GATT con logging persistente. La integración de un servidor BLE GATT sobre Raspberry Pi, combinada con el registro estructurado de datos en SQLite, permite el acceso en tiempo real a los parámetros del vehículo, así como la consulta posterior de sesiones históricas completas, incluyendo las tramas CAN en hexadecimal correspondientes a cada comando ejecutado.

Aplicación móvil multiplataforma. La aplicación React Native desarrollada opera sobre Android e iOS con una única base de código, implementa el protocolo JSON sobre NUS para la comunicación con la Raspberry Pi y ofrece una experiencia de usuario orientada tanto a la monitorización en tiempo real como al análisis de datos históricos.

3.2. Competencias específicas desarrolladas

La realización de este trabajo ha requerido la aplicación y el desarrollo de competencias específicas del Grado de Ingeniería Informática, plan 41, de la Universidad de Las Palmas de Gran Canaria. Las principales competencias implicadas en el desarrollo de este TTF son:

CI1 — Capacidad para diseñar, desarrollar, evaluar y asegurar la accesibilidad, ergonomía, usabilidad y seguridad de los sistemas, servicios y aplicaciones informáticas, así como de la información que gestionan. Esta competencia se ha desarrollado en el diseño de la aplicación móvil, prestando atención a la usabilidad de la interfaz de diagnóstico, y en la implementación de mecanismos de seguridad básicos en la comunicación BLE (autenticación por token, timeout de inactividad).

CI5 — Capacidad para especificar, diseñar, implementar y evaluar interfaces persona-computador que garanticen la accesibilidad y usabilidad de los sistemas, servicios y aplicaciones informáticas. Esta competencia se ha aplicado en el diseño e implementación de la aplicación móvil React Native, con sus cinco pantallas funcionales y el sistema de navegación por pestañas.

CI8 — Capacidad para diseñar y evaluar sistemas operativos y servidores, y aplicaciones y sistemas basados en computación distribuida. Arduino, Raspberry Pi y dispositivo móvil son tres nodos que se comunican mediante protocolos bien definidos. La Raspberry Pi actúa simultáneamente como cliente CAN y como servidor BLE, gestionando concurrencia mediante hilos y comunicación asíncrona.

CI14 — Capacidad para diseñar sistemas de tiempo real y sistemas empujados. Esta es la competencia más directamente relacionada con el trabajo. El simulador Arduino opera en tiempo real con interrupciones hardware, temporización precisa de las tramas ISO-TP y gestión concurrente de múltiples tareas (broadcast CAN, respuesta a peticiones OBD-II, generación de ruido). La Raspberry Pi implementa monitorización continua con periodos de muestreo de 500 ms.

CI16 — Capacidad para diseñar y desarrollar sistemas de comunicaciones. El núcleo del trabajo es la implementación de una pila de protocolos de comunicaciones: CAN (ISO 11898), ISO-TP (ISO 15765-2), OBD-II (SAE J1979) y BLE GATT.

CI18 — Capacidad para diseñar e implementar sistemas de tiempo real y sistemas empotrados. En complemento con CI14, esta competencia abarca el diseño del flujo de control del simulador Arduino, la implementación del servidor BLE con asyncio en Python y la coordinación entre el hilo de monitorización continua y el hilo de atención a comandos BLE.

3.3. Relación con los Objetivos de Desarrollo Sostenible

Los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas establecen un marco global de referencia para orientar el desarrollo tecnológico hacia un impacto positivo en la sociedad y el medio ambiente. La tabla 3.1 recoge el grado de relación de este trabajo con cada uno de los 17 ODS.

Cuadro 3.1: Grado de relación de este trabajo con los 17 Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030.

ODS	Grado de relación			
	0 No procede	1 Bajo	2 Medio	3 Alto
1 Fin de la Pobreza	X			
2 Hambre Cero	X			
3 Salud y Bienestar	X			
4 Educación de Calidad			X	
5 Igualdad de Género	X			
6 Agua Limpia y Saneamiento	X			
7 Energía Asequible y No Contaminante	X			
8 Trabajo Decente y Crecimiento Económico		X		
9 Industria, Innovación e Infraestructuras			X	
10 Reducción de las Desigualdades	X			
11 Ciudades y Comunidades Sostenibles	X			
12 Producción y Consumo Responsables		X		
13 Acción por el Clima	X			
14 Vida Submarina	X			
15 Vida de Ecosistemas Terrestres	X			
16 Paz, Justicia e Instituciones Sólidas	X			
17 Alianzas para Lograr los Objetivos	X			

A continuación se justifica el grado de relación asignado a los ODS con mayor vinculación con el trabajo.

ODS 4 — Educación de Calidad (Medio). El trabajo tiene un carácter formativo puesto que aborda el estudio e implementación de protocolos de comunicación industrial (CAN, ISO-TP, OBD-II y UDS). El banco de pruebas basado en Arduino, junto con la documentación del proceso de desarrollo, constituyen un recurso didáctico reutilizable para la enseñanza de protocolos de comunicación.

ODS 9 — Industria, Innovación e Infraestructuras (Medio). La implementación directa de la pila de protocolos OBD-II y UDS sobre hardware de propósito general, sin depender de firmware propietario como el ELM327, representa una aproximación innovadora al diagnóstico del vehículo. Este tipo de soluciones puede favorecer el acceso a capacidades de diagnóstico en entornos donde las herramientas profesionales resultan económicamente inaccesibles.

ODS 8 — Trabajo Decente y Crecimiento Económico (Bajo). En la medida en que el sistema desarrollado reduce la barrera de entrada al diagnóstico del vehículo, puede contribuir marginalmente a facilitar el trabajo de pequeños talleres o profesionales independientes que carecen de acceso a herramientas de diagnóstico de coste elevado.

ODS 12 — Producción y Consumo Responsables (Bajo). El acceso a diagnóstico del vehículo de bajo coste facilita la detección temprana de averías, lo que puede contribuir a prolongar la vida útil del vehículo y a reducir sustituciones prematuras de componentes. La relación con este ODS es indirecta y su alcance limitado al contexto de uso del sistema.

Capítulo 4

Metodología de desarrollo

En este capítulo se detalla lo conseguido durante las distintas etapas del desarrollo detallando el análisis, diseño.

4.1. Metodología de desarrollo

El desarrollo de este trabajo se ha llevado a cabo siguiendo la metodología basada en la construcción iterativa de prototipos, siguiendo con lo planteado en la propuesta inicial del trabajo (TFT01). Se seleccionó esta metodología dada la dinámica explorativa del proyecto.

La metodología basada en prototipos se estructura en cuatro fases principales: análisis de requisitos, diseño rápido y construcción del prototipo inicial, evaluación y refinamiento iterativo, junto con la documentación más la posterior presentación. Estas fases no son estrictamente secuenciales sino que admiten solapamiento parcial: en particular, las fases de construcción y evaluación se ejecutaron de forma alternada durante la mayor parte del desarrollo, con ciclos cortos de implementación seguidos de pruebas sobre el banco de ensayo Arduino.

4.2. Fases y tareas

4.2.1. Fase 1 — Análisis de requisitos (50 horas estimadas)

Esta fase inicial se dedicó al estudio de los fundamentos técnicos necesarios para abordar el desarrollo del sistema.

Tarea 1.1 — Estudio de la plataforma hardware. Se analizaron las características de la Raspberry Pi como plataforma de ejecución de la herramienta de diagnóstico, incluyendo el soporte de la interfaz SocketCAN en Linux, la compatibilidad con el controlador CAN MCP2515 y las opciones disponibles para el servidor BLE. Asimismo, se evaluó el Arduino MKR como plataforma del simulador, estudiando la librería CAN disponible y sus capacidades de temporización.

Tarea 1.2 — Estudio de los protocolos de comunicación CAN. Se realizó un análisis detallado de los estándares ISO 11898 (CAN), ISO 15765-2 (ISO-TP) y SAE J1979 (OBD-II), prestando especial atención a la estructura de las tramas, los mecanismos de control de flujo y el formato de codificación de cada parámetro. Se revisaron asimismo los conceptos fundamentales de UDS (ISO 14229-1) y sus servicios más relevantes para la implementación también de las respuestas negativas.

Tarea 1.3 — Estudio de Bluetooth Low Energy y librerías disponibles. Se estudiaron los conceptos fundamentales de BLE GATT, el servicio Nordic UART Service (NUS) y las opciones de implementación disponibles en Python para la Raspberry Pi, evaluando las librerías `bleak` y `bless`. En el lado de la aplicación móvil, se analizó la librería `react-native-ble-plx` y su compatibilidad con las características de escritura y notificación del servicio NUS.

Tarea 1.4 — Análisis de herramientas software. Se definió el conjunto de herramientas de desarrollo a utilizar: Python 3 con las librerías `python-can` y `can-isotp` para la capa de transporte CAN, SQLite para la persistencia de datos, React Native con Expo para la aplicación móvil, y el IDE de Arduino para el simulador. Se estableció asimismo la estructura del repositorio y el flujo de trabajo con control de versiones Git usando ramas de independientes por uno de los tres elementos del proyecto.

4.2.2. Fase 2 — Diseño rápido y construcción (75 horas estimadas)

Tarea 2.1 — Diseño de la arquitectura del sistema. Se definió la arquitectura general del sistema en tres capas: simulador Arduino, herramienta de diagnóstico en Python y aplicación móvil. Para la herramienta Python se diseñó una arquitectura modular basada en interfaces abstractas (`ITransport`, `IProtocolBuilder`, `IDataDecoder`, `IDiagnosticSession`), con inyección de dependencias para facilitar las pruebas. Se definieron asimismo el esquema de la base de datos SQLite y el protocolo de comandos JSON utilizado sobre BLE.

Tarea 2.2 — Implementación del primer prototipo funcional. Se desarrolló una primera versión operativa del sistema que incluía: el simulador Arduino respondiendo a peticiones OBD-II básicas en modo 0x01 (RPM, temperatura de refrigerante, velocidad), la herramienta Python con transporte ISO-TP funcional y sesión de diagnóstico básica.

4.2.3. Fase 3 — Evaluación, refinamiento e implementación (125 horas estimadas)

Esta fue la fase de mayor duración e intensidad técnica, en la que el prototipo inicial evolucionó de forma iterativa hasta el sistema final.

Tarea 3.1 — Extensión del simulador Arduino. Se amplió el simulador con soporte completo para los cuatro modos OBD-II (0x01, 0x03, 0x04, 0x09) y los servicios UDS `DiagnosticSessionControl` (0x10) y `ReadDataByIdentifier` (0x22), implementando la respuesta VIN mediante la secuencia ISO-TP multi-frame (FF + CF). Se añadió el modelo físico de simulación del motor, con la lógica de aceleración, velocidad, temperaturas y consumo de combustible. Posteriormente se incorporaron los mecanismos opcionales de variabilidad del

bus (ruido en los sensores y tramas de difusión) y la interrupción hardware para la inyección secuencial de DTCs.

Tarea 3.2 — Extensión de la herramienta Python. Se completó la herramienta con el módulo de logging en SQLite, el decorador `LoggingTransport`, el monitor de datos en tiempo real con hilo dedicado, la sesión de diagnóstico extendida (`LoggedDiagnosticSession`) y el servidor BLE GATT con el manejador de comandos JSON (`BtCommandHandler`).

Tarea 3.3 — Desarrollo completo de la aplicación móvil. Se comenzó el desarrollo completo de la aplicación móvil, implementando las cinco pantallas de la aplicación (Panel, Averías, Sesiones, Consola y Ajustes), incluida la pestaña UDS y el flujo de conexión por modales, los almacenes de estado con Zustand, el adaptador BLE con gestión de timeout y reconexión, y el adaptador simulado (`MockAdapter`) para el desarrollo sin hardware.

Tarea 3.4 — Pruebas de integración. Se realizaron pruebas del sistema completo sobre el banco de ensayo, verificando el comportamiento ante los escenarios de variabilidad del bus configurables en el simulador, la correcta persistencia de los datos en SQLite y la estabilidad de la conexión BLE durante sesiones de monitorización prolongadas. Finalmente, el sistema se validó sobre un vehículo real (Škoda Yeti 2015).

4.2.4. Fase 4 — Documentación y presentación (50 horas estimadas)

Tarea 4.1 — Redacción de la memoria. Redacción del presente documento, incluyendo la elaboración de las tablas y figuras de los protocolos de comunicación.

4.3. Desviaciones respecto a la planificación inicial

La planificación inicial recogida en el TFT01 contemplaba una distribución de 300 horas totales, con especial peso en la fase de evaluación e implementación (125 horas). En líneas generales, esta estimación resultó ajustada a la realidad del desarrollo, si bien se produjeron algunas desviaciones significativas que conviene señalar.

Por otra parte, la incorporación de los mecanismos opcionales de variabilidad del bus en el simulador no estaba prevista en la planificación original, pero se identificó como útil durante las primeras pruebas de integración. En cuanto a la validación sobre vehículo real, el vehículo objetivo inicial (SEAT Ibiza 6J) no pudo emplearse por problemas de acceso dado que la plataforma del modelo que se había planteado no implementa el protocolo OBD-II sobre el bus CAN en los pines 6 y 14 sino que lo implementa en la antigua línea k-line, por lo que finalmente se llevó a cabo sobre un Škoda Yeti de 2015, también del grupo VAG.

La decisión de estructurar el código Python con una arquitectura de interfaces y capas desacopladas supuso una inversión inicial de tiempo superior a la contemplada en el diseño rápido, pero resultó una buena práctica que se aprovechó en las fases posteriores puesto que facilitó considerablemente la extensión del sistema y la realización de pruebas con el transporte simulado.

Capítulo 5

Desarrollo del sistema

5.1. Arquitectura general del sistema

El sistema desarrollado se articula en torno a tres nodos de procesamiento independientes que se comunican mediante dos canales físicos distintos: el bus CAN entre el simulador Arduino y la herramienta de diagnóstico, y Bluetooth Low Energy entre la herramienta de diagnóstico y la aplicación móvil. La figura 5.1 ilustra la arquitectura global del sistema.

El **simulador de ECU** (Arduino MKR con shield CAN) reproduce el comportamiento de una unidad de control electrónico vehicular sobre el bus CAN a 500 kbps. Responde a peticiones OBD-II en los cuatro modos implementados (0x01, 0x03, 0x04 y 0x09) y a dos servicios UDS (0x10 DiagnosticSessionControl y 0x22 ReadDataByIdentifier), genera datos de telemetría mediante un modelo físico simplificado del motor, y permite inyectar averías (DTCs) mediante un pulsador hardware. Opcionalmente puede activarse ruido en los valores de los sensores y la emisión de tramas de difusión periódicas, ambos desactivados por defecto.

La **herramienta de diagnóstico** (Raspberry Pi con controlador MCP2515) actúa como nodo central del sistema: implementa la pila de protocolos completa (ISO-TP + OBD-II + UDS), persiste los datos en SQLite y expone el servicio BLE GATT hacia la aplicación móvil. Internamente opera con dos hilos de ejecución concurrentes: el hilo de monitorización continua y el hilo de atención a comandos BLE.

La **aplicación móvil** (React Native) se conecta a la Raspberry Pi vía BLE e interactúa con el sistema a través de comandos JSON sobre el servicio Nordic UART Service. Ofrece una interfaz de usuario estructurada en cinco pantallas funcionales.

La separación de responsabilidades entre los tres componentes permite que cada uno sea sustituido o extendido de forma independiente. En particular, la herramienta Python puede operar con un transporte simulado (*mock*) sin necesidad de hardware CAN, y la aplicación móvil incorpora un adaptador simulado que permite su desarrollo sin conexión Bluetooth activa.

Arquitectura General del Sistema

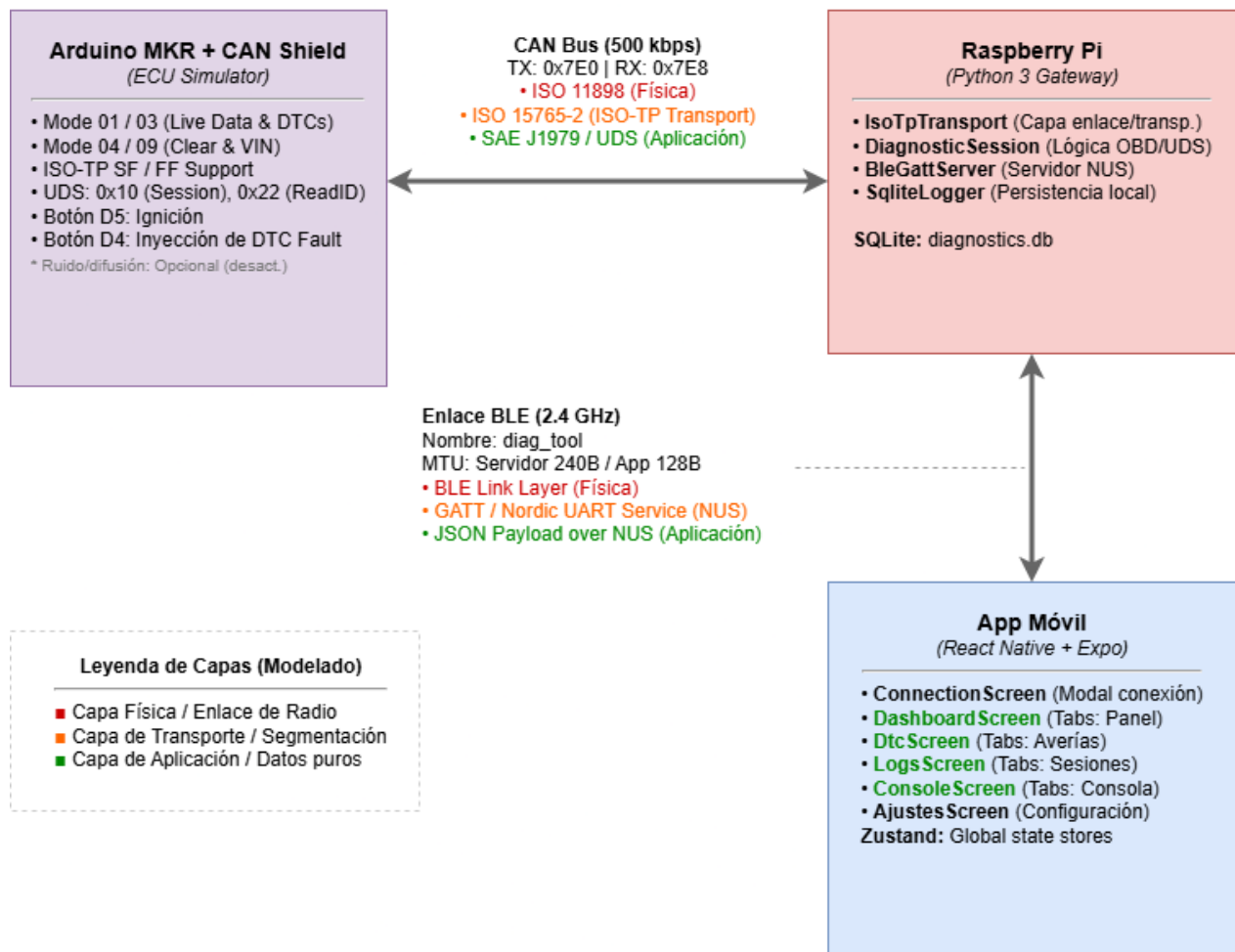


Ilustración 5.1: Arquitectura general del sistema de diagnóstico del vehículo.

5.2. Dependencias y entorno de desarrollo

Cada uno de los tres componentes se apoya en un conjunto reducido de dependencias de terceros, seleccionadas por implementar estándares abiertos. A continuación se describe el entorno y las librerías de cada nodo, así como el uso concreto que se hace de cada una.

Simulador de ECU (Arduino)

El firmware se desarrolla en C++ sobre el *core* de Arduino y se compila con el Arduino IDE. Su única dependencia externa es:

- **arduino-CAN** de Sandeep Mistry [19]: abstrae el controlador CAN MCP2515 [10] a través del bus SPI. Se utiliza para inicializar el bus a 500 kbps (`CAN.begin`) y para enviar y recibir las tramas crudas de 8 bytes (`beginPacket/write/endPacket`, `parsePacket/read`). Toda la lógica ISO-TP, OBD-II y UDS se implementa manualmente sobre estas primitivas.

Herramienta de diagnóstico (Raspberry Pi, Python 3)

La herramienta se ejecuta sobre Python 3 en Raspberry Pi OS, apoyándose en el subsistema SocketCAN del kernel Linux [13] para el acceso al bus a través del CAN HAT [11]. Sus dependencias (recogidas en `requirements.txt`) son:

- **python-can** [20]: proporciona la abstracción de alto nivel sobre la interfaz SocketCAN (`can.Bus`). Se usa como capa de acceso al bus por debajo de ISO-TP.
- **can-isotp** [21]: implementa el protocolo de transporte ISO 15765-2. Se usa (`isotp.CanStack`) para segmentar y reensamblar de forma transparente los mensajes que superan los 8 bytes de una trama CAN, gestionando *First/Consecutive Frames* y *Flow Control*.
- **bleess** [22]: servidor GATT BLE multiplataforma sobre `asyncio` y BlueZ. Se usa para publicar el servicio Nordic UART Service y atender las escrituras y notificaciones del cliente móvil.

La persistencia (`sqlite3`), la concurrencia (`threading`, `asyncio`) y el registro (`logging`) se resuelven con la biblioteca estándar de Python, sin dependencias adicionales.

Aplicación móvil (React Native)

La aplicación se construye con React Native sobre el *framework* Expo [23], lo que permite una única base de código para Android e iOS. Sus dependencias principales son:

- **react-native-ble-plx** [24]: cliente BLE nativo. Se usa para el escaneo, la conexión, la negociación de MTU, la escritura en la característica RX y la suscripción a notificaciones de la característica TX del servicio NUS.
- **zustand** [25]: gestión de estado global ligera. Se usa para los almacenes que mantienen el estado de conexión, la telemetría, los DTCs, las sesiones y la configuración.
- **@react-navigation** (`bottom-tabs`): navegación por pestañas de la interfaz.

- **@react-native-async-storage**: persistencia local de preferencias; **expo-file-system** y **expo-sharing**: exportación del registro de la aplicación.

5.3. Proceso de desarrollo iterativo

El sistema se construyó de forma incremental en cuatro fases, cada una verificable de forma independiente antes de añadir la siguiente capa de funcionalidad. Este enfoque permitió detectar problemas en la capa de transporte antes de añadir la lógica de protocolo, y validar la lógica de protocolo antes de integrar la interfaz de usuario.

5.3.1. Fase 1: Validación del enlace CAN

El primer paso fue verificar que los dos nodos hardware —Arduino MKR y Raspberry Pi con CAN HAT RS485— podían intercambiar tramas CAN correctamente antes de introducir ninguna lógica de protocolo.

Arduino — transmisión básica CAN



```

Output Serial Monitor X
Message (Enter to send message to 'Arduino MKR WAN 1310' on 'COM3')
10:02:47.220 -> CAN Link Test -- Arduino MKR
10:02:47.220 -> [OK] CAN bus at 500 kbps
10:02:47.220 -> [INFO] Waiting for frames...
10:02:47.220 ->
10:02:51.587 -> [RX] 0x7E0 len=8 | 12 23 34 45 56 67 78 89
10:02:51.626 -> [TX] 0x7E8 len=8 | 12 23 34 45 56 67 78 89
10:02:51.626 ->
  
```

Ilustración 5.2: Arduino IDE — monitor serie durante la fase de validación CAN. Se aprecia la confirmación de inicialización del bus ([OK] CAN Bus at 500 kbps) y el log de cada trama enviada.

El sketch inicial se apoyó en la librería **arduino-CAN** de Sandeep Mistry, que abstrae el controlador MCP2515 vía SPI. La inicialización del bus se reduce a una sola llamada, como se resume a continuación:

- `CAN.begin(CAN_SPEED)` inicializa el controlador a 500 kbps según la constante definida en `ecu_protocol.h` (`#define CAN_SPEED 500E3`).
- Cada trama se envía con `CAN.beginPacket(id)`, `CAN.write(data, len)` y `CAN.endPacket()`.
- El Serial a 115200 baud reporta el resultado de cada operación, lo que permite supervisar el comportamiento desde el IDE de Arduino.

Raspberry Pi — recepción y herramientas SocketCAN



```

gorka@gorka: ~
gorka@gorka:~$ cansend can0 7E0#1223344556677889
gorka@gorka:~$

gorka@gorka: ~
gorka@gorka:~$ candump can0
can0 7E0 [8] 12 23 34 45 56 67 78 89
can0 7E8 [8] 12 23 34 45 56 67 78 89
  
```

Ilustración 5.3: Sesión SSH en la Raspberry Pi mostrando la salida de `candump can0` y una petición manual con `cansend` durante la validación del enlace CAN.

En la Raspberry Pi, la interfaz CAN se levanta mediante el subsistema SocketCAN de Linux:

```
sudo ip link set can0 up type can bitrate 500000
```

Las utilidades `candump` y `cansend` del paquete `can-utils` permitieron verificar la recepción de tramas y enviar peticiones manuales sin necesidad de ningún código Python:

```
candump can0
cansend can0 7E0#0201050000000000
```

La salida de `candump` confirma el identificador, la longitud y los bytes de datos de cada trama recibida, lo que permitió verificar que la recepción sobre el controlador MCP2515 estaba correctamente configurada.

5.3.2. Fase 2: Primer prototipo — simulador ECU y CLI de diagnóstico

Ya validado el enlace CAN se desarrolla el primer prototipo funcional: el Arduino responde a peticiones OBD-II reales, y la Raspberry Pi script de python interactivo.

Arduino — respuesta a peticiones OBD-II

```
20:34:03.675 ->
20:34:03.675 -> ECU SIMULATOR v1
20:34:03.675 -> OBD-II / ISO-TP - Fase 2
20:34:03.675 ->
20:34:03.675 -> [OK] CAN at 500 kbps
20:34:03.675 -> [INFO] VIN: WWMZZZ1JZ3W386752
20:34:03.675 -> [INFO] Listen: 0x7E0
20:34:03.675 -> [INFO] Reply: 0x7E8
20:34:03.675 -> [INFO] Pre-loaded DTC: P0300
20:34:03.675 ->
20:48:48.487 ->
20:48:48.487 -> [RX] 0x7E0 Mode:0x1 PID:0xC
20:48:48.487 -> [TX SF] 0x7E8 | 04 41 0C 44 10 AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x5
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 05 91 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0xD
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 0D 29 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x11
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 11 C1 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x4
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 04 AD AA AA AA AA
20:48:56.763 ->
20:48:56.763 -> [RX] 0x7E0 Mode:0x9 PID:0x2
20:48:56.763 -> [INFO] VIN request - sending: WWMZZZ1JZ3W386752
20:48:56.763 -> [TX FF] VIN First Frame sent
20:48:56.763 -> [INFO] Waiting for FC...
20:48:56.763 -> [RX FC] Flow Control received
20:48:56.763 -> [TX CF] SN=0x1
20:48:56.853 -> [TX CF] SN=0x2
20:48:56.853 -> [INFO] VIN transmission complete
20:49:07.973 ->
20:49:07.973 -> [RX] 0x7E0 Mode:0x3 PID:0x2A
20:49:07.973 -> [TX SF] 0x7E8 | 04 43 01 03 00 AA AA AA
20:49:07.973 -> [INFO] Sent 1 DTC(s)
```

Ilustración 5.4: Monitor serie del Arduino durante la Fase 2: cada petición recibida ([RX]) y su respuesta ([TX SF]) se registran indicando modo, PID y bytes de datos.

Se incorpora la función `processCANMessages()` que analiza el modo de servicio (segundo byte de datos, tras el byte PCI de ISO-TP) y el PID solicitado para construir la respuesta en el formato ISO-TP Single Frame. Los modos implementados en esta fase fueron 0x01 (datos en vivo, PIDs básicos) y 0x03 (lectura de DTCs preconfigurados).

Además se añadió el modo 0x09 (información del vehículo), cuyo PID 0x02 devuelve el VIN de 17 caracteres. Al superar la respuesta los 7 bytes útiles de un Single Frame, la función `sendVINMultiFrame()` implementa la transmisión ISO-TP multi-frame del lado emisor: envía primero un First Frame (FF, PCI 0x1X) con la longitud total de la carga, espera el Flow Control del escáner mediante `waitForFlowControl()` y a continuación emite los Consecutive Frames (CF, PCI 0x2X) con número de secuencia incremental y una separación temporal entre tramas. Es la primera segmentación manual del protocolo en el Arduino, ya que las librerías ISO-TP residen solo en el lado de la Raspberry Pi.

La constante `ECU_CAN_ID = 0x7E0` define el identificador de petición escuchado, y `ECU_RESPONSE_ID = 0x7E8` el identificador de respuesta emitido, ambos definidos en `ecu_protocol.h` en concordancia con el direccionamiento físico OBD-II sobre CAN.

Raspberry Pi — pila ISO-TP y CLI

```
(.venv) gorka@gorka:~/TFT-CAN-BLE-system/src $ python scripts/cli.py

OBD-II Diagnostic Tool - Fase 2
Transport: IsoTpTransport (can0)

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 1
[20:48:48]
RPM          : 4356.00 rpm
Coolant Temp  : 105.00 °C
Speed         : 41.00 km/h
Throttle Pos  : 75.69 %
Engine Load   : 67.84 %

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 5
[20:48:56]
VIN: WVVZZZ1JZ3W386752

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 3
[20:49:07]
[P0300]

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option:
```

Ilustración 5.5: CLI de diagnóstico (`cli.py`) en la Fase 2: menú interactivo con opciones de live data y lectura de DTCs, ejecutado en la Raspberry Pi vía SSH.

En la Raspberry Pi se introdujeron las librerías `python-can` [20] y `can-isotp` [21], que gestionan respectivamente el acceso al bus SocketCAN y la segmentación/reensamblado ISO-

TP. La clase `IsoTpTransport` encapsula ambas en la interfaz `ITransport` del sistema:

```

1 # Identificadores escáner (0x7E0) y ECU (0x7E8) en modo Normal 11 bits
2 self._address = isotp.Address(
3     addressing_mode=isotp.AddressingMode.Normal_11bits,
4     txid=tx_id,      # 0x7E0 -- peticiones del escáner
5     rxid=rx_id,      # 0x7E8 -- respuestas de la ECU
6 )
7
8 # CanStack gestiona de forma automática los Flow Control frames
9 # y el reensamblado de las respuestas multi-frame
10 self._stack = isotp.CanStack(self._bus, address=self._address,
11                               params=self._params)
12
13 # El timeout de recepción absorbe la latencia de la transmisión
14 # multi-frame sin generar falsos errores de comunicación
15 deadline = time.monotonic() + self._timeout    # timeout = 5.0 s
16 while time.monotonic() < deadline:
17     self._stack.process()
18     if self._stack.available():
19         return self._stack.recv()

```

Listado 5.1: Configuración de la pila ISO-TP en `IsoTpTransport`

Sobre este transporte se desarrolló `cli.py`, un menú interactivo que permitió ejecutar consultas OBD-II desde la línea de comandos sin necesidad de la aplicación móvil. El menú inicial ofrecía operaciones como:

- **Opción 1 — Live data:** consulta instantánea de los cinco PIDs más relevantes (temperatura del refrigerante, carga, RPM, velocidad y posición del acelerador).
- **Opción 3 — Read DTCs:** lectura de la lista de códigos de avería almacenados en la ECU simulada.

5.3.3. Fase 3: Simulación avanzada, monitorización y persistencia

La tercera fase añadió varias mejoras simultáneas: el simulador Arduino incorporó interrupciones hardware y los servicios UDS (ISO 14229-1), y la herramienta Python extendió el CLI con monitorización continua, logging en SQLite y un cliente UDS.

Arduino — interrupciones hardware, servicios UDS y ruido opcional

Se implementaron dos interrupciones hardware para simular eventos espontáneos del vehículo. Ambas siguen el patrón de ISR mínima con histéresis temporal por `millis()` (300 ms) para el antirrebote, y la lógica de efecto se ejecuta fuera de la ISR, en el bucle principal, donde es seguro realizar operaciones bloqueantes (`Serial`, acceso al bus CAN):

- **Pin D5 (ignición, ENGINE_START_PIN):** la ISR `onIgnitionPress()` incrementa el contador `ignitionFlag`; `handleIgnitionButton()` conmuta el estado del motor y fija las RPM al ralentí (850 rpm) o a cero.
- **Pin D4 (fallo DTC, DTC_FAULT_PIN):** la ISR `onDTCPress()` incrementa `dtcFlag`; `handleDTCButton()` inyecta secuencialmente uno de los tres códigos del conjunto `DTC_FAULT_SET`

```

08:31:57.968 -> =====
08:31:57.968 ->     GENERIC ECU SIMULATOR
08:31:57.968 ->     ISO-TP (ISO 15765-2)
08:31:57.968 -> =====
08:31:57.968 -> [OK] CAN Bus at 500.00 kbps
08:31:57.968 -> [OK] Ignition button on pin D5
08:31:57.968 -> [OK] DTC fault button on pin D4
08:31:57.968 -> [INFO] VIN: ARDUINO000000000000
08:31:57.968 -> [INFO] Listen: 0x7E0
08:31:57.968 -> [INFO] Reply: 0x7E8
08:31:57.968 -> =====
08:31:57.968 ->
08:32:01.100 -> [BC TX] 0x3D0 | 3C 00 30 A2 00 00 00 00
08:32:03.697 ->
08:32:03.697 -> [RX] 0x7E0 Mode:0x3 PID:0xAA
08:32:03.697 -> [TX SF] 0x7E8 | 02 43 00 AA AA AA AA AA
08:32:03.697 -> [INFO] Sent 0 DTCs
08:32:07.520 -> [DTC] Fault injected - active DTCs: 1
08:32:09.963 -> [DTC] Fault injected - active DTCs: 2
08:32:11.137 -> [BC TX] 0x3D0 | 3C 00 30 A2 00 00 00 00
08:32:16.226 ->
08:32:16.226 -> [RX] 0x7E0 Mode:0x3 PID:0xAA
08:32:16.265 -> [TX SF] 0x7E8 | 06 43 02 03 00 01 71 AA
08:32:16.265 -> [INFO] Sent 2 DTCs
08:32:21.153 -> [BC TX] 0x3D0 | 3C 00 30 A2 00 00 00 00
08:32:22.245 ->
08:32:22.245 -> [RX] 0x7E0 Mode:0x22 PID:0xF1
08:32:22.277 -> [TX FF] 0x7E8 | 10 14 62 F1 90 41 52 44
08:32:22.277 -> [INFO] Waiting for FC...
08:32:22.277 -> [RX FC] flag=0x0
08:32:22.277 -> [TX CF] SN=0x1 | 21 55 49 4E 4F 30 30 30
08:32:22.277 -> [TX CF] SN=0x2 | 22 30 30 30 30 30 30 30
08:32:22.308 -> [UDS] DID 0xF190 -> 17 bytes
08:32:22.308 ->
08:32:22.308 -> [RX] 0x7E0 Mode:0x22 PID:0xF1
08:32:22.308 -> [TX FF] 0x7E8 | 10 0B 62 F1 8C 45 43 55
08:32:22.308 -> [INFO] Waiting for FC...
08:32:22.308 -> [RX FC] flag=0x0
08:32:22.308 -> [TX CF] SN=0x1 | 21 30 30 30 30 31 AA AA
08:32:22.308 -> [UDS] DID 0xF18C -> 8 bytes
08:32:22.308 ->
08:32:22.308 -> [RX] 0x7E0 Mode:0x22 PID:0xF1
08:32:22.346 -> [TX FF] 0x7E8 | 10 09 62 F1 89 56 31 2E
08:32:22.346 -> [INFO] Waiting for FC...
08:32:22.346 -> [RX FC] flag=0x0
08:32:22.346 -> [TX CF] SN=0x1 | 21 30 2E 30 AA AA AA AA
08:32:22.346 -> [UDS] DID 0xF189 -> 6 bytes

```

Ilustración 5.6: Monitor serie del Arduino en la Fase 3: se aprecian los eventos de ignición ([IGN] Key ON - engine starting) y la inyección de DTCs por interrupción hardware ([DTC] Fault injected).

(P0300, P0171, P0420) en `dtcList[]` y activa el testigo `checkEngine`. Una cuarta pulsación, cuando ya hay tres averías activas, borra todas.

Adicionalmente, el simulador admite dos mecanismos opcionales de variabilidad, controlados por sendas macros en `ecu_protocol.h` y **desactivados por defecto** para obtener un bus determinista durante la depuración:

- **ADD_NOISE** (por defecto 0): cuando se activa, añade *jitter* pseudoaleatorio a los valores de los sensores en cada ciclo de actualización (RPM, temperaturas, MAF, tensión de batería y *fuel trim*), con amplitudes por sensor parametrizadas (`NOISE_RPM_MAX`, etc.).
- **BROADCAST_ENABLE** (por defecto 0): cuando se activa, el simulador emite cada `BROADCAST_INTERVAL_M` (100 ms) tres tramas CAN no solicitadas con identificadores `0x280` (RPM/carga), `0x320` (velocidad) y `0x3D0` (refrigerante/acelerador/batería), imitando el tráfico periódico de una ECU real.

El simulador incorporó además dos servicios UDS (ISO 14229-1) sobre la misma pila ISO-TP: *DiagnosticSessionControl* (`0x10`), que conmuta entre la sesión por defecto y la extendida, y *ReadDataByIdentifier* (`0x22`), que expone DID estándar (VIN, número de serie y versión de software) y DID propietarios de telemetría accesibles solo en sesión extendida. Su implementación se detalla en el apartado 5.4.3.

Raspberry Pi — monitorización continua, SQLite y cliente UDS

La herramienta Python extendió el CLI con tres nuevas capacidades:

Monitorización continua (`live_data_monitor.py`): la clase `LiveDataMonitor` ejecuta un hilo *daemon* que itera sobre el conjunto de PIDs configurado, envía cada petición ISO-TP, decodifica la respuesta e invoca un callback `on_sample` con un objeto `MonitorSample` (PID, nombre, valor, unidad y *timestamp* monotónico) por cada PID decodificado con éxito. Los errores se notifican por separado mediante el callback opcional `on_error(pid, exc)`. El periodo de muestreo por defecto es 500 ms. El hilo se inicia y detiene a través de los métodos `start()` y `stop()`, o mediante el protocolo de contexto `with`:

```
with monitor:
    input("Press Enter to stop")
```

Logging SQLite (`sqlite_logger.py`): `SqliteDataLogger` almacena cada muestra en la tabla `samples` (PID, nombre, valor, unidad, timestamp monotónico y timestamp de pared ISO-8601) dentro de una base de datos SQLite en modo WAL, que permite escrituras concurrentes desde el hilo de monitorización sin bloqueos. Las tramas CAN en hexadecimal de cada comando se registran por separado en la tabla `commands`.

Cliente UDS (`uds_session.py`): la clase `UdsSession` envuelve el `ITransport` con `UdsProtocolBuilder` y `UdsDataDecoder` para exponer dos operaciones de alto nivel del estándar ISO 14229-1: `set_session()`, que emite *DiagnosticSessionControl* (servicio `0x10`) y registra la sesión activa, y `read_did()`, que emite *ReadDataByIdentifier* (servicio `0x22`) y decodifica el valor según el registro de DIDs (`uds_dids.py`). Este registro contiene tres DID estándar (`0xF190` VIN, `0xF18C` número de serie, `0xF189` versión de software) y ocho DID propietarios de da-

tos en vivo (0x2001–0x2008, marcados `extended_only`). El cliente verifica la sesión antes de enviar: si se solicita un DID propietario fuera de la sesión extendida, rechaza la petición localmente con NRC 0x7E sin acceder al bus.

El CLI añade la opción 6 (*Live monitor*) que arranca el monitor y actualiza la pantalla en tiempo real, la opción 2 (*Extended live data*) que interroga todos los PIDs disponibles en una sola pasada, y las opciones 7–9 para UDS: cambio de sesión (*change session*), lectura de los DID estándar y lectura de los DID extendidos.

```
(.venv) gorka@gorka:~/TFT-CAN-BLE-system/src $ python scripts/cli.py
VAG Group OBD-II Diagnostic Tool
Transport: IsoTpTransport (can0)
Logging: diagnostics.db

[LOG] Session #71 → /home/gorka/TFT-CAN-BLE-system/diagnostics.db

UDS session: default
[1] Live data (5 core PIDs, single snapshot)
[2] Extended live data (all PIDs, skips unsupported)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[6] Live monitor (5 core PIDs, continuous)
[7] UDS - change session
[8] UDS - read standard DIDs (VIN / Serial / SW)
[9] UDS - read extended DIDs (requires extended session)
[0] Exit

Select option: 3
08:32:03 TX - [0BD2]
RAW : 03
DECODED: Mode 03 → ReadDTCs
08:32:03 RX - [0BD2]
RAW : 43 00
DECODED: ReadDTCs response → 0 DTC(s)

No DTCs stored.

UDS session: default
[1] Live data (5 core PIDs, single snapshot)
[2] Extended live data (all PIDs, skips unsupported)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[6] Live monitor (5 core PIDs, continuous)
[7] UDS - change session
[8] UDS - read standard DIDs (VIN / Serial / SW)
[9] UDS - read extended DIDs (requires extended session)
[0] Exit

Select option: 3
08:32:16 TX - [0BD2]
RAW : 03
DECODED: Mode 03 → ReadDTCs
08:32:16 RX - [0BD2]
RAW : 43 02 03 00 01 71
DECODED: ReadDTCs response → 2 DTC(s)

[P0300] Se detectan fallos de encendido aleatorios/múltiples cilindros
[P0171] Sistema demasiado pobre Banco 1

UDS session: default
[1] Live data (5 core PIDs, single snapshot)
[2] Extended live data (all PIDs, skips unsupported)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[6] Live monitor (5 core PIDs, continuous)
[7] UDS - change session
[8] UDS - read standard DIDs (VIN / Serial / SW)
[9] UDS - read extended DIDs (requires extended session)
[0] Exit

Select option: 8
08:32:22 TX - [UDS]
RAW : 22 F1 90
DECODED: ReadDataByIdentifier DID=0xF190
```

Ilustración 5.7: CLI en modo de monitorización continua (opción 6): actualización periódica de los cinco PIDs principales con timestamp, valores decodificados y unidades, ejecutado vía SSH en la Raspberry Pi.

5.3.4. Fase 4: Integración BLE y validación en vehículo real

La fase final añadió el canal de comunicación inalámbrico entre la Raspberry Pi y el teléfono móvil, y se comenzó con las pruebas en vehiculos reales.

Raspberry Pi — servidor BLE GATT

El servidor BLE se implementó en `bluetooth_server.py` mediante la librería `bless`, que expone una interfaz asíncrona (`asyncio`) sobre el stack BlueZ de Linux. El servidor anuncia el **Nordic UART Service (NUS)** con el nombre `diag_tool` y sus dos características estándar:

- **RX** (6E400002-...): característica de escritura. Recibe los comandos JSON del cliente móvil.
- **TX** (6E400003-...): característica de notificación. Envía las respuestas JSON al cliente.

El manejador `_on_write()` acumula los *chunks* recibidos en un buffer hasta encontrar el delimitador `\n`, deserializa el JSON y despacha el comando al `BtCommandHandler`, que orquesta la sesión de diagnóstico y devuelve el resultado como objeto JSON serializado. Las respuestas que superan el MTU del servidor (240 bytes) se fragmentan automáticamente en múltiples notificaciones.

El servidor implementa un mecanismo de *watchdog* que detecta la desconexión del cliente (inactividad superior a 15 s en RX o 20 s en TX) y reinicia el estado de sesión, y un *heartbeat* periódico cada 8 s para mantener el enlace activo durante sesiones de monitorización prolongadas.

```
(.venv) gorka@gorka:~/TFT-CAN-BLE-system $ python src/scripts/server.py

diag_tool - BLE OBD-II Server
Transport: IsoTpTransport (can0)
Logging: diagnostics.db + ble_comms.log

[BLE] Auth token loaded from: default
[LOG] Session #75 -> /home/gorka/TFT-CAN-BLE-system/diagnostics.db
09:05:13 [INFO] can.interfaces.socketcan.socketcan: Created a socket
[BLE] Advertising 'diag_tool' - waiting for connection...
09:05:13 [INFO] server.bluetooth_server: [BLE] Advertising 'diag_tool'
[BLE] write callback: uuid='6e400002-b5a3-f393-e0a9-e50e24dcca9e' len=30
[BLE] Client connected - first write received
09:05:46 [INFO] server.bluetooth_server: [BLE] Client connected
09:05:46 [INFO] server.bluetooth_server: RX - [BLE]
RAW : {"cmd":"auth","token":"1234"}
DECODED: command='auth'
RX - [BLE]
RAW : {"cmd":"auth","token":"1234"}
DECODED: command='auth'
09:05:46 [INFO] server.bluetooth_server: TX - [BLE]
RAW : {"status":"ok","data":"authenticated"}
DECODED: status='ok' data='authenticated'
TX - [BLE]
RAW : {"status":"ok","data":"authenticated"}
DECODED: status='ok' data='authenticated'
[BLE] write callback: uuid='6e400002-b5a3-f393-e0a9-e50e24dcca9e' len=14
09:05:46 [INFO] server.bluetooth_server: RX - [BLE]
RAW : {"cmd":"vin"}
DECODED: command='vin'
RX - [BLE]
RAW : {"cmd":"vin"}
DECODED: command='vin'
09:05:46 TX - [OBD2]
RAW : 09 02
DECODED: Mode 09 -> VIN
[BLE] write callback: uuid='6e400002-b5a3-f393-e0a9-e50e24dcca9e' len=21
09:05:46 [INFO] server.bluetooth_server: RX - [BLE]
RAW : {"cmd":"probe_pids"}
DECODED: command='probe_pids'
RX - [BLE]
RAW : {"cmd":"probe_pids"}
DECODED: command='probe_pids'
09:05:46 RX - [OBD2]
RAW : 49 02 01 41 52 44 55 49 4E 4F 30 30 30 30 30 30 30 30
DECODED: Mode 09 response -> VIN=ARDUINO0000000000
09:05:46 TX - [OBD2]
RAW : 01 00
DECODED: Mode 01 request -> PID 0x00 (PID 0x00)
09:05:46 [INFO] server.bluetooth_server: TX - [BLE]
RAW : {"status":"ok","data":"ARDUINO0000000000"}
```

Ilustración 5.8: Consola de la Raspberry Pi ejecutando el servidor BLE: anuncio del servicio `diag_tool` y registro de los comandos JSON recibidos (RX) y respuestas enviadas (TX).

Protocolo de comunicación JSON

Los comandos intercambiados entre la aplicación y la Raspberry Pi siguen un protocolo JSON delimitado por saltos de línea (NDJSON) sobre NUS. Los comandos principales son:

- `{"cmd": "auth", "token": "..."}:` autenticación de sesión (primer comando obligatorio).
- `{"cmd": "snapshot"}:` lectura puntual de todos los PIDs soportados del modo 0x01.
- `{"cmd": "dtcs"} / {"cmd": "clear_dtcs"} / {"cmd": "vin"}:` gestión de averías y lectura del VIN.
- `{"cmd": "monitor_start", "pids": [12,13,5]} / {"cmd": "monitor_stop"}:` inicio y parada de monitorización continua con notificaciones periódicas.
- `{"cmd": "sessions"} / {"cmd": "session_samples", "session_id": N}:` consulta del historial SQLite.
- `{"cmd": "uds_session", "session_type": 3} / {"cmd": "uds_read_did", "did": "0xF190"}:` control de sesión UDS y lectura de DIDs.

- {"cmd": "probe_pids"}: descubrimiento automático de los PIDs soportados por la ECU.

Aplicación móvil — React Native + react-native-ble-plx

La aplicación móvil implementa la comunicación BLE mediante la librería `react-native-ble-plx`, que proporciona acceso al stack BLE nativo de Android e iOS. Las operaciones clave son:

- **Escaneo y conexión:** `manager.startDeviceScan(null, ...)` enumera todos los dispositivos BLE cercanos (sin filtro de UUID) y el usuario selecciona el adaptador, que la Raspberry Pi anuncia con el nombre `diag_tool`. `connectToDevice()` establece el enlace físico, a continuación se negocia el MTU, se descubren servicios y se activa la suscripción a notificaciones TX.
- **Envío de comandos:** `writeCharacteristicWithResponse()` garantiza la entrega del JSON de comando antes de esperar la respuesta.
- **Recepción de notificaciones:** `monitorCharacteristicForService()` invoca un callback con cada fragmento recibido, el adaptador los reasambla por líneas (NDJSON) antes de procesar el JSON.

La Figura 5.9 muestra las pantallas principales de la aplicación durante las pruebas de integración BLE.

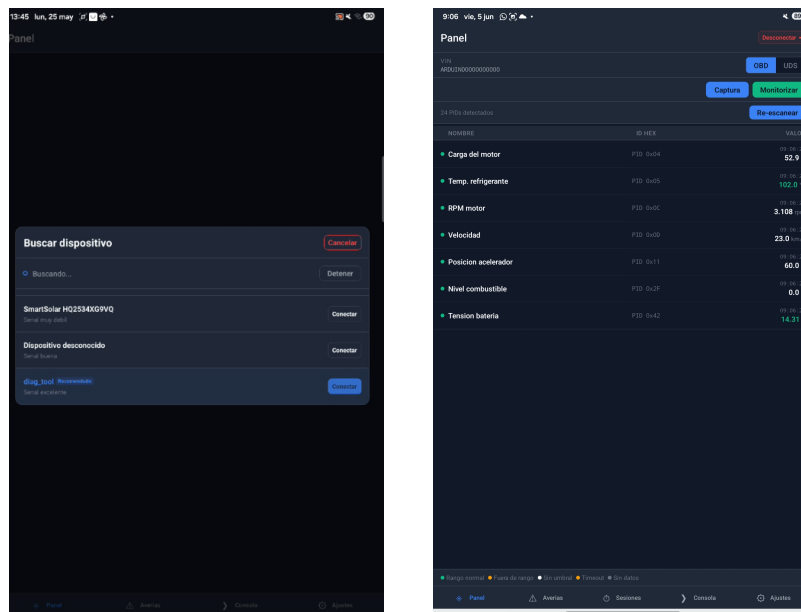
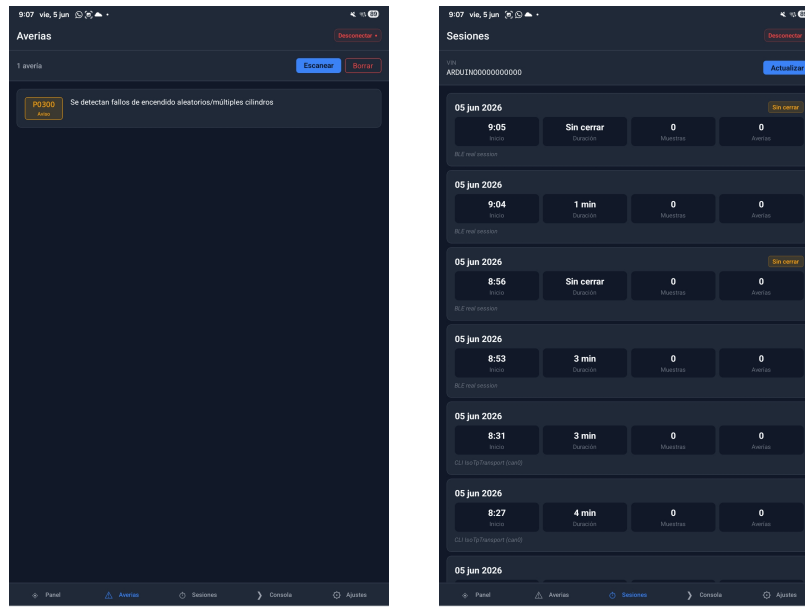


Ilustración 5.9: *Conexión* (izquierda) y *Panel* (derecha)

Ilustración 5.10: *Averías* (izquierda) y *Sesiones* (derecha)

5.4. Simulador de ECU — Arduino MKR

El simulador de ECU es el componente que permite validar la herramienta de diagnóstico de forma independiente de un vehículo real. Se implementa sobre un Arduino MKR con shield CAN basado en el controlador MCP2515, conectado al bus CAN a 500 kbps. El código fuente completo del simulador está disponible en el repositorio del proyecto.

5.4.1. Hardware y configuración del bus CAN

El Arduino MKR opera a 3,3V y dispone de comunicación SPI para el controlador MCP2515. La librería `arduino-CAN` inicializa el controlador a 500 kbps. El simulador acepta únicamente las tramas dirigidas a la dirección física de la ECU (0x7E0), descartando en software cualquier otro identificador recibido.

Las constantes de configuración del protocolo se concentran en el fichero `ecu_protocol.h`, separando la parametrización del código de lógica. Entre los identificadores definidos destacan:

- `ECU_CAN_ID = 0x7E0`: identificador escuchado (peticiones del escáner)
- `ECU_RESPONSE_ID = 0x7E8`: identificador de respuesta de la ECU
- `ISOTP_CF_SEP_MS = 25 ms`: separación temporal entre *Consecutive Frames* consecutivos
- `ISOTP_FC_TIMEOUT_MS = 1000 ms`: tiempo máximo de espera del *Flow Control* del escáner
- `ISOTP_PADDING_BYTE = 0xAA`: byte de relleno hasta los 8 bytes de la trama CAN

5.4.2. Modelo físico del motor

Para que los datos de telemetría devueltos por el simulador presenten un comportamiento realista y dinámico, se implementa un modelo físico simplificado del motor que actualiza sus variables de estado en cada iteración del bucle principal.

Las variables del modelo físico son:

- **Posición del acelerador:** actúa como entrada del modelo y evoluciona mediante una máquina de estados de rampa automática (ralentí $\rightarrow$ subida $\rightarrow$ crucero $\rightarrow$ bajada $\rightarrow$ repetir), con incrementos de `RAMP_STEP` (5 %) por ciclo hasta un máximo de `RAMP_MAX_THROTTLE` (65 %) y permanencias configurables en cada extremo (~ 3 s).
- **RPM:** sigue un retardo de primer orden (filtro exponencial con factor α) hacia un objetivo proporcional al acelerador, calculado como `map(throttle, 0-100, 850-5500)`, y se acota finalmente al rango 750–6500 rpm.
- **Velocidad:** integra un modelo longitudinal simplificado en el que una fuerza de tracción (proporcional a $\text{RPM} \times \text{carga}$) compite con una resistencia cuadrática (proporcional a la velocidad al cuadrado), acotada a 0–180 km/h.
- **Carga del motor:** derivada del acelerador mediante `map(throttle, 0-100, 15-85) %`. A partir de ella se calculan el caudal MAF, el avance de encendido y las presiones de combustible y admisión.
- **Temperaturas de refrigerante y aceite:** siguen una curva de calentamiento incremental desde la temperatura ambiente hasta su régimen de operación ($\sim 90^\circ\text{C}$ y $\sim 95^\circ\text{C}$), y se enfrían progresivamente con el motor apagado.
- **Tensión de batería y nivel de combustible:** la tensión sube con el motor en marcha (alternador) y cae en reposo; el combustible se consume a un ritmo dependiente de la carga.

El modelo se actualiza cada `SIM_UPDATE_INTERVAL_DEFAULT` (200 ms) en el bucle principal, y los valores calculados se codifican directamente en los bytes de respuesta OBD-II según las fórmulas del estándar SAE J1979 para cada PID.

5.4.3. Implementación de los modos OBD-II y servicios UDS

La función central `processCANMessages()` analiza el byte de modo de servicio de cada trama recibida y despacha al manejador correspondiente; en los modos 0x01 y 0x09 utiliza además el byte de PID.

Modo 0x01 — Datos en vivo

Se implementan 25 PIDs de datos. La respuesta se construye en un buffer de 8 bytes con el formato OBD-II estándar: byte de longitud ISO-TP (tipo SF + longitud), byte de modo de respuesta (0x41), byte de PID y bytes de datos codificados.

La decodificación inversa sigue las fórmulas del estándar para cada PID. Como ejemplo, el PID 0x0C (RPM) codifica el valor en dos bytes según:

$$\text{valor raw} = \text{RPM} \times 4 \quad \Rightarrow \quad \text{byte_A} = \lfloor \text{valor raw} / 256 \rfloor, \quad \text{byte_B} = \text{valor raw} \bmod 256$$

Los PIDs de soporte 0x00, 0x20 y 0x40 devuelven cada uno una máscara de bits de 32 bits que indica qué PIDs del rango siguiente están disponibles (el bit menos significativo señala la existencia del siguiente bloque), permitiendo a la herramienta de diagnóstico realizar una enumeración automática de capacidades a lo largo de los tres rangos.

Modo 0x03 — Lectura de DTCs

El simulador mantiene una lista de DTCs activos (`dtcList[]`) con un contador `numStoredDTCs`. La respuesta del modo 0x03 incluye la cuenta de DTCs seguida de los pares de bytes correspondientes a cada código, codificados según el formato J2012: los dos bits más significativos del primer byte indican la categoría (00=Powertrain, 01=Chassis, 10=Body, 11=User (Network)) y los 14 bits restantes el código numérico.

Modo 0x04 — Borrado de DTCs

Al recibir una petición de modo 0x04, el simulador reinicia `numStoredDTCs` a cero, desactiva el flag `checkEngine` y responde con el byte de confirmación 0x44.

Modo 0x09 — Información del vehículo (VIN)

El PID 0x02 del modo 0x09 devuelve el VIN del vehículo, una cadena ASCII de 17 caracteres. Dado que la respuesta completa ocupa 20 bytes (3 de cabecera más 17 del VIN), supera la capacidad de una Single Frame y requiere transmisión multi-frame.

La secuencia de transmisión es:

1. **First Frame:** PCI 0x10 0x14 (longitud = 20 bytes), modo 0x49, PID 0x02, contador 0x01 y primeros 3 bytes del VIN.
2. Espera del **Flow Control** del escáner.
3. **Consecutive Frame 1:** SN 0x21, bytes 4–10 del VIN.
4. **Consecutive Frame 2:** SN 0x22, bytes 11–17 del VIN (con padding).

El Algoritmo 1 detalla la generación de esta secuencia en el simulador.

Esta implementación manual de la secuencia multi-frame en el simulador fue clave, dado que cualquier error en el contador de secuencia o en el timing entre tramas provocaba el rechazo de la sesión por parte de la librería `can-isotp` en la Raspberry Pi.

Servicios UDS (ISO 14229-1)

Además de los cuatro modos OBD-II, el simulador implementa dos servicios UDS que permiten un diagnóstico más rico que el conjunto de PIDs estándar:

Algoritmo 1 Transmisión ISO-TP multi-frame del VIN en el simulador

```

1:  $payload \leftarrow [0x49, 0x02, 0x01] \parallel VIN$  ▷ 20 bytes
2:  $len \leftarrow |payload|$ 
3:  $FF[0] \leftarrow 0x10 \vee (len \gg 8)$ ;  $FF[1] \leftarrow len \wedge 0xFF$ 
4: copiar  $payload[0..5]$  en  $FF[2..7]$ ; ENVIARCAN( $0x7E8$ ,  $FF$ )
5: if not ESPERARFLOWCONTROL then ▷ timeout 1 s
6:     registrar aviso y continuar
7: end if
8:  $enviados \leftarrow 6$ ;  $SN \leftarrow 1$ 
9: while  $enviados < len$  do
10:     $CF[0] \leftarrow 0x20 \vee (SN \wedge 0x0F)$ 
11:    copiar los siguientes 7 bytes (rellenando con  $0xAA$ ) en  $CF[1..7]$ 
12:    ENVIARCAN( $0x7E8$ ,  $CF$ )
13:     $enviados \leftarrow enviados + 7$ ;  $SN \leftarrow (SN + 1) \wedge 0x0F$ 
14:    ESPERAR(ISOTP_CF_SEP_MS) ▷ 25 ms entre Consecutive Frames
15: end while

```

- **Servicio 0x10 — DiagnosticSessionControl:** `handleMode10()` admite las subfunciones `0x01` (sesión por defecto) y `0x03` (sesión extendida). La respuesta positiva (`0x50`) incluye los tiempos de servidor `P2` (25 ms) y `P2*` (500 ms). Cualquier otra subfunción se rechaza con NRC `0x12`.
- **Servicio 0x22 — ReadDataByIdentifier:** `handleMode22()` responde a los DID estándar `0xF190` (VIN), `0xF18C` (número de serie de la ECU) y `0xF189` (versión de software), accesibles en cualquier sesión, y a los DID propietarios de telemetría `0x2001–0x2008` (carga, refrigerante, RPM, velocidad, acelerador, combustible, aceite y batería), accesibles **solo en sesión extendida**. Una petición de un DID propietario fuera de la sesión extendida se rechaza con NRC `0x22` (*conditionsNotCorrect*), y un DID desconocido con NRC `0x31` (*requestOutOfRange*). Las respuestas que superan 7 bytes (como el VIN) se transmiten mediante la secuencia ISO-TP multi-frame genérica `sendMultiFrame()`.

5.4.4. Mecanismos opcionales de variabilidad del bus

Para poder validar la herramienta de diagnóstico tanto en condiciones ideales como realistas, el simulador puede operar de forma **determinista** (configuración por defecto, bus silencioso salvo las respuestas a peticiones) o activar dos mecanismos opcionales de variabilidad mediante macros de compilación en `ecu_protocol.h`.

Ruido en los sensores (ADD_NOISE)

Cuando `ADD_NOISE` se compila a 1 (por defecto 0), el simulador añade *jitter* pseudoaleatorio a los valores de los sensores en cada ciclo de actualización. La amplitud máxima de cada sensor se parametriza de forma independiente mediante constantes (`NOISE_RPM_MAX = 25 rpm`, `NOISE_VOLTAGE_MAX = 30 mV`, `NOISE_MAF_MAX = 8`, etc.). Tras añadir el ruido, cada variable se acota a su rango físico válido, de modo que la telemetría conserva un compor-

tamiento creíble pero no perfectamente estable, reproduciendo la lectura real de sensores analógicos.

Tramas de difusión (BROADCAST_ENABLE)

Cuando `BROADCAST_ENABLE` se compila a 1 (por defecto 0), el simulador emite cada `BROADCAST_INTERVAL_MS` (100 ms) tres tramas CAN no solicitadas que imitan el tráfico periódico de una ECU real: `0x280` (RPM y carga del motor), `0x320` (velocidad) y `0x3D0` (temperatura de refrigerante, acelerador y tensión de batería). Este tráfico de fondo permite verificar que el filtro de la herramienta de diagnóstico descarta correctamente los identificadores ajenos a la dirección de respuesta de la ECU (`0x7E8`) y no los interpreta como respuestas válidas.

5.4.5. Inyección de DTCs por interrupción hardware

Para simular la aparición espontánea de averías durante una sesión de diagnóstico, el simulador implementa un mecanismo de inyección de DTCs activado por la señal en el pin digital D4 (`DTC_FAULT_PIN`).

El patrón de implementación sigue la práctica estándar para ISR en Arduino:

1. La ISR `onDTCPress()` es minimalista: aplica una histéresis temporal de 300 ms (`DTC_HYSTERESIS_MS`) mediante `millis()` para el prevenir el rebote e incrementa el contador `volatile uint16_t dtcFlag`.
2. La inyección propiamente dicha se realiza en `handleDTCButton()`, llamada desde el bucle principal, donde es seguro ejecutar operaciones con efectos de bloqueo (`Serial`).
3. Cada pulsación inserta secuencialmente el siguiente código del conjunto `DTC_FAULT_SET` (`P0300` (fallo de encendido), `P0171` (mezcla pobre) y `P0420` (eficiencia del catalizador)) en `dtcList[]` y activa el testigo `checkEngine`.
4. Cuando ya hay tres averías activas, una pulsación adicional borra todas, permitiendo reiniciar el ciclo de prueba.

Esta funcionalidad permite al operador del banco de pruebas introducir averías de forma controlada durante una sesión de diagnóstico activa y verificar que la herramienta Python las detecta correctamente en la siguiente consulta del modo `0x03`.

5.5. Herramienta de diagnóstico — Raspberry Pi

La herramienta de diagnóstico es el componente central del sistema: recibe peticiones desde la aplicación móvil vía BLE, las traduce a secuencias ISO-TP sobre el bus CAN, interpreta las respuestas de la ECU, almacena los datos en SQLite y devuelve los resultados al cliente móvil. Se implementa en Python 3 sobre Raspberry Pi con controlador CAN MCP2515. El código fuente está disponible en el repositorio del proyecto.

5.5.1. Arquitectura software

La herramienta sigue un diseño basado en interfaces y capas desacopladas mediante inyección de dependencias. Esta decisión, supuso una inversión inicial de tiempo mayor a la contemplada en el diseño rápido pero resultó acertada pues en las fases posteriores facilitó las pruebas sin hardware y la extensión progresiva del sistema.

Las interfaces abstractas principales son:

- **ITransport**: define las operaciones de conexión y de envío/recepción de mensajes ISO-TP (`connect`, `disconnect`, `send`, `receive`). Las implementaciones concretas son `IsoTpTransport` (bus CAN real mediante `python-can` + `can-isotp` sobre `SocketCAN`) y `MockTransport` (respuestas predefinidas para pruebas sin hardware).
- **IProtocolBuilder**: define la construcción de las tramas de petición OBD-II mediante métodos específicos por operación (`build_read_rpm_request`, `build_read_dtcs_request`, `build_read_vin_request`, etc.). La implementación `Obd2ProtocolBuilder` las genera según el estándar SAE J1979.
- **IDataDecoder**: valida la trama de respuesta (`validate_response`, que detecta respuestas negativas NRC) y la decodifica por parámetro (`decode_rpm`, `decode_coolant_temp`, `decode_dtcs`, `decode_vin`, etc.). La implementación `Obd2DataDecoder` aplica las fórmulas del estándar para cada PID y devuelve valores con unidades.
- **IDiagnosticSession**: define la interfaz de sesión de diagnóstico (`get_engine_rpm`, `get_dtcs`, `clear_dtcs`, `get_vin`, `get_snapshot`, etc.). La implementación `DiagnosticSession` orquesta las demás interfaces ejecutando la secuencia enviar → recibir → validar → decodificar.

Sobre esta estructura base se aplica el patrón decorador en dos niveles. **LoggingTransport** envuelve cualquier **ITransport** e intercepta todas las operaciones de envío y recepción para volcarlas al registro textual de comunicaciones (`ble_comms.log`) y mostrar la última trama enviada/recibida. A su vez, **LoggedDiagnosticSession** envuelve la `DiagnosticSession` y, tras cada operación, almacena en la base de datos SQLite el comando ejecutado junto con sus tramas de petición y respuesta (y los DTCs leídos). La composición resultante es:

```
LoggedDiagnosticSession(DiagnosticSession(LoggingTransport(IsoTpTransport),
                                           Obd2ProtocolBuilder, Obd2DataDecoder),
                        SqliteDataLogger)
```

El núcleo funcional de la herramienta es el ciclo de consulta de un PID, que orquesta las cuatro interfaces en la secuencia construir → enviar → recibir → validar → decodificar. El Algoritmo 2 describe este flujo, común a todas las operaciones del modo 0x01.

5.5.2. Capa de transporte CAN e ISO-TP

La comunicación con el bus CAN se realiza a través de la interfaz `SocketCAN` de Linux, configurada mediante `ip link set can0 up type can bitrate 500000`. La librería

Algoritmo 2 Ciclo de consulta de un PID OBD-II (Modo 0x01)**Require:** *pid* (identificador de parámetro), *modo* $\leftarrow$ 0x01**Ensure:** valor físico decodificado, con unidad

```

1: peticion  $\leftarrow$  [modo, pid]                                ▷ p. ej. 0x01 0x0C para RPM
2: TRANSPORTE.ENVIAR(peticion)                                ▷ ISO-TP segmenta si > 7 bytes
3: resp  $\leftarrow$  TRANSPORTE.RECIBIR                             ▷ reensambla ISO-TP; timeout 5 s
4: if resp[0] = 0x7F then                                       ▷ respuesta negativa (NRC)
5:   lanzar NRCEXCEPTION(resp[1], resp[2])
6: else if resp[0]  $\neq$  modo + 0x40 then
7:   lanzar INVALIDRESPONSEERROR                                ▷ eco de modo inesperado
8: end if
9: datos  $\leftarrow$  resp[2:]
10: return DECODIFICAR(pid, datos)                             ▷ fórmula SAE J1979 del PID

```

python-can proporciona una abstracción de alto nivel sobre esta interfaz, mientras que can-isotp gestiona de forma automática la segmentación y reensamblado ISO-TP, el control de flujo y la temporización entre tramas.

La configuración del socket ISO-TP (`isotp.Address` en modo *Normal 11 bits*) define la dirección del escáner (`txid = 0x7E0`) y la dirección de respuesta de la ECU (`rxid = 0x7E8`), con `tx_padding = 0xAA` para el relleno a 8 bytes, `blocksize = 0` (se aceptan todas las *Consecutive Frames*) y `stmin = 25 ms`. El timeout de recepción se fija en 5 segundos, valor holgado para absorber la latencia de la transmisión multi-frame sin generar falsos errores.

5.5.3. Módulo de logging en SQLite

Toda la información de diagnóstico se almacena en una base de datos SQLite cuyo esquema se describe a continuación. La base de datos opera en modo WAL (*Write-Ahead Logging*), que permite escrituras concurrentes desde el hilo de monitorización y el hilo de atención a comandos BLE sin bloqueos mutuos.

Las cuatro tablas del esquema son:

- **sessions:** registra cada sesión de diagnóstico con una etiqueta (`label`) y los *timestamps* de inicio y fin (`started_at`, `ended_at`). El número de muestras de cada sesión no se almacena, sino que se calcula mediante una consulta agregada sobre la tabla `samples`.
- **samples:** almacena cada muestra de telemetría con el PID, el nombre, el valor decodificado, la unidad, un *timestamp* monótonico (`monotonic_ts`) y un *timestamp* de pared en formato ISO-8601 (`wall_ts`).
- **commands:** registra cada comando ejecutado con su nombre (`command`), las tramas de petición y respuesta en hexadecimal (`request_hex`, `response_hex`) y el *timestamp*. Estos campos permiten reconstruir la secuencia completa de comunicación ISO-TP para análisis forense.
- **dtcs:** almacena los códigos de avería leídos en cada sesión (código en formato J2012,

bytes en bruto, descripción y *timestamp*).

El `SqliteDataLogger` acumula las muestras en un buffer en memoria y las vuelca a SQLite en lotes de 50 (`_BUFFER_SIZE`), reduciendo la frecuencia de escritura a disco y mejorando el rendimiento general del sistema durante sesiones de monitorización prolongadas.

5.5.4. Monitor de datos en tiempo real

El monitor de datos (`LiveDataMonitor`) se ejecuta en un hilo *daemon* en segundo plano con un periodo de muestreo de 500 ms por defecto. En cada ciclo consulta un conjunto configurable de PIDs del modo 0x01, decodifica los valores recibidos e invoca un *callback on_sample* por cada muestra. El `BtCommandHandler` agrupa las muestras de cada ciclo y las reenvía al cliente móvil como una única notificación BLE, además de almacenarlas en SQLite. El acceso al bus CAN se serializa con el hilo de atención a comandos mediante un *lock* (candado de mutua exclusión) compartido.

El conjunto de PIDs monitorizados por defecto incluye temperatura del refrigerante, carga calculada del motor, RPM, velocidad y posición del acelerador. La lista es configurable en tiempo de ejecución a través del comando JSON correspondiente.

5.5.5. Servidor BLE GATT

El servidor BLE se implementa mediante la librería `bless`, que proporciona una interfaz asíncrona (`asyncio`) para la gestión del servicio GATT. El servidor expone el servicio NUS con sus dos características (TX y RX) y gestiona la conexión del cliente BLE.

El manejador de comandos `BtCommandHandler` recibe los objetos JSON escritos por el cliente en la característica RX, los despacha al método correspondiente (sobre la `LoggedDiagnosticSession` el logger o el transporte según el comando) y envía la respuesta JSON como notificación en la característica TX.

Se implementa un mecanismo de autenticación por token: el primer comando de cada sesión BLE debe incluir el campo `token` con el valor configurado en el servidor. Las conexiones no autenticadas reciben una respuesta de error y se cierran tras un timeout de inactividad.

Los comandos JSON implementados son:

- `{"cmd": "auth", "token": "..."}:` autenticación de sesión.
- `{"cmd": "ping"}:` comprobación de enlace (responde `pong`).
- `{"cmd": "snapshot"}:` lectura puntual de todos los PIDs soportados del modo 0x01.
- `{"cmd": "dtcs"} / {"cmd": "clear_dtcs"}:` lectura y borrado de DTCs.
- `{"cmd": "vin"}:` obtención del VIN del vehículo.
- `{"cmd": "monitor_start", "pids": [12,13,5]} / {"cmd": "monitor_stop"}:` inicio y detención de la monitorización continua.

- {"cmd": "probe_pids"}: descubrimiento automático de los PIDs soportados por la ECU.
- {"cmd": "sessions"}, {"cmd": "session_samples", "session_id": N}, {"cmd": "session_confirmed", "session_id": N}, {"cmd": "session_dtcs", "session_id": N}: consulta del historial SQLite.
- {"cmd": "uds_session", "session_type": 3} / {"cmd": "uds_read_did", "did": "0xF190"}: control de sesión UDS y lectura de DIDs.
- {"cmd": "disconnect"}: cierre ordenado de la sesión.

El comando `probe_pids` merece una mención aparte por su lógica en dos etapas: primero declara los PIDs soportados leyendo las máscaras de bits estándar (0x00, 0x20, ...), y después confirma cada uno con una consulta real para descartar falsos positivos. El Algoritmo 3 explica este procedimiento.

Algoritmo 3 Descubrimiento de PIDs soportados (`probe_pids`)

```

1: declarados ← ∅
2: for cada PID de soporte  $s \in \{0x00, 0x20, \dots, 0xE0\}$  do
3:   enviar [0x01,  $s$ ]; resp ← recibir()
4:   if resp es respuesta válida a  $s$  then
5:     mask ← 32 bits de resp[2..5]
6:     for  $i \leftarrow 0$  to 31 do
7:       if bit  $i$  de mask está activo then
8:         añadir ( $s + i + 1$ ) a declarados
9:       end if
10:    end for
11:    if bit menos significativo de mask = 0 then
12:      break ▷ no se anuncia el siguiente rango
13:    end if
14:  end if
15: end for
16: confirmados ← ∅
17: for cada pid ∈ declarados (ordenados) do
18:   enviar [0x01, pid]; resp ← recibir()
19:   if resp es positiva y hace eco de pid then
20:     añadir pid a confirmados
21:   end if
22: end for
23: return confirmados ∩ {PIDs con decodificador}

```

5.6. Aplicación móvil — React Native

La aplicación móvil es la interfaz de usuario del sistema de diagnóstico. Se implementa con React Native y Expo, lo que permite compartir una única base de código entre Android e iOS. El código fuente está disponible en el repositorio del proyecto.

5.6.1. Arquitectura de la aplicación

La aplicación sigue una arquitectura basada en tres capas:

1. **Capa de adaptadores:** abstrae el acceso al hardware BLE mediante la interfaz `IVehicleAdapter`. La implementación concreta `BleAdapter` utiliza la librería `react-native-ble-plx` para las operaciones de escaneo, conexión, escritura y suscripción a notificaciones. La implementación `MockAdapter` simula un dispositivo BLE para el desarrollo sin hardware, con respuestas predefinidas para todos los comandos. La fábrica `adapterFactory` selecciona una u otra según el modo de la aplicación.
2. **Capa de estado:** gestiona el estado global de la aplicación mediante almacenes `Zustand`. Los almacenes principales son `useConnectionStore` (estado de la conexión BLE y dispositivo activo), `useDashboardStore` y `useVehicleStore` (últimos valores de los PIDs monitorizados), `useDtcStore` (lista de DTCs), `useSessionStore` (historial de sesiones y muestras), `useUdsStore` (sesión y DIDs UDS), `usePidSupportStore` (PIDs soportados tras el sondeo) y `useSettingsStore` (modo simulación y preferencias).
3. **Capa de presentación:** las cinco pantallas de la aplicación consumen el estado de los almacenes `Zustand` y despachan comandos a través del adaptador BLE.

5.6.2. Pantallas y funcionalidades

La navegación principal se organiza en cinco pestañas (*bottom tab navigator*): *Panel*, *Averías*, *Sesiones*, *Consola* y *Ajustes*. El flujo de conexión (escaneo, selección de dispositivo y autenticación) se presenta mediante diálogos modales lanzados desde la pantalla de Ajustes.

Flujo de conexión

El usuario inicia el escaneo de dispositivos BLE cercanos desde la pantalla de Ajustes, la aplicación enumera todos los dispositivos y el usuario selecciona el adaptador (la Raspberry Pi se anuncia como `diag_tool`). Durante el proceso se muestra el progreso de las fases: conexión física, negociación de MTU, descubrimiento de servicios, suscripción a notificaciones y autenticación por token. Una vez conectado exitosamente con la Pi le solicita el VIN del vehículo y los PIDs soportados. El adaptador BLE implementa un *watchdog* de inactividad que desconecta automáticamente la sesión si no hay actividad durante el periodo establecido.

Pantalla Panel

La pantalla principal del diagnóstico muestra las opciones de obtener una snapshot (captura de datos) en un momento determinado o de obtener en tiempo real los parámetros del vehículo recibidos por monitorización continua. Los indicadores se presentan en forma de tarjetas con el valor actual, la unidad y cuyo color refleja la calidad del dato (si representa valor crítico o dentro de los parámetros normales). Incluye además una pestaña dedicada a UDS, que permite cambiar la sesión de diagnóstico y leer los identificadores de datos (DID) estándar y extendidos.

Al establecer la conexión, la aplicación inicia automáticamente una solicitud para obtener una máscara que refleja que PIDs soporta la ECU conectada y posteriormente realiza una

prueba de cada uno de ellos verificando que no haya falsos positivos.

Pantalla Averías (DTCs)

La pantalla de códigos de avería da la opción de realizar un escaneo de la lista de DTCs activos leída desde la ECU. Cada código se presenta con su identificador alfanumérico (formato J2012), su descripción en texto y un indicador de categoría (P/C/B/U). La pantalla ofrece la acción de borrado de todos los DTCs, con un diálogo de confirmación previo por tratarse de una operación irreversible.

Pantalla Sesiones

La pantalla de historial muestra la lista de sesiones de diagnóstico almacenadas en la base de datos SQLite de la Raspberry Pi, ordenadas de la más reciente a la más antigua. Al seleccionar una sesión, se entra en la pantalla de detalles de sesión donde se muestra lista de muestras de telemetría agrupadas por PID desplegable que nos muestra las distintas muestras obtenidas con su timestamp, así mismo también hay una pestaña para ver las DTCs registradas en esa sesión.

Pantalla Consola

Esta es una pantalla añadida para poder monitorizar y depurar el estado de la aplicación y poder obtener más detalles de los distintos protocolos. En esta pantalla se muestran los logs de la comunicación BLE mostrando el registro de comunicación BLE (RX/TX), del protocolo OBD-II, UDS y de el estado de la App en distintas pestañas. Asimismo cuenta con un botón de exportación de logs a csv.

Pantalla Ajustes

La pantalla de ajustes gestiona la conexión con el adaptador (escaneo y selección de dispositivo), el modo de simulación (*mock*) para usar la aplicación sin hardware, la frecuencia de sondeo para el modo de monitorización, reorganizar los PIDs mostrados en el panel y realizar nuevamente una prueba de PIDs.

5.6.3. Comunicación BLE

La librería `react-native-ble-plx` gestiona las operaciones BLE de bajo nivel. La característica RX del servicio NUS se escribe mediante operaciones `writeCharacteristicWithResponse`, que garantizan la entrega del comando antes de esperar la respuesta. La característica TX se monitoriza mediante `monitorCharacteristicForService`, que invoca un callback cada vez que la Raspberry Pi envía una notificación.

El protocolo se basa en mensajes JSON delimitados por saltos de línea (NDJSON): tanto la Raspberry Pi como la aplicación fragmentan los mensajes que superan el tamaño de escritura BLE (240 bytes) y los reensamblan en el receptor acumulando los fragmentos hasta el delimitador `\n`. La aplicación negocia un MTU de 128 bytes en la conexión. En la práctica, la fragmentación solo se produce en las respuestas de historial de sesiones con un número elevado de muestras.

El receptor debe resolver dos problemas simultáneamente: reensamblar las notificaciones BLE (que pueden llegar fragmentadas o agrupar varios mensajes) y emparejar cada respuesta con su petición sin bloquear el flujo de muestras de la monitorización continua. El Algoritmo 4 describe la lógica del adaptador: un búfer de recepción acumula los fragmentos y los separa por el delimitador `\n`, mientras que un despachador encamina cada mensaje según su tipo. Los mensajes asíncronos (muestras de telemetría y *heartbeats*) se entregan a los *callbacks* del monitor o se responden de inmediato; el resto se empareja con la petición pendiente más antigua de una cola FIFO, que resuelve o rechaza la promesa correspondiente. La implementación completa figura en los listados del Anexo ??.

Algoritmo 4 Reensamblado NDJSON y emparejamiento petición-respuesta

```

1: Estado: buffer  $\leftarrow$  “”; cola  $\leftarrow$  FIFO vacía de peticiones pendientes

2: function ALRECIBIRNOTIFICACIÓN(fragmento)
3:   buffer  $\leftarrow$  buffer + DECODIFICAR(fragmento)
4:   lineas  $\leftarrow$  SEPARAR(buffer, \n)
5:   buffer  $\leftarrow$  último elemento de lineas ▷ fragmento incompleto
6:   for cada linea completa de lineas do
7:     if linea no está vacía then
8:       msg  $\leftarrow$  PARSEARJSON(linea); DESPACHAR(msg)
9:     end if
10:  end for
11: end function

12: function DESPACHAR(msg)
13:  if msg.type  $\in$  {samples, sample, error} then
14:    entregar msg a los callbacks del monitor ▷ flujo asíncrono
15:  else if msg.type = heartbeat then
16:    enviar heartbeat_ack
17:  else
18:    pendiente  $\leftarrow$  COLA.EXTRAER ▷ petición más antigua
19:    if msg.status = ok then
20:      resolver pendiente con msg.data
21:    else
22:      rechazar pendiente con msg.message
23:    end if
24:  end if
25: end function

```

5.7. Integración y pruebas

La validación del sistema completo se realizó sobre el banco de ensayo formado por el Arduino MKR con shield CAN, la Raspberry Pi con controlador MCP2515 y una tableta

Android con la aplicación instalada. Los tres nodos se conectaron mediante un bus CAN de dos hilos con terminación de 120Ω en cada extremo.

5.7.1. Escenarios de prueba

Se definieron los siguientes escenarios de prueba que cubren los flujos principales del sistema y los casos de error más relevantes:

Escenario 1 — Ciclo OBD-II básico. Verificación del ciclo completo de consulta de un PID del modo 0x01 (RPM, 0x0C): envío de la petición desde la aplicación móvil, transmisión CAN de la petición ISO-TP, respuesta del simulador con el valor codificado, decodificación en la Raspberry Pi y presentación en el Panel.

Escenario 2 — Respuesta VIN multi-frame. Verificación de la secuencia ISO-TP completa para la consulta del VIN (0x09 0x02): First Frame, Flow Control y dos Consecutive Frames. Se comprobó que el VIN reconstruido coincide exactamente con la cadena configurada en el simulador y que las tramas CAN correspondientes se registran en la base de datos.

Escenario 3 — Gestión de DTCs. Verificación del ciclo completo de gestión de averías: lectura de DTCs inicial (lista vacía), inyección de DTCs mediante la interrupción hardware del pin D4, lectura tras la inyección (DTCs presentes), borrado de DTCs mediante la aplicación y lectura final (lista vacía de nuevo).

Escenario 4 — Monitorización continua prolongada. Sesión de monitorización de 10 minutos con cinco PIDs activos (refrigerante, carga, RPM, velocidad y acelerador) a 500 ms de periodo. Se verificó la estabilidad de la conexión BLE, la ausencia de pérdidas de muestras y el correcto comportamiento del sistema bajo carga sostenida.

Escenario 5 — Sesión y DIDs UDS. Verificación del cambio de sesión UDS (0x10) a sesión extendida y la lectura de los DID estándar y propietarios (0x22). Se comprobó que los DID propietarios solo responden en sesión extendida y que, en sesión por defecto, el simulador devuelve la respuesta negativa NRC 0x22, que la herramienta registra sin bloquear la sesión.

Escenario 6 — Robustez ante variabilidad del bus. Con los mecanismos opcionales del simulador activos (ADD_NOISE y BROADCAST_ENABLE), se comprobó que las tramas de difusión con identificadores ajenos (0x280, 0x320, 0x3D0) son descartadas por la pila ISO-TP sin interferir en la comunicación OBD-II, y que el ruido en los sensores no impide la correcta decodificación de los valores.

Escenario 7 — Reconexión BLE. Verificación del comportamiento ante la pérdida de la conexión Bluetooth: el adaptador detecta la desconexión, libera los recursos y notifica al usuario, que puede reiniciar la conexión sin pérdida de los datos históricos persistidos en SQLite.

5.7.2. Resultados de las pruebas de integración

Todos los escenarios de prueba definidos se completaron con resultado satisfactorio en el banco de ensayo.

La principal dificultad encontrada durante las pruebas de integración fue la reconexión BLE ante caídas inesperadas del enlace por parte de la Raspberry Pi. Cuando la Pi cerraba la conexión de forma abrupta, el sistema operativo Android conservaba un descriptor GATT obsoleto, de modo que el siguiente intento de conexión fallaba con un error de *dispositivo ya conectado*. Además, las peticiones que quedaban a la espera de respuesta dejaban la interfaz bloqueada indefinidamente. La corrección consistió en tres medidas: registrar un *callback* `onDisconnected` que libera todos los recursos (intervalos de *ping*, suscripción a notificaciones, búfer de recepción) y rechaza las peticiones pendientes de la cola para desbloquear la interfaz, cancelar explícitamente cualquier conexión GATT residual con `cancelDeviceConnection()` antes de cada nuevo intento y un *watchdog* de inactividad que fuerza la desconexión limpia si no se recibe actividad durante 20 s, evitando que la aplicación quede atrapada en un estado de conexión fantasma. Tras estos cambios, el usuario puede reiniciar la conexión sin reiniciar la aplicación.

Las pruebas con los mecanismos opcionales de variabilidad activos confirmaron que el filtro de la pila ISO-TP descarta correctamente las tramas de difusión ajenas a la dirección de respuesta de la ECU, y que el timeout de recepción de 5 segundos absorbe con holgura la latencia de la transmisión multi-frame, manteniendo una tasa de errores no recuperables despreciable en todas las sesiones de prueba.

Capítulo 6

Resultados

En este capítulo se estudian los resultados obtenidos, realizando una comparación con una herramienta comercial que nos ofrece la realización de escaneo.

6.1. Resultados obtenidos

Este capítulo presenta los resultados de la validación del sistema en dos entornos complementarios: el banco de ensayo Arduino, que permitió reproducir condiciones adversas de forma controlada, y un vehículo real, en el que se verificó la interoperabilidad del sistema con una ECU de producción a través del conector OBD-II estándar. Los resultados se organizan en torno a cuatro dimensiones: funcionalidad del protocolo, rendimiento de la comunicación, comportamiento ante condiciones adversas y grado de consecución de los objetivos planteados en el Capítulo 2.

Para verificar que los resultados obtenidos por la herramienta desarrollada son verídicos, previamente se realizó un escaneo de los DTCs y una lectura de PIDs con una herramienta comercial genérica KONWEII [26] junto con la aplicación móvil Car Scanner ELM OBD2 [1], tomada como referencia. La Figura 6.1 muestra el cuadro de instrumentos y la lista de DTCs obtenidos con la herramienta de referencia, que sirvieron de patrón de comparación para validar los valores leídos por la herramienta desarrollada.



(a) Cuadro de instrumentos (lectura de PIDs).



(b) Lista de DTCs.

Ilustración 6.1: Herramienta comercial de referencia (Car Scanner ELM OBD2 [1]) usada como patrón de comparación: cuadro de instrumentos (a) y lista de DTCs (b).

6.2. Validación sobre vehículo real

Tras completar la validación sobre el banco de ensayo, el sistema se probó sobre un vehículo real. El vehículo objetivo inicial era un SEAT Ibiza 6J, pero debido a que la plataforma de ese modelo no implementa el protocolo OBD-II sobre el bus CAN en los pines 6 y 14 del puerto OBD, la validación final se realizó sobre un **Škoda Yeti de 2015**, perteneciente igualmente al grupo VAG y, por tanto, representativo de la misma arquitectura de diagnóstico. La conexión se efectuó mediante un cable OBD-II estándar que nos revela todos los

pinos del puerto de diagnóstico del vehículo: la Raspberry Pi actuó como interfaz entre el bus CAN del vehículo conectado a través de los pines 6 y 14 (CAN_L y CAN_H respectivamente) del puerto OBD-II y la aplicación móvil, manteniendo exactamente el mismo stack de software validado en el banco (IsoTpTransport sobre SocketCAN, servidor BLE GATT con NUS y aplicación React Native).

La conexión al vehículo demostró que el protocolo OBD-II implementado es compatible con ECUs de producción: la herramienta fue capaz de consultar los PIDs del modo 0x01 soportados por el vehículo, leer el VIN real mediante la secuencia ISO-TP multi-frame y recibir datos de telemetría en tiempo real a través de la aplicación móvil. Las respuestas del vehículo presentaron tiempos de respuesta ligeramente superiores a los del simulador (10–30 ms frente a 2–8 ms).

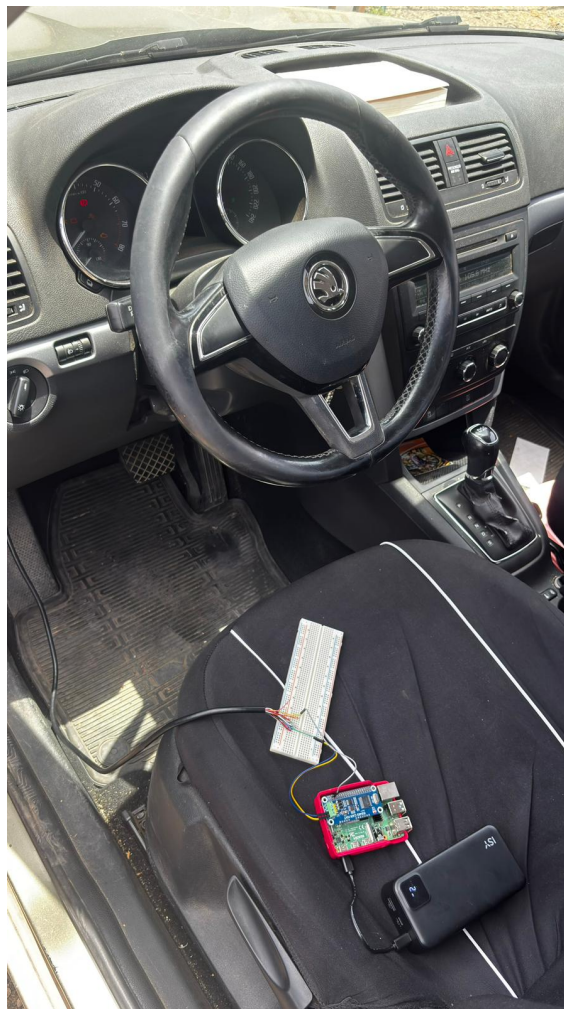


Ilustración 6.2: Sistema montado en el vehículo real: cable OBD-II conectado al puerto de diagnóstico, Raspberry Pi con el servidor activo y teléfono móvil con la aplicación mostrando datos de telemetría en tiempo real.

6.3. Validación funcional del protocolo OBD-II

La herramienta de diagnóstico fue capaz de completar satisfactoriamente el ciclo completo de consulta para los 25 PIDs implementados en el modo 0x01, la lectura del VIN del modo 0x09 (mediante la secuencia multi-frame), los modos de gestión de DTCs 0x03 y 0x04, y los servicios UDS 0x10 (DiagnosticSessionControl) y 0x22 (ReadDataByIdentifier).

La tabla 6.1 resume los parámetros validados más representativos, indicando el PID, la fórmula de decodificación aplicada y el rango de valores observados durante las sesiones de prueba.

Cuadro 6.1: PIDs del modo 0x01 validados en el banco de ensayo.

PID	Parámetro	Fórmula (A, B = bytes de respuesta)	Rango observado
0x0C	RPM del motor	$((A \times 256) + B)/4$	800 – 6000 rpm
0x0D	Velocidad	A	0 – 255 km/h
0x05	Temp. refrigerante	$A - 40$	15 – 95 °C
0x0F	Temp. aire admisión	$A - 40$	15 – 45 °C
0x0A	Presión combustible	$A \times 3$	240 – 360 kPa
0x11	Posición acelerador	$A \times 100/255$	0 – 100 %
0x04	Carga calculada motor	$A \times 100/255$	10 – 85 %
0x5E	Consumo combustible	$((A \times 256) + B)/20$	0,4 – 8,5 L/h
0x42	Tensión de batería	$((A \times 256) + B)/1000$	12,4 – 14,8 V

La respuesta VIN se validó con una cadena de 17 caracteres ASCII, verificando que la secuencia ISO-TP multi-frame (FF + FC + CF $\times$ 2) se completaba correctamente en el 100 % de los intentos realizados con el bus en modo determinista. El VIN reconstruido en la herramienta Python coincidió en todos los casos con la cadena configurada en el simulador Arduino.

El ciclo de gestión de DTCs se validó íntegramente: lectura de lista vacía, inyección de los tres DTCs del conjunto (P0300, P0171, P0420) mediante la interrupción hardware del pin D4, lectura de la lista con los códigos presentes, borrado y confirmación de lista vacía. La codificación J2012 de los DTCs fue correcta en todos los casos.

6.4. Métricas de rendimiento de la comunicación

Se midió la latencia extremo a extremo de una consulta OBD-II completa, desde el envío del comando JSON desde la aplicación móvil hasta la recepción de la respuesta decodificada, en condiciones de bus sin ruido. Las mediciones se realizaron sobre 100 consultas consecutivas del PID 0x0C (RPM).

Los resultados obtenidos se resumen en la tabla 6.2:

Cuadro 6.2: Latencia extremo a extremo de una consulta OBD-II (100 muestras, bus en modo determinista).

Componente	Latencia media (ms)
BLE Write (app → Raspberry Pi)	12 – 25
Procesamiento JSON + construcción trama	< 1
Transacción ISO-TP CAN (SF)	2 – 8
Decodificación + logging SQLite	3 – 7
BLE Notify (Raspberry Pi → app)	10 – 20
Total extremo a extremo	28 – 62

La latencia dominante en el sistema es la introducida por la capa BLE, que suma las contribuciones del Write y la Notify. La latencia CAN+ISO-TP es despreciable frente a la BLE a las velocidades de bus utilizadas.

En el modo de monitorización continua con cinco PIDs a 500 ms de periodo, el sistema fue capaz de mantener una tasa de actualización estable sin pérdidas de muestras durante sesiones de 10 minutos. El reenvío de las muestras del hilo de monitorización al cliente BLE no presentó pérdidas ni saturación en ninguno de los ensayos.

6.5. Comportamiento ante condiciones de bus realistas y errores

Se evaluó el comportamiento del sistema con los mecanismos opcionales de variabilidad del simulador activos (ADD_NOISE y BROADCAST_ENABLE) y ante respuestas negativas (NRC) de la ECU.

Los resultados más relevantes de estas pruebas son:

- **Tramas de difusión:** Con BROADCAST_ENABLE activo, las tramas no solicitadas (identificadores 0x280, 0x320 y 0x3D0) fueron ignoradas por la pila `can-isotp`, que solo entrega los mensajes dirigidos a la dirección de respuesta de la ECU (0x7E8). En ningún caso una trama de difusión se interpretó como respuesta válida a una petición.
- **Ruido en los sensores:** Con ADD_NOISE activo, los valores de telemetría presentaron el *jitter* esperado dentro de sus rangos físicos, sin afectar a la correcta decodificación de las respuestas ni a la estabilidad de la monitorización continua.
- **Respuestas negativas UDS:** Las respuestas NRC (por ejemplo, 0x22 al leer un DID propietario fuera de la sesión extendida) fueron detectadas y registradas correctamente. El hilo de monitorización omite la muestra afectada y prosigue con el siguiente PID del ciclo, sin bloquear la sesión.
- **Robustez del transporte:** El timeout de recepción de 5 segundos y el reensamblado automático de `can-isotp` absorbieron la latencia de las transmisiones multi-frame sin

generar falsos errores, manteniendo la sesión estable durante toda la prueba.

La Figura 6.3 muestra la traza de ejecución durante una de estas pruebas: la consola del simulador Arduino, que emite las tramas de difusión y el ruido inyectado en los sensores, y la consola de la Raspberry Pi, donde la pila `can-isotp` descarta las tramas ajenas y decodifica únicamente las respuestas dirigidas a la dirección de la ECU.

```

20:34:03.675 -> =====
20:34:03.675 -> ECU SIMULATOR v1
20:34:03.675 -> OBD-II / ISO-TP - Fase 2
20:34:03.675 -> =====
20:34:03.675 -> [OK] CAN at 500 kbps
20:34:03.675 -> [INFO] VIN: WVWZZZ1JZ3W386752
20:34:03.675 -> [INFO] Listen: 0x7E0
20:34:03.675 -> [INFO] Reply: 0x7E8
20:34:03.675 -> [INFO] Pre-loaded DTC: P0300
20:34:03.675 -> =====
20:34:03.675 ->
20:48:48.487 ->
20:48:48.487 -> [RX] 0x7E0 Mode:0x1 PID:0xC
20:48:48.487 -> [TX SF] 0x7E8 | 04 41 0C 44 10 AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x5
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 05 91 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0xD
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 0D 29 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x11
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 11 C1 AA AA AA AA
20:48:48.578 ->
20:48:48.578 -> [RX] 0x7E0 Mode:0x1 PID:0x4
20:48:48.578 -> [TX SF] 0x7E8 | 03 41 04 AD AA AA AA AA
20:48:56.763 ->
20:48:56.763 -> [RX] 0x7E0 Mode:0x9 PID:0x2
20:48:56.763 -> [INFO] VIN request - sending: WVWZZZ1JZ3W386752
20:48:56.763 -> [TX FF] VIN First Frame sent
20:48:56.763 -> [INFO] Waiting for FC...
20:48:56.763 -> [RX FC] Flow Control received
20:48:56.763 -> [TX CF] SN=0x1
20:48:56.853 -> [TX CF] SN=0x2
20:48:56.853 -> [INFO] VIN transmission complete
20:49:07.973 ->
20:49:07.973 -> [RX] 0x7E0 Mode:0x3 PID:0xAA
20:49:07.973 -> [TX SF] 0x7E8 | 04 43 01 03 00 AA AA AA
20:49:07.973 -> [INFO] Sent 1 DTC(s)

```

(a) Consola del simulador Arduino.

```

(.venv) gorka@gorka:~/TFT-CAN-BLE-system/src $ python scripts/cli.py

OBD-II Diagnostic Tool - Fase 2
Transport: IsoTpTransport (can0)

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 1
[20:48:48]

RPM          : 4356.00 rpm
Coolant Temp : 105.00 °C
Speed        : 41.00 km/h
Throttle Pos : 75.69 %
Engine Load  : 67.84 %

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 5
[20:48:56]

VIN: WVWZZZ1JZ3W386752

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option: 3
[20:49:07]

[P0300]

[1] Live data          (5 PIDs principales)
[2] Extended live data (todos los PIDs)
[3] Read DTCs
[4] Clear DTCs
[5] Read VIN
[0] Exit

Select option:

```

(b) Consola de la Raspberry Pi.

Ilustración 6.3: Traza de ejecución bajo condiciones de bus realistas: consola del simulador Arduino emitiendo tramas de difusión y ruido (a) y consola de la Raspberry Pi filtrando y decodificando las respuestas válidas (b).

6.6. Interfaz de la aplicación móvil

A continuación se recogen las capturas de las distintas pantallas de la aplicación móvil obtenidas durante las pruebas en el vehículo real, que ilustran el resultado final de la interfaz de usuario sobre las cinco pestañas de navegación y el flujo de conexión.

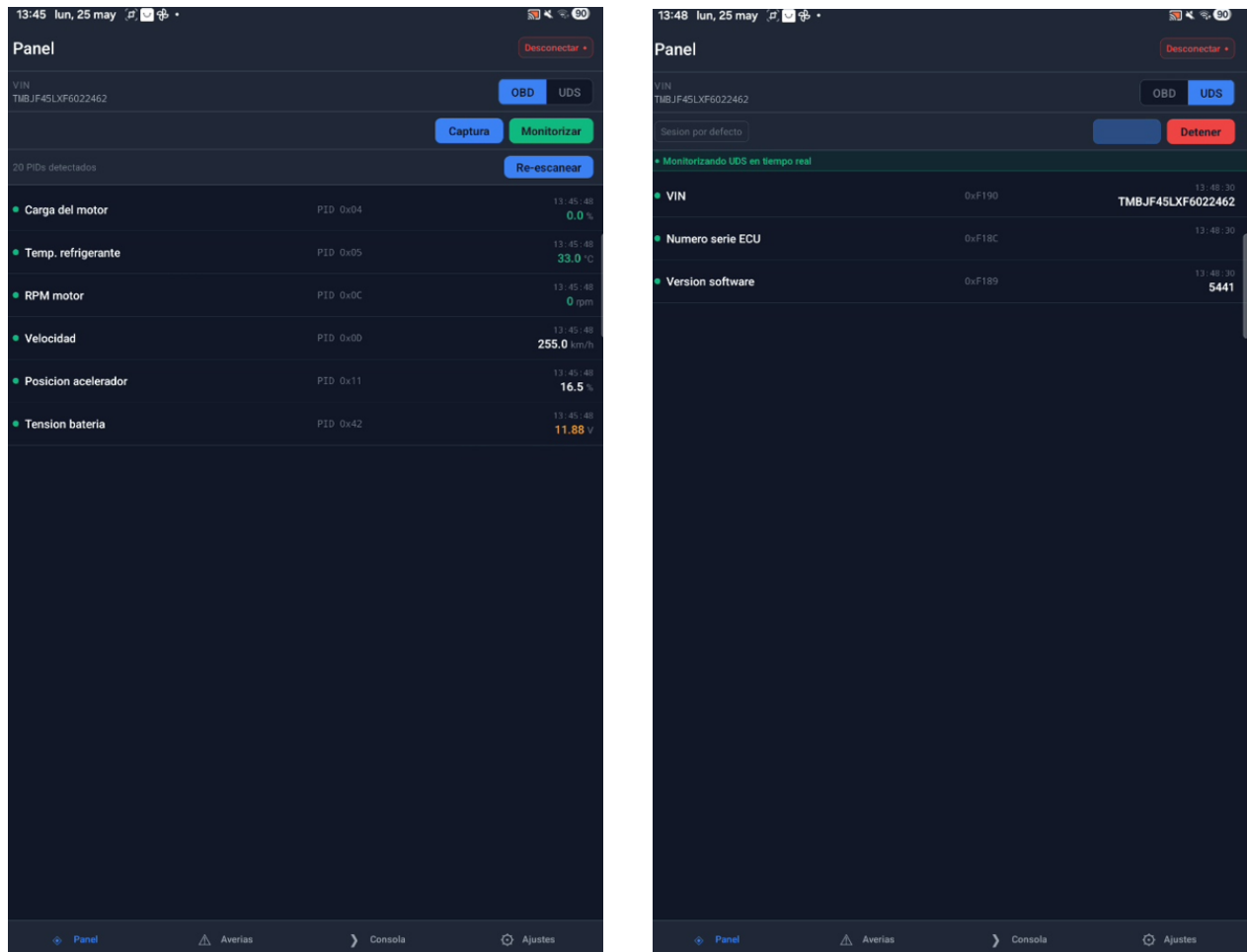


Ilustración 6.4: Pantalla *Panel*: telemetría en tiempo real (izquierda) y pestaña UDS con control de sesión y lectura de DIDs (derecha).

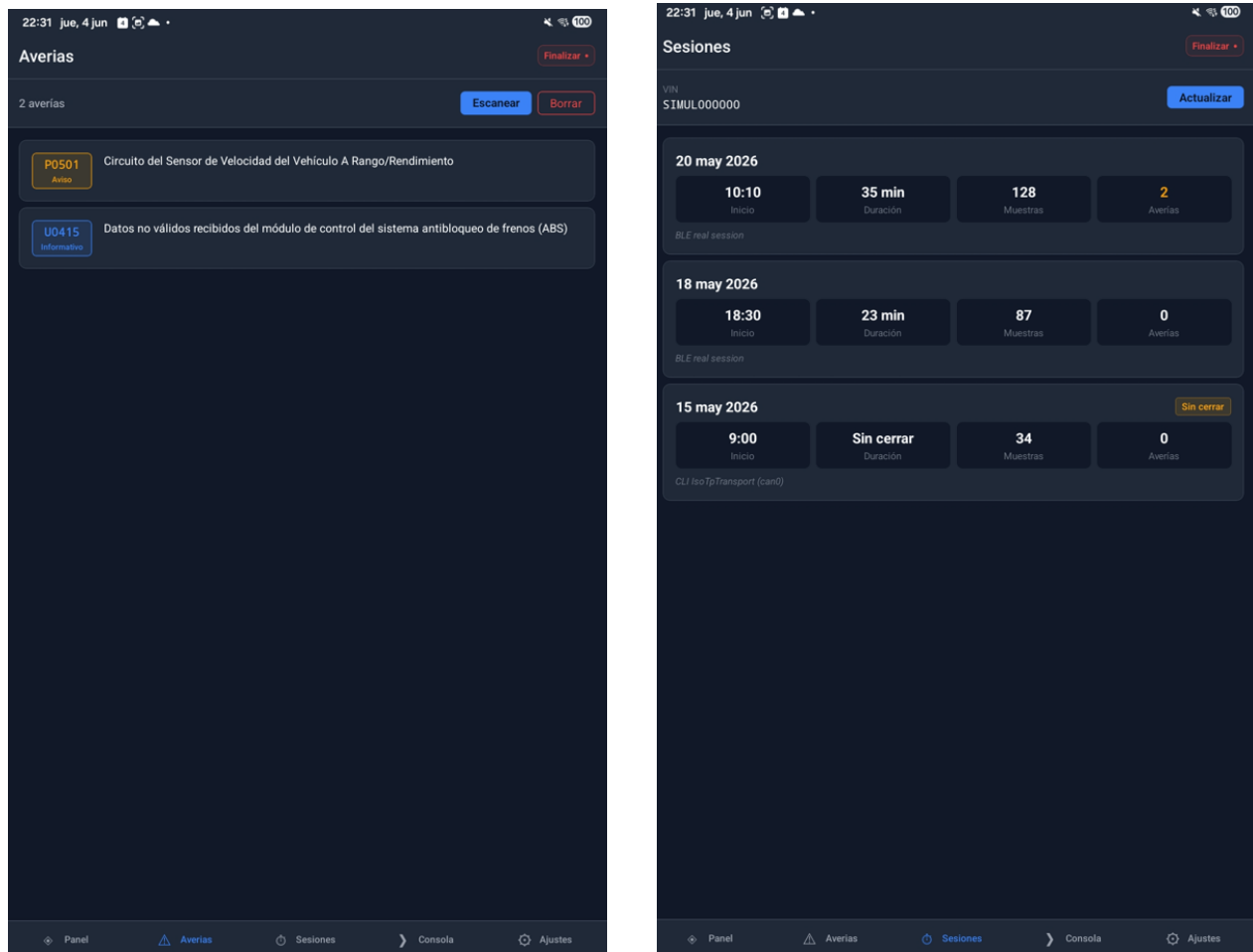


Ilustración 6.5: Pantallas *Averías* (izquierda), con la lista de DTCs leídos de la ECU, y *Sesiones* (derecha), con el historial de sesiones almacenado en SQLite.

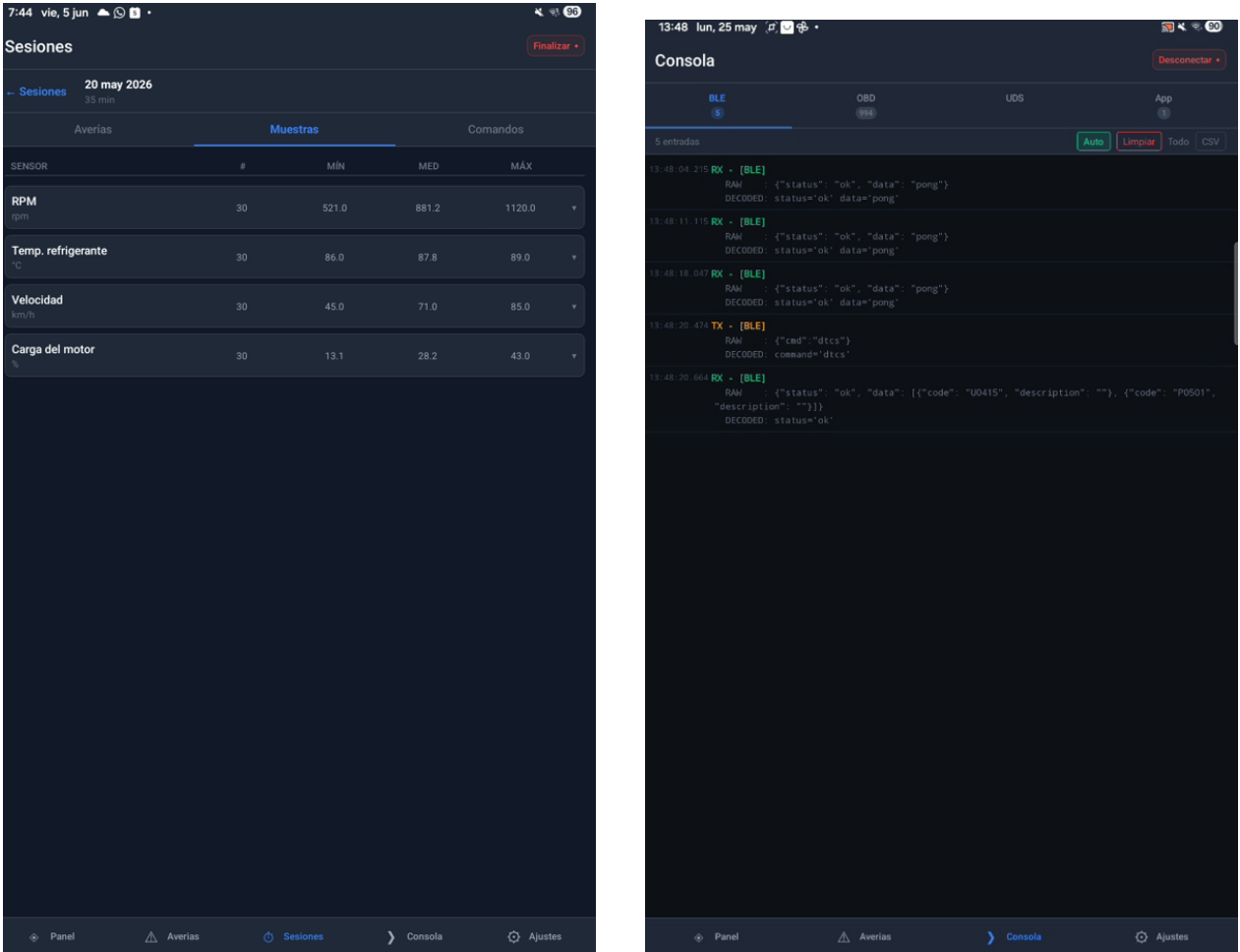


Ilustración 6.6: Detalle de una sesión con las muestras agrupadas por sensor (izquierda) y pantalla *Consola* con el envío de comandos JSON y el registro de comunicación BLE (derecha).

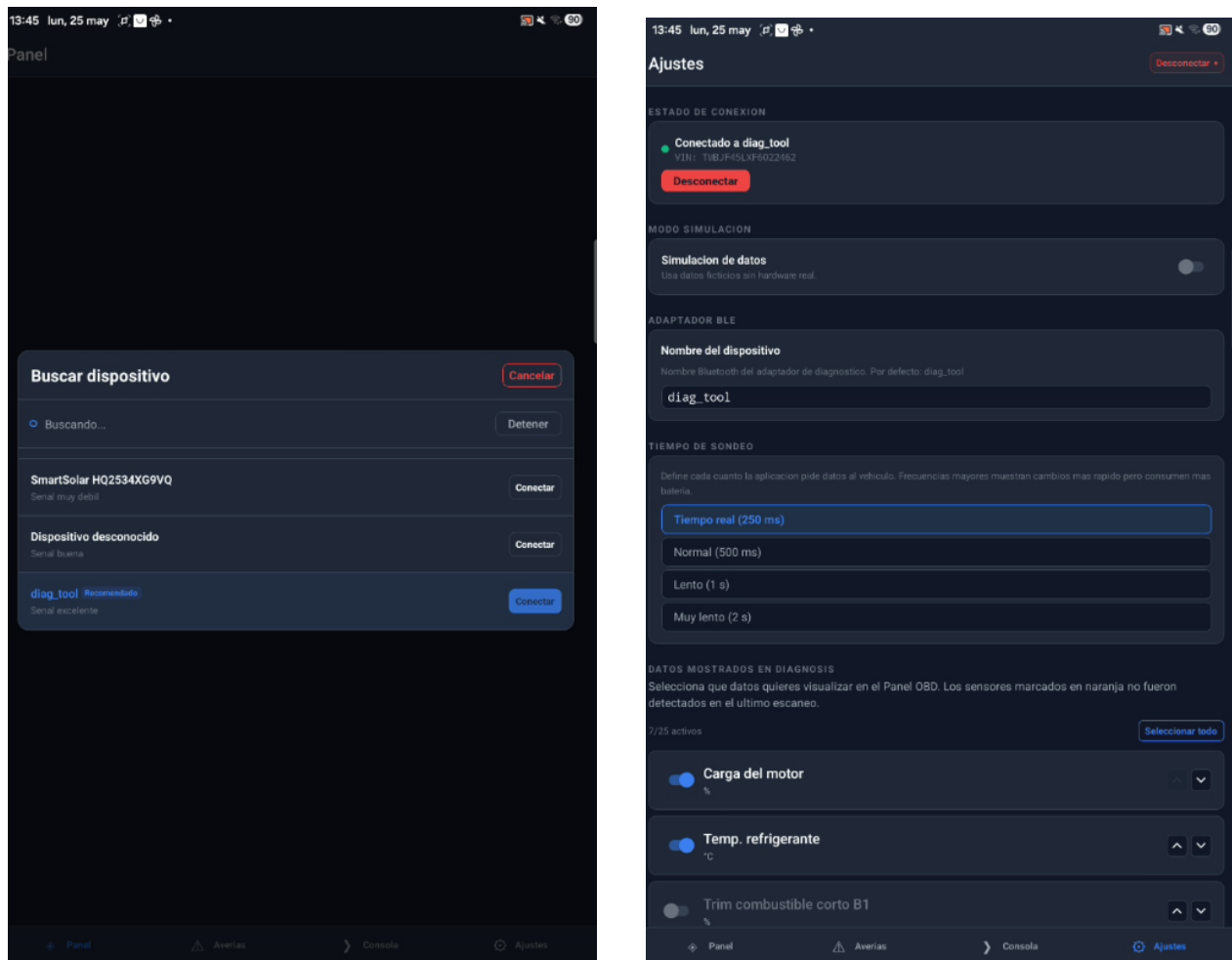


Ilustración 6.7: Pantalla *Ajustes* (izquierda), con la gestión de conexión y el modo de simulación, y la pantalla de ajustes (derecha).

6.7. Grado de consecución de los objetivos

La tabla 6.3 recoge el grado de consecución de cada uno de los objetivos específicos definidos en la Sección 2.2, evaluado sobre la base de los resultados de las pruebas de integración.

Cuadro 6.3: Evaluación del grado de consecución de los objetivos específicos.

Obj.	Indicador de consecución	Resultado
O1	Pila ISO-TP + OBD-II modos 0x01/03/04/09 operativa	Completo
O2	Simulador Arduino responde a los 4 modos con datos realistas	Completo
O3	Servicios UDS (0x10/0x22) y mecanismos opcionales de ruido/difusión	Completo
O4	Servidor BLE GATT con NUS operativo en Raspberry Pi	Completo
O5	App React Native Android/iOS con 5 pantallas funcionales	Completo
O6	Logging SQLite con sesiones, muestras y comandos	Completo

Los seis objetivos específicos se han alcanzado en su totalidad. La validación sobre vehículo real (Sección 6.2) confirmó la compatibilidad del sistema con ECUs de producción, completando la cadena de validación desde el banco de ensayo controlado hasta un entorno vehicular real.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

Este Trabajo de Fin de Título ha demostrado la viabilidad técnica de implementar un sistema completo de diagnóstico del vehículo OBD-II. Los resultados obtenidos en el banco de ensayo Arduino, junto con la validación sobre un vehículo real del grupo VAG (Škoda Yeti de 2015), confirman que la plataforma desarrollada es funcional, interoperable con ECUs de producción y extensible en sus tres componentes de forma independiente.

Las conclusiones más relevantes del trabajo se pueden agrupar en torno a los tres componentes del sistema:

Sobre la pila de protocolos. La implementación directa del UDS y OBD-II sobre `SocketCAN` en Python ha resultado viable. La librería `can-isotp` proporciona una abstracción suficiente para gestionar la segmentación y el Flow Control de forma transparente, si bien la correcta sincronización de la secuencia multi-frame para el VIN requirió un ajuste cuidadoso de los parámetros de timing en el simulador. La latencia extremo a extremo del sistema (28–62 ms por consulta) es despreciable pues se trata de una herramienta de diagnóstico para uso personal que no trata factores críticos.

Sobre el simulador Arduino. El simulador de ECU desarrollado se ha revelado como una herramienta de validación de alto valor, que va más allá de la simple emulación de respuestas fijas. El modelo físico del motor, los mecanismos opcionales de variabilidad del bus (ruido en sensores y tramas de difusión) y la inyección de DTCs por interrupción hardware permiten al banco de ensayo la capacidad de simulación de escenarios reales que resulta difícil de reproducir sin acceso continuo a un vehículo.

Sobre la arquitectura software. La decisión de estructurar la herramienta Python con interfaces abstractas e inyección de dependencias ha demostrado alto valor a lo largo del desarrollo: la posibilidad de sustituir el transporte CAN real por un *mock* en cualquier momento redujo significativamente los ciclos de depuración y permitió trabajar con la capa de aplicación de forma independiente del hardware.

Desde el punto de vista del proceso de desarrollo, la metodología basada en prototipos incrementales resultó especialmente adecuada para un proyecto de esta naturaleza, en el que la validez de las hipótesis técnicas no podía garantizarse a priori. Los ciclos cortos de implementación y prueba sobre el banco de ensayo permitieron detectar y corregir problemas de sincronización ISO-TP en etapas tempranas, antes de que estos afectaran al desarrollo de las capas superiores.

La validación sobre vehículo real se llevó a cabo finalmente sobre un Škoda Yeti de 2015 (no sobre el SEAT Ibiza 6J planteado inicialmente, por problemas de acceso ya comentados previamente). Como limitación cabe señalar, por tanto, que la validación se realizó sobre un único vehículo y a través del conector OBD-II, lo que da acceso al bus de diagnóstico.

7.2. Líneas de trabajo futuro

Los resultados obtenidos y las limitaciones identificadas abren varias líneas de continuación para este trabajo:

Validación sobre flota y acceso al bus de tracción. La validación se ha realizado sobre un único vehículo (Škoda Yeti 2015). Una continuación natural sería ampliarla a una flota de varios modelos de distintos fabricantes y calibrar el modelo físico del simulador con datos reales de cada plataforma.

Soporte de servicios UDS adicionales. El sistema implementa ya los servicios UDS DiagnosticSessionControl (0x10) y ReadDataByIdentifier (0x22). Una extensión es incorporar otros servicios de ISO 14229-1, como la escritura de datos por identificador (0x2E), el control de rutinas (0x31), el acceso de seguridad (0x27) o la lectura de información de DTC (0x19), que permiten acceder a parámetros de configuración y calibración de las ECUs no expuestos por el estándar OBD-II.

7.3. Reflexión sobre el uso de herramientas de inteligencia artificial

Durante el desarrollo de este trabajo se hizo uso de herramientas de inteligencia artificial generativa como asistente de programación, concretamente mediante el acceso a modelos de lenguaje de gran escala a través de interfaces de línea de comandos.

El uso de estas herramientas se circunscribió a tres contextos concretos: la generación de fragmentos de código repetitivos o bien definidos por un estándar (como las tablas de decodificación de PIDs OBD-II), la resolución de dudas puntuales sobre la API de las librerías utilizadas (`python-can`, `bless`, `react-native-ble-plx`) y la revisión de la sintaxis de LaTeX en secciones con tablas complejas.

En ningún caso se delegó en estas herramientas el diseño de la arquitectura del sistema, la implementación de los protocolos de comunicación ni la interpretación de los resultados experimentales.

La experiencia ha puesto de manifiesto tanto el valor como los límites de estas herramientas en el contexto de un proyecto de ingeniería. Son especialmente útiles para reducir el tiempo invertido en tareas de codificación mecánica y para obtener una primera orientación ante problemas con librerías poco documentadas. Sin embargo, la generación de código incorrecto sin diagnóstico claro del error sigue siendo habitual, y la verificación de cada sugerencia mediante prueba real es indispensable.

Anexo A

Tabla completa de PIDs OBD-II Modo 0x01

La siguiente tabla recoge todos los PIDs definidos por SAE J1979 [4] para el Modo 0x01 (*Show current data*). Los PIDs marcados con (*) son obligatorios para todo vehículo OBD-II conforme; el resto son opcionales y su soporte depende del fabricante y modelo. Las letras A, B, C, D denotan los bytes de la respuesta en orden (primero = A).

Cuadro A.1: Tabla completa de PIDs del Modo 0x01 OBD-II (*Show current data*) según SAE J1979 [4]. PIDs marcados con (*): obligatorios. A, B, C, D: bytes de respuesta en orden.

PID	Descripción	B	Fórmula	Unidad
0x00	PIDs soportados [0x01–0x20] (*)	4	Máscara de bits	—
0x01	Estado del monitor desde borrado de DTCs (*)	4	Máscara de bits	—
0x02	DTC de congelación (<i>Freeze DTC</i>)	2	—	—
0x03	Estado del sistema de combustible (*)	2	Máscara de bits	—
0x04	Carga calculada del motor (*)	1	$A/2,55$	%
0x05	Temperatura del refrigerante (*)	1	$A - 40$	°C
0x06	Corrección de combustible a corto plazo (banco 1)	1	$(A/1,28) - 100$	%
0x07	Corrección de combustible a largo plazo (banco 1)	1	$(A/1,28) - 100$	%
0x08	Corrección de combustible a corto plazo (banco 2)	1	$(A/1,28) - 100$	%
0x09	Corrección de combustible a largo plazo (banco 2)	1	$(A/1,28) - 100$	%

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x0A	Presión de combustible (manométrica)	1	$3A$	kPa
0x0B	Presión absoluta del colector de admisión (MAP)	1	A	kPa
0x0C	Régimen del motor (RPM) (*)	2	$(256A + B)/4$	rpm
0x0D	Velocidad del vehículo (*)	1	A	km/h
0x0E	Avance del encendido	1	$A/2 - 64$	°
0x0F	Temperatura del aire de admisión	1	$A - 40$	°C
0x10	Caudal másico de aire (MAF)	2	$(256A + B)/100$	g/s
0x11	Posición del acelerador (*)	1	$100A/255$	%
0x12	Estado del aire secundario comandado	1	Máscara de bits	—
0x13	Sondas λ presentes (2 bancos)	1	Máscara de bits	—
0x14	Sonda λ 1 (banco 1, sensor 1): tensión y STFT	2	$V = A/200$; STFT = $(B/1,28) - 100$	V; %
0x15	Sonda λ 2 (banco 1, sensor 2)	2	Ídem 0x14	V; %
0x16	Sonda λ 3 (banco 2, sensor 1)	2	Ídem 0x14	V; %
0x17	Sonda λ 4 (banco 2, sensor 2)	2	Ídem 0x14	V; %
0x18	Sonda λ 5 (banco 3, sensor 1)	2	Ídem 0x14	V; %
0x19	Sonda λ 6 (banco 3, sensor 2)	2	Ídem 0x14	V; %
0x1A	Sonda λ 7 (banco 4, sensor 1)	2	Ídem 0x14	V; %
0x1B	Sonda λ 8 (banco 4, sensor 2)	2	Ídem 0x14	V; %
0x1C	Estándar OBD al que se ajusta el vehículo (*)	1	Enumeración	—
0x1D	Sondas λ presentes (4 bancos)	1	Máscara de bits	—
0x1E	Estado de la entrada auxiliar (PTO)	1	Bit 0 = PTO activo	—
0x1F	Tiempo de marcha desde arranque del motor	2	$256A + B$	s
0x20	PIDs soportados [0x21–0x40]	4	Máscara de bits	—
0x21	Distancia recorrida con MIL activado	2	$256A + B$	km

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x22	Presión de rampa de combustible (relativa al vacío)	2	$0,079 \cdot (256A + B)$	kPa
0x23	Presión manométrica de rampa de combustible	2	$10 \cdot (256A + B)$	kPa
0x24	Sonda λ 1 banda ancha (relación equiv. + tensión)	4	$\lambda = \frac{2(256A+B)}{65536}$; $V = \frac{8(256C+D)}{65536}$	—; V
0x25	Sonda λ 2 banda ancha	4	Ídem 0x24	—; V
0x26	Sonda λ 3 banda ancha	4	Ídem 0x24	—; V
0x27	Sonda λ 4 banda ancha	4	Ídem 0x24	—; V
0x28	Sonda λ 5 banda ancha	4	Ídem 0x24	—; V
0x29	Sonda λ 6 banda ancha	4	Ídem 0x24	—; V
0x2A	Sonda λ 7 banda ancha	4	Ídem 0x24	—; V
0x2B	Sonda λ 8 banda ancha	4	Ídem 0x24	—; V
0x2C	EGR comandado	1	$100A/255$	%
0x2D	Error de EGR	1	$(A/1,28) - 100$	%
0x2E	Purga de evaporación comandada	1	$100A/255$	%
0x2F	Nivel de combustible en depósito	1	$100A/255$	%
0x30	Número de calentamientos desde borrado de DTCs	1	A	—
0x31	Distancia recorrida desde borrado de DTCs	2	$256A + B$	km
0x32	Presión del vapor del sistema evap.	2	$(256A + B)/4$ (con signo)	Pa
0x33	Presión barométrica absoluta	1	A	kPa
0x34	Sonda λ 1 banda ancha (relación equiv. + corriente)	4	$\lambda = \frac{2(256A+B)}{65536}$; $I = \frac{256C+D}{256} - 128$	—; mA
0x35	Sonda λ 2 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x36	Sonda λ 3 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x37	Sonda λ 4 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x38	Sonda λ 5 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x39	Sonda λ 6 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x3A	Sonda λ 7 banda ancha (+ corriente)	4	Ídem 0x34	—; mA
0x3B	Sonda λ 8 banda ancha (+ corriente)	4	Ídem 0x34	—; mA

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x3C	Temperatura del catalizador (banco 1, sensor 1)	2	$(256A + B)/10 - 40$	°C
0x3D	Temperatura del catalizador (banco 2, sensor 1)	2	Ídem 0x3C	°C
0x3E	Temperatura del catalizador (banco 1, sensor 2)	2	Ídem 0x3C	°C
0x3F	Temperatura del catalizador (banco 2, sensor 2)	2	Ídem 0x3C	°C
0x40	PIDs soportados [0x41–0x60]	4	Máscara de bits	—
0x41	Estado del monitor en este ciclo de conducción	4	Máscara de bits	—
0x42	Tensión del módulo de control	2	$(256A + B)/1000$	V
0x43	Valor de carga absoluta	2	$100(256A + B)/255$	%
0x44	Relación aire-combustible comandada (λ)	2	$2(256A + B)/65536$	—
0x45	Posición relativa del acelerador	1	$100A/255$	%
0x46	Temperatura del aire ambiente	1	$A - 40$	°C
0x47	Posición absoluta del acelerador B	1	$100A/255$	%
0x48	Posición absoluta del acelerador C	1	$100A/255$	%
0x49	Posición del pedal del acelerador D	1	$100A/255$	%
0x4A	Posición del pedal del acelerador E	1	$100A/255$	%
0x4B	Posición del pedal del acelerador F	1	$100A/255$	%
0x4C	Actuador del acelerador comandado	1	$100A/255$	%
0x4D	Tiempo de marcha con MIL activado	2	$256A + B$	min
0x4E	Tiempo transcurrido desde borrado de DTCs	2	$256A + B$	min
0x4F	Valores máx. (λ , tensión O ₂ , corriente O ₂ , MAP)	4	$A [\lambda]; B [V]; C [mA]; D \times 10 [kPa]$	—
0x50	Valor máximo del sensor MAF	4	$A \times 10$	g/s
0x51	Tipo de combustible	1	Enumeración	—
0x52	Porcentaje de etanol en el combustible	1	$100A/255$	%

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x53	Presión vapor sistema evap. (absoluta)	2	$(256A + B)/200$	kPa
0x54	Presión vapor sistema evap. (con signo)	2	$256A + B$ (con signo)	Pa
0x55	Corrección O2 secundaria a corto plazo (bancos 1 y 3)	2	$(A/1,28) - 100; (B/1,28) - 100$	%
0x56	Corrección O2 secundaria a largo plazo (bancos 1 y 3)	2	Ídem 0x55	%
0x57	Corrección O2 secundaria a corto plazo (bancos 2 y 4)	2	Ídem 0x55	%
0x58	Corrección O2 secundaria a largo plazo (bancos 2 y 4)	2	Ídem 0x55	%
0x59	Presión absoluta de la rampa de combustible	2	$10(256A + B)$	kPa
0x5A	Posición relativa del pedal del acelerador	1	$100A/255$	%
0x5B	Vida útil restante de la batería híbrida	1	$100A/255$	%
0x5C	Temperatura del aceite de motor	1	$A - 40$	°C
0x5D	Temporización de la inyección de combustible	2	$(256A + B)/128 - 210$	°
0x5E	Caudal de combustible del motor	2	$(256A + B)/20$	L/h
0x5F	Requisitos de emisiones del vehículo	1	Máscara de bits	—
0x60	PIDs soportados [0x61–0x80]	4	Máscara de bits	—
0x61	Par motor demandado por el conductor	1	$A - 125$	%
0x62	Par motor real	1	$A - 125$	%
0x63	Par motor de referencia	2	$256A + B$	Nm
0x64	Datos de par motor por punto de operación (5 puntos)	5	$X - 125$ (cada byte)	%
0x65	Entradas/salidas auxiliares soportadas	2	Máscara de bits	—
0x66	Sensor MAF dual (masa aire, dos sensores)	5	Complejo (ver J1979)	g/s
0x67	Temperatura del refrigerante (2 sensores)	3	$B - 40; C - 40$	°C
0x68	Sensor de temperatura del aire de admisión	7	Complejo (ver J1979)	°C
0x69	EGR comandado y error de EGR	7	Complejo (ver J1979)	—

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x6A	Control flujo de aire admisión diesel	5	Complejo (ver J1979)	—
0x6B	Temperatura de los gases de recirculación (EGR)	5	Complejo (ver J1979)	°C
0x6C	Actuador del acelerador comandado (posición relativa)	5	Complejo (ver J1979)	%
0x6D	Sistema de control de presión de combustible	6	Complejo (ver J1979)	kPa
0x6E	Sistema de control de presión de inyección	5	Complejo (ver J1979)	kPa
0x6F	Presión a la entrada del compresor del turbo	3	Complejo (ver J1979)	kPa
0x70	Control de sobrealimentación (<i>boost pressure</i>)	9	Complejo (ver J1979)	kPa
0x71	Control de geometría variable del turbo (VGT)	5	Complejo (ver J1979)	%
0x72	Control de la válvula de descarga (<i>wastegate</i>)	5	Complejo (ver J1979)	%
0x73	Presión de los gases de escape	5	Complejo (ver J1979)	kPa
0x74	Régimen del turbocompresor	5	$256A + B$ (por turbo)	rpm
0x75	Temperatura del turbocompresor (turbo 1)	7	$(256A + B)/10 - 40$	°C
0x76	Temperatura del turbocompresor (turbo 2)	7	Ídem 0x75	°C
0x77	Temperatura del intercooler (CACT)	5	$(256A + B)/10 - 40$	°C
0x78	Temperatura de gases de escape EGT (banco 1)	9	$(256A + B)/10 - 40$	°C
0x79	Temperatura de gases de escape EGT (banco 2)	9	Ídem 0x78	°C
0x7A	Presión diferencial del filtro de partículas (DPF)	7	Complejo (ver J1979)	kPa
0x7B	Filtro de partículas DPF (presión y temperatura)	7	Complejo (ver J1979)	—
0x7C	Temperatura del filtro de partículas DPF	9	$(256A + B)/10 - 40$	°C
0x7D	Estado de zona de control NO _x NTE	1	Máscara de bits	—
0x7E	Estado de zona de control PM NTE	1	Máscara de bits	—
0x7F	Tiempo de marcha del motor (formato extendido)	13	Complejo (ver J1979)	s

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0x80	PIDs soportados [0x81–0xA0]	4	Máscara de bits	—
0x81	Tiempo de marcha del motor AECD (ciclo 1)	12	Complejo (ver J1979)	s
0x82	Tiempo de marcha del motor AECD (ciclo 2)	12	Complejo (ver J1979)	s
0x83	Sensor de NOx	5	Complejo (ver J1979)	ppm
0x84	Temperatura superficial del colector	1	$A - 40$	°C
0x85	Sistema de advertencia e inducción de NOx	12	Máscara de bits	—
0x86–0x97	Reservados (no definidos en J1979)	—	—	—
0x98	Sensor de temperatura EGT (conjunto 1)	9	Complejo (ver J1979)	°C
0x99	Sensor de temperatura EGT (conjunto 2)	9	Complejo (ver J1979)	°C
0x9A	Datos del sistema híbrido/eléctrico	6	Complejo (ver J1979)	—
0x9B	Datos del sensor de fluido de escape diésel (DEF)	4	Complejo (ver J1979)	—
0x9C	Datos del sensor de O2	17	Complejo (ver J1979)	—
0x9D	Caudal de combustible del motor (multipunto)	9	Complejo (ver J1979)	L/h
0x9E	Caudal másico de gases de escape	2	$256A + B$	kg/h
0x9F	Uso porcentual del sistema de combustible	9	Complejo (ver J1979)	%
0xA0	PIDs soportados [0xA1–0xC0]	4	Máscara de bits	—
0xA1	Datos corregidos del sensor NOx	5	Complejo (ver J1979)	ppm
0xA2	Caudal de combustible por cilindro	2	$(256A + B)/512$	mg/ciclo
0xA3	Transmisión de vehículo eléctrico	9	Complejo (ver J1979)	—
0xA4	Marcha real de la transmisión	4	Complejo (ver J1979)	—
0xA5	Dosificación de fluido de escape diésel	4	Complejo (ver J1979)	%
0xA6	Odómetro	4	$(2^{24}A + 2^{16}B + 256C + D)/10$	km
0xA7–0xAF	Reservados (no definidos en J1979)	—	—	—

Continúa en la siguiente página...

PID	Descripción	B	Fórmula	Unidad
0xC0	PIDs soportados [0xC1–0xE0]	4	Máscara de bits	—

Anexo B

Catálogo de comandos del protocolo JSON sobre NUS

La siguiente tabla recoge el conjunto completo de comandos del protocolo NDJSON descrito en la Sección 2.1.5. El campo **cmd** identifica la operación; los parámetros opcionales se incluyen en el mismo objeto JSON de petición, y la columna **data** indica el contenido del campo de datos de la respuesta positiva.

Cuadro B.1: Comandos del protocolo JSON sobre NUS.

cmd	Parámetros	data en respuesta
auth	token: str	"authenticated"
ping	-	"pong"
probe_pids	-	[int] - PIDs soportados por la ECU
snapshot	-	{pid: {value, unit}} - lectura puntual de todos los PIDs activos
monitor_start	pids: [int], interval_ms: int	"monitor started"; las muestras se envían como <i>push</i> {type:"samples"}
monitor_stop	-	"monitor stopped"
dtcs	-	[{code, description}]
clear_dtcs	-	null
vin	-	str - cadena VIN de 17 caracteres ASCII
uds_session	session_type: int	{session_type, p2_server_ms, p2_extended_ms}
uds_read_did	did: str (hex, ej. "0xF190")	{did, name, value, unit}
sessions	limit: int	[{session_id, label, started_at, ended_at, sample_count}]
session_samples	session_id, pid?, limit?	[{pid, name, value, unit, ts}]
session_commands	session_id: int	[{command, request_hex, response_hex, timestamp}]
session_dtcs	session_id: int	[{code, description, raw}]
disconnect	-	-

Bibliografía

- [1] Ovz. Car Scanner ELM OBD2 — Aplicación de diagnóstico OBD-II. <https://apps.apple.com/es/app/car-scanner-elm-obd2/id1259933623>, 2024. Aplicación móvil OBD-II comercial usada como referencia de validación. Accedido: 2026-06-05.
- [2] Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical coding sublayer. Standard ISO 11898-1:2024, International Organization for Standardization, Geneva, Switzerland, 2024. URL <https://www.iso.org/standard/86384.html>.
- [3] Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 2: Transport protocol and network layer services. Standard ISO 15765-2:2016, International Organization for Standardization, Geneva, Switzerland, 2016. URL <https://www.iso.org/standard/66574.html>.
- [4] E/E Diagnostic Test Modes. Standard SAE J1979, SAE International, Warrendale, PA, USA, 2017. URL https://www.sae.org/standards/j1979_201702-e-e-diagnostic-test-modes/.
- [5] Road vehicles — Unified diagnostic services (UDS) — Part 1: Application layer. Standard ISO 14229-1:2020, International Organization for Standardization, Geneva, Switzerland, 2020. URL <https://www.iso.org/standard/72439.html>.
- [6] CSS Electronics. OBD2 Explained — A Simple Intro. <https://www.csselectronics.com/pages/obd2-explained-simple-intro>, 2026. CSS Electronics. Referencia técnica de ingeniería industrial. Accedido: 2026-05-23.
- [7] Road vehicles — Interchange of digital information — Controller area network (CAN) for high-speed communication. Standard ISO 11898:1993, International Organization for Standardization, Geneva, Switzerland, 1993. URL <https://www.iso.org/standard/20380.html>.
- [8] CAN in Automation (CiA). History of CAN Technology. <https://www.can-cia.org/can-knowledge/history-of-can-technology>, 2024. CAN in Automation e.V. Organismo normativo oficial del bus CAN. Accedido: 2026-05-23.
- [9] Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 4: Requirements for emissions-related systems. Standard ISO 15765-4:2021,

- International Organization for Standardization, Geneva, Switzerland, 2021. URL <https://www.iso.org/standard/78384.html>.
- [10] Microchip Technology Inc. *MCP2515 Stand-Alone CAN Controller with SPI Interface — Data Sheet*. Microchip Technology Inc., 2021. URL <https://www.microchip.com/en-us/product/mcp2515>. Accedido: 2026-05-23.
- [11] Waveshare Electronics. *RS485 CAN HAT for Raspberry Pi — User Manual*. Waveshare Electronics, 2023. URL https://www.waveshare.com/wiki/RS485_CAN_HAT. Accedido: 2026-05-23.
- [12] Road vehicles — Controller area network (CAN) — Part 2: High-speed physical medium attachment (PMA) sublayer. Standard ISO 11898-2:2024, International Organization for Standardization, Geneva, Switzerland, 2024. URL <https://www.iso.org/standard/85120.html>.
- [13] The Linux Kernel Organization. ISO 15765-2 (ISO-TP) — The Linux Kernel Documentation. <https://docs.kernel.org/networking/iso15765-2.html>, 2024. Documentación oficial del kernel Linux para el socket CAN ISO-TP. Accedido: 2026-05-23.
- [14] California Air Resources Board. On-Board Diagnostic II (OBD-II) Systems Fact Sheet. <https://ww2.arb.ca.gov/resources/fact-sheets/board-diagnostic-ii-obd-ii-systems-fact-sheet>, 2023. California Air Resources Board (CARB). Accedido: 2026-05-23.
- [15] Diagnostic Trouble Code Definitions. Standard SAE J2012, SAE International, Warrendale, PA, USA, 2016. URL https://www.sae.org/standards/j2012_201612-diagnostic-trouble-code-definitions/.
- [16] Waleed Judah. DTC Database — Diagnostic Trouble Code Reference. <https://github.com/Wal33D/dtc-database>, 2024. Base de datos de referencia de códigos DTC para diagnóstico OBD-II. Accedido: 2026-05-27.
- [17] Bluetooth SIG. Bluetooth Core Specification, Version 5.4. Technical report, Bluetooth Special Interest Group, 2023. Generic Attribute Profile (GATT). Accedido: 2026-05-23.
- [18] Nordic Semiconductor ASA. Nordic UART Service (NUS) — nRF Connect SDK Documentation. <https://docs.nordicsemi.com/bundle/ncs-3.0.2/page/nrf/libraries/bluetooth/services/nus.html>, 2024. Nordic Semiconductor ASA. Accedido: 2026-05-23.
- [19] Sandeep Mistry. Arduino CAN — An Arduino library for sending and receiving data using CAN bus. <https://github.com/sandeepmistry/arduino-CAN>, 2023. Abstracción del controlador MCP2515 vía SPI para Arduino. Accedido: 2026-05-23.
- [20] python-can contributors. python-can — Controller Area Network interface module for Python. <https://python-can.readthedocs.io/>, 2024. Librería de acceso al bus CAN sobre SocketCAN. Accedido: 2026-05-23.

- [21] Pier-Yves Lessard. `can-isotp` — ISO-TP (ISO 15765-2) support for python-can. <https://can-isotp.readthedocs.io/>, 2024. Implementación del protocolo de transporte ISO-TP en Python. Accedido: 2026-05-23.
- [22] Kevin Davis and contributors. `ble` — Bluetooth Low Energy Server Supplement for Python. <https://github.com/kevincar/ble>, 2024. Servidor GATT BLE multiplataforma sobre BlueZ/asyncio. Accedido: 2026-05-23.
- [23] Expo. Expo — Framework and platform for universal React applications. <https://docs.expo.dev/>, 2024. Plataforma de desarrollo para React Native. Accedido: 2026-05-23.
- [24] Dotintent. `react-native-ble-plx` — BLE library for React Native. <https://github.com/dotintent/react-native-ble-plx>, 2024. Cliente BLE nativo para React Native (Android/iOS). Accedido: 2026-05-23.
- [25] Poimandres. Zustand — A small, fast and scalable state management solution. <https://github.com/pmndrs/zustand>, 2024. Gestión de estado para React. Accedido: 2026-05-23.
- [26] KONWEII. KONWEII — Escáner de diagnóstico OBD-II. <https://es.aliexpress.com/item/1005007493967100.html>, 2024. Lector OBD-II comercial genérico usado como referencia de validación. Accedido: 2026-06-05.